

소스코드

총 7,392줄

주석을 포함한 전체 소스다. 이 프로젝트는 '왜 그렇게 했는지'를 주석에 남기는 방식으로 작성되어, 코드만으로도 판단 근거를 따라갈 수 있다.

애플리케이션

nfinity/app/anomaly_model.py

286줄

```
1 """
2 app/anomaly_model.py - Day 4 작업: Isolation Forest 이상 탐지 모델 (은경 담당)
3
4 이 모듈이 하는 일: 룰 엔진(app/rule_engine.py)의 R001~R007은 "미리 정해진 규칙"만 잡습니다.
5 정답지(data/mock_transactions_answer_key.csv)를 까보면 이상 거래 263건 중 36건
6 (R00x_위치급변 21건 + R00y_신규기기 15건)은 애초에 규칙으로 설계되지 않은 유형이라 룰
7 엔진으로는 절대 못 잡습니다 - 이 두 유형을 잡는 게 이 모델의 목표입니다.
8
9 사용법:
10 1) 학습 (최초 1회, 또는 데이터가 바뀌면 다시):
11     python -m app.anomaly_model
12     data/mock_transactions.csv 전체로 학습해서 models/anomaly_iforest.pkl로 저장하고,
13     정답지 대비 이 모델 단독 성능(이 두 유형을 얼마나 잡는지)을 바로 출력합니다.
14 2) 추론 (app/routers/risk.py가 자동으로 사용 - 직접 호출할 일은 거의 없음):
15     from app.anomaly_model import score_anomaly
16     anomaly_prob = score_anomaly(txn, profile, history) # 0.0(정상) ~ 1.0(이상)
17
18 ★ 모델 파일(models/anomaly_iforest.pkl)이 아직 없으면 score_anomaly()는 항상 0.0을 돌려줘서
19 룰 엔진만 동작하던 기존 동작을 그대로 유지합니다 (학습 전이라고 서버가 죽지 않음).
20
21 피쳐 4개 - 전부 "그 거래 시점까지의 과거 이력만" 사용합니다(미래 정보 누출 없음):
22 - amount_zscore      유저 평균 대비 이번 결제가 몇 표준편차 떨어져 있는지
23 - is_new_device      이 device_id를 이 유저가 이전에 쓴 적 있는지 → R00y_신규기기 타깃
24 - distance_from_prev_km  직전 결제 위치에서 얼마나 떨어져 있는지 → R00x_위치급변 타깃
25                      (R004 룰은 "물리적으로 불가능한 속도"만 잡는데, 그 정도까진
26                      아니어도 "갑자기 먼 곳"인 패턴은 이 피쳐로 IF가 잡아냅니다)
27 - merchant_rarity    이 가맹점을 이 유저가 그동안 얼마나 드물게 갔는지 (0~1, 1에
28                      가까울수록 처음 보는 가맹점)
29
30 ※ 처음엔 hour_sin/cos, dow_sin/cos(결제 시각/요일)도 넣어서 8개 피쳐로 만들었는데,
31 실제로 정답지 대비 검증해보니 이 두 피쳐가 오히려 신호를 희석시켰습니다 - IsolationForest는
32 각 분기(split)마다 피쳐를 무작위로 하나씩 골라 쓰는 방식이라, 의미 없는 피쳐가 섞여있으면
33 진짜 신호가 있는 피쳐(is_new_device, distance_from_prev_km)가 뽐힐 확률이 그만큼 줄어듭니다.
34 8개 피쳐일 때: 타깃 유형 탐지 6/36건 → 4개로 줄이니 20/36건으로 항상 (아래 self-evaluate
35 결과 참고). 그래서 시간대 관련 이상 패턴은 이미 룰 엔진의 R005(심야 대액)가 담당하는 걸로
36 역할을 나누고, 여기서 룰 엔진이 원천적으로 못 잡는 두 유형에 집중하도록 피쳐를 줄였습니다.
37 """
38 from __future__ import annotations
39
40 import pickle
41 from datetime import datetime
42 from math import radians, sin, cos, sqrt, atan2
43 from pathlib import Path
44 from typing import Optional
45
46 import numpy as np
47 import pandas as pd
48 from sklearn.ensemble import IsolationForest
49 from sklearn.preprocessing import StandardScaler
50
51 from app.models import Transaction, UserRiskProfile
52
53 DATA_DIR = Path(__file__).resolve().parent.parent / "data"
54 MODEL_DIR = Path(__file__).resolve().parent.parent / "models"
55 MODEL_PATH = MODEL_DIR / "anomaly_iforest.pkl"
56
57 FEATURE_NAMES = [
58     "amount_zscore",
59     "is_new_device",
60     "distance_from_prev_km",
```

```

61     "merchant_rarity",
62 ]
63
64 # 이 확률 이상이면, 걸린 룰이 하나도 없어도 "AI 이상탐지"로 risk_events에 별도 기록합니다.
65 # 0.6 기준 자체 검증: 타겟 유형(R00x/R00y) 36건 중 20건 탐지, 정상 거래 오탐율 1.0%.
66 ANOMALY_ALERT_THRESHOLD = 0.6
67 ANOMALY_HIGH_THRESHOLD = 0.8 # 이 이상이면 HIGH로, 아니면 MEDIUM으로 기록
68
69
70 def _haversine_km(lat1, lon1, lat2, lon2) -> float:
71     R = 6371.0
72     dlat = radians(lat2 - lat1)
73     dlon = radians(lon2 - lon1)
74     a = sin(dlat / 2) ** 2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon / 2) ** 2
75     return R * 2 * atan2(sqrt(a), sqrt(1 - a))
76
77
78 # =====
79 # 1. 추론 시점 피쳐 계산 (실시간 API에서 사용)
80 # =====
81
82
83 def compute_features(
84     txn: Transaction, profile: UserRiskProfile, history: list[Transaction]
85 ) -> np.ndarray:
86     """history는 이 거래보다 "이전"인, 같은 유저의 거래 목록(시간순 정렬)이어야 합니다."""
87     std = profile.std_transaction_amount or (profile.avg_transaction_amount * 0.3) or 1.0
88     amount_zscore = (txn.amount - profile.avg_transaction_amount) / (std + 1e-6)
89
90     seen_devices = {t.device_id for t in history if t.device_id}
91     is_new_device = 0.0 if (not txn.device_id or txn.device_id in seen_devices) else 1.0
92
93     distance_from_prev = 0.0
94     if history and history[-1].latitude is not None and txn.latitude is not None:
95         last = history[-1]
96         distance_from_prev = _haversine_km(last.latitude, last.longitude, txn.latitude, txn.longitude)
97
98     merchant_hits = sum(1 for t in history if t.merchant_name == txn.merchant_name)
99     merchant_rarity = 1.0 / (1.0 + merchant_hits)
100
101     return np.array(
102         [[amount_zscore, is_new_device, distance_from_prev, merchant_rarity]],
103         dtype=float,
104     )
105
106
107 # =====
108 # 2. 학습용 피쳐 계산 (CSV 전체를 시간순으로 훑으면서, 미래 정보 누출 없이 계산)
109 # =====
110
111
112 def _build_training_dataframe(
113     transactions_csv: Optional[str] = None, persona_csv: Optional[str] = None
114 ) -> tuple[pd.DataFrame, pd.Series]:
115     transactions_csv = transactions_csv or str(DATA_DIR / "mock_transactions.csv")
116     df = pd.read_csv(transactions_csv)
117     df["timestamp"] = pd.to_datetime(df["timestamp"])
118     df = df.sort_values(["user_id", "timestamp"]).reset_index(drop=True)
119
120     rows = []
121     for user_id, g in df.groupby("user_id", sort=False):
122         g = g.sort_values("timestamp")
123         amounts = g["amount"].to_numpy(dtype=float)
124         # 학습 시점엔 user_risk_profiles가 아직 없을 수 있으니, 그 유저의 전체 이력으로
125         # 평균/표준편차를 근사합니다 (scripts/build_user_profiles.py와 같은 방식).
126         avg_amt = float(amounts.mean())
127         std_amt = float(amounts.std(ddof=0)) or (avg_amt * 0.3) or 1.0
128

```

```

129     seen_devices: set[str] = set()
130     merchant_counts: dict[str, int] = {}
131     prev_lat = prev_lon = None
132
133     for _, r in g.iterrows():
134         amount_zscore = (r["amount"] - avg_amt) / (std_amt + 1e-6)
135
136         device_id = r.get("device_id")
137         is_new_device = 0.0 if (pd.isna(device_id) or device_id in seen_devices) else 1.0
138
139         distance_from_prev = 0.0
140         if prev_lat is not None and not pd.isna(r["latitude"]):
141             distance_from_prev = _haversine_km(prev_lat, prev_lon, r["latitude"], r["longitude"])
142
143         merchant = r["merchant_name"]
144         hits = merchant_counts.get(merchant, 0)
145         merchant_rarity = 1.0 / (1.0 + hits)
146
147         rows.append(
148             {
149                 "row_no": r["row_no"],
150                 "user_id": user_id,
151                 "amount_zscore": amount_zscore,
152                 "is_new_device": is_new_device,
153                 "distance_from_prev_km": distance_from_prev,
154                 "merchant_rarity": merchant_rarity,
155             }
156         )
157
158         # 이번 거래를 이력에 반영 (다음 거래부터는 "과거"가 됨 - 미래 정보 누출 방지)
159         if device_id and not pd.isna(device_id):
160             seen_devices.add(device_id)
161         merchant_counts[merchant] = hits + 1
162         if not pd.isna(r["latitude"]):
163             prev_lat, prev_lon = r["latitude"], r["longitude"]
164
165     feat_df = pd.DataFrame(rows)
166     return feat_df, feat_df["row_no"]
167
168
169 # =====
170 # 3. 학습 & 저장
171 # =====
172
173
174 def train(transactions_csv: Optional[str] = None, contamination: float = 0.06) -> dict:
175     print("1. 학습용 피쳐 계산 중 (유저별 시간순 진행, 미래 정보 누출 없이)...")
176     feat_df, row_nos = _build_training_dataframe(transactions_csv)
177     X = feat_df[FEATURE_NAMES].to_numpy(dtype=float)
178
179     print(f"2. IsolationForest 학습 중... (샘플 {len(X)}건, contamination={contamination})")
180     scaler = StandardScaler()
181     X_scaled = scaler.fit_transform(X)
182
183     model = IsolationForest(
184         n_estimators=200, contamination=contamination, random_state=42, n_jobs=-1
185     )
186     model.fit(X_scaled)
187
188     raw_scores = -model.score_samples(X_scaled) # 높을수록 이상
189     train_min, train_max = float(raw_scores.min()), float(raw_scores.max())
190
191     MODEL_DIR.mkdir(exist_ok=True)
192     with open(MODEL_PATH, "wb") as f:
193         pickle.dump(
194             {
195                 "model": model,
196                 "scaler": scaler,

```

```

197         "train_min": train_min,
198         "train_max": train_max,
199         "feature_names": FEATURE_NAMES,
200         "trained_at": datetime.utcnow().isoformat(),
201     },
202     f,
203 )
204 print(f"✅ 저장 완료: {MODEL_PATH}")
205
206 return {"feat_df": feat_df, "row_nos": row_nos, "raw_scores": raw_scores,
207        "train_min": train_min, "train_max": train_max}
208
209
210 # =====
211 # 4. 추론 (risk.py에서 사용)
212 # =====
213
214 _cached_bundle: Optional[dict] = None
215 _cache_checked = False
216
217
218 def _load_bundle() -> Optional[dict]:
219     global _cached_bundle, _cache_checked
220     if _cache_checked:
221         return _cached_bundle
222     _cache_checked = True
223     if MODEL_PATH.exists():
224         with open(MODEL_PATH, "rb") as f:
225             _cached_bundle = pickle.load(f)
226     return _cached_bundle
227
228
229 def score_anomaly(txn: Transaction, profile: UserRiskProfile, history: list[Transaction]) -> float:
230     """0.0(정상) ~ 1.0(이상). 모델이 아직 학습 전이면 항상 0.0 - 룰 엔진만 있던 기존 동작 유지."""
231     bundle = _load_bundle()
232     if bundle is None:
233         return 0.0
234
235     X = compute_features(txn, profile, history)
236     X_scaled = bundle["scaler"].transform(X)
237     raw = float(-bundle["model"].score_samples(X_scaled)[0])
238
239     train_min, train_max = bundle["train_min"], bundle["train_max"]
240     span = (train_max - train_min) or 1.0
241     return float(np.clip((raw - train_min) / span, 0.0, 1.0))
242
243
244 # =====
245 # 5. 자체 검증: 정답지 대비 "이 모델 단독" 성능 (학습 직후 바로 출력)
246 # =====
247
248
249 def _self_evaluate(feat_df: pd.DataFrame, raw_scores: np.ndarray, train_min: float, train_max: float):
250     answer_path = DATA_DIR / "mock_transactions_answer_key.csv"
251     if not answer_path.exists():
252         print("(정답지가 없어 자체 검증은 건너뛰니다)")
253         return
254
255     answers = pd.read_csv(answer_path)
256     span = (train_max - train_min) or 1.0
257     anomaly_prob = np.clip((raw_scores - train_min) / span, 0.0, 1.0)
258
259     merged = feat_df[["row_no"]].copy()
260     merged["anomaly_prob"] = anomaly_prob
261     merged = merged.merge(answers, on="row_no", how="left")
262
263     target_types = {"R00x_위치급변", "R00y_신규기기"}
264     is_target = merged["anomaly_reason"].isin(target_types)

```

```

265     flagged = merged["anomaly_prob"] >= ANOMALY_ALERT_THRESHOLD
266
267     tp = int((is_target & flagged).sum())
268     fn = int((is_target & ~flagged).sum())
269     fp = int(((merged["anomaly_reason"] == "정상") & flagged).sum())
270     normal_total = int((merged["anomaly_reason"] == "정상").sum())
271
272     print("\n=== 자체 검증: IsolationForest 단독 (룰 엔진 없이) - 임계값 "
273           f"{ANOMALY_ALERT_THRESHOLD} 기준 ===")
274     print(f"타겟 유형(R00x_위치급변 + R00y_신규기기) 총 {int(is_target.sum())}건 중 "
275           f"{tp}건 탐지 (놓친 것 {fn}건)")
276     print(f"정상 거래 {normal_total}건 중 {fp}건을 이상으로 오탐 "
277           f"(오탐율 {fp / normal_total * 100:.1f}%)")
278     print("→ 이 모델이 잡아내는 건 위 두 유형(원래 룰 엔진이 아예 설계 안 된 것)이 메인 타겟이고,")
279     print(" R001/R005/R006/R007처럼 이미 룰이 잘 잡는 유형까지 억지로 잡을 필요는 없습니다 -")
280     print(" 최종 점수는 app/routers/risk.py에서 룰 엔진 결과와 함께 앙상블됩니다.")
281
282
283 if __name__ == "__main__":
284     result = train()
285     _self_evaluate(result["feat_df"], result["raw_scores"], result["train_min"], result["train_max"])
286

```

```

1  """
2  Risk Shield / Dynamic Parking 계산 엔진 (9/2 세 번째 업데이트)
3
4  기획서(2026 금융 AI Challenge 기획서 1차) 4번·5·3번이 설명하는 두 핵심 기능을
5  실제 계산 로직으로 옮긴 파일입니다.
6
7  - Risk Shield: "월별 합계가 아닌 일별 예상 잔액, 최저잔액, 다음 현금 부족 예상일"
8  - Dynamic Parking: "수입 변동성과 납부 예정액을 반영한 세금·건보료 준비 권장액"
9
10 ★ 중요: 이 앱은 실제 계좌 잔액을 갖고 있지 않습니다(오픈뱅킹 연동은 MVP 범위 밖).
11 대신 이미 있는 두 데이터(수입: income_events, 지출: transactions)의 누적 차이로
12 "가상 잔액"을 근사합니다 – 실제 계좌잔액이 아니라 이 데모 안에서의 추정치라는 점을
13 API 응답과 화면 양쪽에 항상 명시합니다(app/security_headers.py처럼 이 파일도 안전장치
14 목적이 큼니다).
15
16 세금·건보료 준비율(TAX_RESERVE_RATE, INSURANCE_RESERVE_RATE)은 실제 세법·건강보험료
17 산정식이 아니라, 프리랜서에게 흔히 권장되는 "수입의 일정 비율을 미리 떼어놓기" 관행을
18 단순화한 예시 비율입니다. 기획서 5-2/8번이 요구하는 대로 "확정 세액을 가장하지
19 않는다"는 원칙을 지키기 위해, 화면에는 반드시 "간이 추정치, 실제 세액/보험료 아님"
20 문구를 같이 보여줘야 합니다.
21 """
22 from __future__ import annotations
23
24 from dataclasses import dataclass, field
25 from datetime import date, datetime, timedelta
26 from typing import Optional
27
28 import logging
29
30 import pandas as pd
31 from sqlalchemy import text
32 from sqlalchemy.orm import Session
33
34 from data_pipeline.recurring_detector import detect_recurring_payments
35
36 # -----
37 # 0. 설정값 (전부 "간이 추정치"라는 전제 – 코드 상단 docstring 참고)
38 # -----
39 logger = logging.getLogger("nfinity.cashflow")
40
41 TAX_RESERVE_RATE = 0.08 # 종합소득세 예비율(간이 추정)
42 INSURANCE_RESERVE_RATE = 0.07 # 지역가입자 건강보험료 예비율(간이 추정)
43 DEFAULT_MIN_SAFETY_BALANCE = 300_000 # 사용자가 직접 설정 안 했을 때 기본 최소 안전잔액
44 SIMULATION_HORIZON_DAYS = 45
45 INCOME_LOOKBACK_DAYS = 90
46
47
48 @dataclass
49 class DailyBalancePoint:
50     day: date
51     balance: float
52     events: list[str] = field(default_factory=list) # 그 날 반영된 사건(설명가능성용)
53
54
55 def _load_income_events(db: Session, user_id: str) -> pd.DataFrame:
56     """연결된(connected=true) 플랫폼의 수입 이벤트만 불러옵니다. 연결 안 한 플랫폼은
57     사용자가 아직 앱에 알려주지 않은 수입이라, 실제 오픈뱅킹 연동과 마찬가지로
58     Risk Shield 계산에서도 제외합니다 – '연결할수록 예측이 정확해진다'는 동기부여와
59     맞닿아 있습니다."""
60     rows = db.execute(
61         text(
62             "SELECT ie.source_id, s.platform_name, ie.amount, ie.status, ie.settled_at "
63             "FROM income_events ie JOIN income_sources s ON s.source_id = ie.source_id "

```

```

64         "WHERE ie.user_id = :uid AND s.connected = TRUE "
65         "ORDER BY ie.settled_at"
66     ),
67     {"uid": user_id},
68 ).mappings().all()
69 return pd.DataFrame(
70     [
71         {
72             "source_id": str(r["source_id"]),
73             "platform_name": r["platform_name"],
74             "amount": float(r["amount"]),
75             "status": r["status"],
76             "settled_at": r["settled_at"],
77         }
78         for r in rows
79     ]
80 )
81
82
83 def _load_transactions(db: Session, user_id: str) -> pd.DataFrame:
84     rows = db.execute(
85         text(
86             "SELECT merchant_name, amount, timestamp FROM transactions "
87             "WHERE user_id = :uid ORDER BY timestamp"
88         ),
89         {"uid": user_id},
90     ).mappings().all()
91     return pd.DataFrame(
92         [{"merchant_name": r["merchant_name"], "amount": float(r["amount"]), "timestamp": r["timestamp"]} for r in r
ows]
93     )
94
95
96 def _get_min_safety_balance(db: Session, user_id: str) -> float:
97     row = db.execute(
98         text("SELECT min_safety_balance FROM user_settings WHERE user_id = :uid"),
99         {"uid": user_id},
100     ).mappings().first()
101     if row and row["min_safety_balance"] is not None:
102         return float(row["min_safety_balance"])
103     return float(DEFAULT_MIN_SAFETY_BALANCE)
104
105
106 def _get_current_balance(db: Session, user_id: str, income_df: pd.DataFrame, tx_df: pd.DataFrame) -> tuple[float, st
r]:
107     """시뮬레이션의 시작 잔액과 그 출처를 함께 돌려줍니다.
108
109     9/5 수정 - 이전에는 무조건 `연결된 플랫폼의 정산완료 수입 합계 - 전체 지출 합계`로
110     잔액을 유도했습니다. 그런데 이 앱은 (설계상) 사용자가 '연결'한 플랫폼의 수입만
111     집계하는 반면 지출은 카드 거래 전부를 집계하기 때문에, 미연결 플랫폼이 하나라도
112     있으면 그만큼 수입이 통째로 빠진 채 지출만 온전히 반영되어 잔액이 구조적으로
113     음수 쪽으로 치우칩니다. 실제로 데모 페르소나 5명 중 3명이 마이너스 수백만원으로
114     나왔고, 그 결과 45일 시뮬레이션이 전부 "1일 후 잔고 부족"만 반환해서 예측 기능이
115     사실상 죽어 있었습니다.
116
117     오픈뱅킹 연동이 없는 이 MVP에서 정직한 모델은 "사용자가 알려준 현재 잔액"을
118     기준으로 삼는 것입니다(실서비스라면 마이데이터로 채울 자리). 설정값이 없으면
119     수입·지출을 같은 기간(최근 30일)으로 맞춰 순현금흐름을 추정하되, 위와 같은 이유로
120     음수가 될 수 있어 0 밑으로는 내려가지 않게 막고 출처를 'estimated'로 표시합니다.
121     """
122     row = db.execute(
123         text("SELECT current_balance FROM user_settings WHERE user_id = :uid"),
124         {"uid": user_id},
125     ).mappings().first()
126     if row and row["current_balance"] is not None:
127         return float(row["current_balance"]), "user_set"
128
129     from app.demo_clock import get_demo_now

```

```

130
131 demo_now = get_demo_now(db)
132 window_start = demo_now - timedelta(days=30)
133 income_30d = 0.0
134 if not income_df.empty:
135     settled = income_df[income_df["status"] == "정산완료"]
136     income_30d = float(settled[settled["settled_at"] >= window_start]["amount"].sum())
137 spend_30d = float(tx_df[tx_df["timestamp"] >= window_start]["amount"].sum()) if not tx_df.empty else 0.0
138 return max(0.0, income_30d - spend_30d), "estimated"
139
140
141 def _project_future_income(income_df: pd.DataFrame, demo_now: datetime, horizon_end: datetime) -> list[dict]:
142     """소스(플랫폼)별로 과거 '정산완료' 이벤트의 간격·금액 패턴을 보고, 이미 알고 있는
143     '정산예정' 건 이후 구간까지 미래 수입을 이어서 추정합니다.
144
145     예: 배달의민족이 지난 넉 달간 7일 간격으로 평균 27만원씩 들어왔다면, 앞으로도
146     7일마다 27만원이 들어올 거라고 가정합니다. 실제 오픈뱅킹이라면 예정된 정산
147     스케줄을 그대로 가져오겠지만, 이 데모는 과거 패턴으로 근사합니다."""
148     projected: list[dict] = []
149     if income_df.empty:
150         return projected
151
152     settled = income_df[income_df["status"] == "정산완료"].copy()
153     for source_id, grp in settled.groupby("source_id"):
154         grp = grp.sort_values("settled_at")
155         platform_name = grp["platform_name"].iloc[0]
156         if len(grp) >= 2:
157             intervals = grp["settled_at"].diff().dt.total_seconds().dropna() / 86400
158             avg_interval_days = max(1, round(float(intervals.median())))
159         else:
160             avg_interval_days = 30 # 이력이 1건뿐이면 월 1회로 가정(보수적 기본값)
161         avg_amount = float(grp["amount"].tail(4).mean())
162
163         # 이 소스의 '가장 마지막으로 알고 있는' 이벤트(정산완료든 정산예정이든) 이후부터 투사
164         all_known = income_df[income_df["source_id"] == source_id]
165         last_known_date = all_known["settled_at"].max()
166
167         next_date = last_known_date + timedelta(days=avg_interval_days)
168         while next_date <= horizon_end:
169             if next_date > demo_now:
170                 projected.append(
171                     {"date": next_date, "amount": avg_amount, "platform_name": platform_name, "kind": "예상 수입(패
172                     )
173                 next_date += timedelta(days=avg_interval_days)
174         return projected
175
176
177 def _project_recurring_expenses(tx_df: pd.DataFrame, demo_now: datetime, horizon_end: datetime) -> list[dict]:
178     """data_pipeline/recurring_detector.py(이미 budgets/forecast가 쓰는 모듈)로 찾은
179     정기결제를, 감지된 간격으로 시뮬레이션 기간 끝까지 이어서 투사합니다."""
180     projected: list[dict] = []
181     if tx_df.empty:
182         return projected
183     tx_for_detect = tx_df.rename(columns={})
184     tx_for_detect["user_id"] = "u" # detect_recurring_payments는 user_id 컬럼을 기대함(단일 유저라 더미값)
185     recurring_df = detect_recurring_payments(tx_for_detect)
186     if recurring_df.empty:
187         return projected
188
189     for _, r in recurring_df.iterrows():
190         interval = max(7, int(round(r["avg_interval_days"])))
191         next_date = pd.to_datetime(r["next_expected_date"])
192         while next_date <= horizon_end:
193             if next_date > demo_now:
194                 projected.append(
195                     {"date": next_date, "amount": float(r["avg_amount"]), "merchant_name": r["merchant_name"], "kin
196                     d": "정기결제(고정비)"}

```

```

196         )
197         next_date += timedelta(days=interval)
198     return projected
199
200
201 def _income_variability(income_df: pd.DataFrame, demo_now: datetime) -> float:
202     """최근 3개월 '월별 수입 합계'의 변동계수(표준편차/평균). 값이 클수록 수입이
203     들쭉날쭉하다는 뜻이라, 세금/건보료 준비금을 더 넉넉히 잡도록 반영합니다."""
204     settled = income_df[income_df["status"] == "정산완료"].copy()
205     if settled.empty:
206         return 0.0
207     window_start = demo_now - timedelta(days=INCOME_LOOKBACK_DAYS)
208     settled = settled[settled["settled_at"] >= window_start]
209     if settled.empty:
210         return 0.0
211     settled["month"] = settled["settled_at"].dt.to_period("M")
212     monthly = settled.groupby("month")["amount"].sum()
213     if len(monthly) < 2 or monthly.mean() == 0:
214         return 0.0
215     return float(monthly.std(ddof=0) / monthly.mean())
216
217
218 def compute_shield(db: Session, user_id: str) -> dict:
219     """Risk Shield + Dynamic Parking 통합 계산. 기획서 7-2 데모 API 계약의 필드명을
220     그대로 따릅니다(current_balance/tax_reserve/insurance_reserve/available_cash/
221     minimum_balance_date/risk_level/risk_reasons/recommended_reserve)."""
222     from app.demo_clock import get_demo_now
223
224     demo_now = get_demo_now(db)
225     income_df = _load_income_events(db, user_id)
226     tx_df = _load_transactions(db, user_id)
227
228     if income_df.empty and tx_df.empty:
229         return None # 라우터에서 404 처리
230
231     current_balance, balance_source = _get_current_balance(db, user_id, income_df, tx_df)
232
233     # --- Dynamic Parking: 월 평균 수입 기준 간이 준비금 ---
234     window_start = demo_now - timedelta(days=INCOME_LOOKBACK_DAYS)
235     trailing_income = 0.0
236     if not income_df.empty:
237         settled = income_df[income_df["status"] == "정산완료"]
238         trailing_income = float(settled[settled["settled_at"] >= window_start]["amount"].sum())
239     monthly_avg_income = trailing_income / 3 if trailing_income else 0.0
240
241     cv = _income_variability(income_df, demo_now)
242     variability_multiplier = 1 + min(cv, 1.0) * 0.5 # cv 0→1.0배, cv>=1→최대 1.5배
243
244     # 9/5 변경 - 예전에는 월평균 수입에 고정 비율(8%/7%)을 곱했습니다. 그 숫자의 근거가
245     # "프리랜서에게 흔히 권장되는 관행"뿐이라, 실제 제도 계산식(app/tax.py)으로 바꿉니다:
246     # 종합소득세는 단순경비율·누진세율·기납부 3.3%까지 반영한 뒤 12로 나눈 월 적립액,
247     # 건강보험료는 요율 7.19% + 장기요양보험료를 적용한 월 보험료입니다.
248     # 수입 데이터가 없는 유저(연결된 플랫폼이 없음)는 기존 방식으로 조용히 되돌립니다.
249     try:
250         from app.routers.tax import load_annual_income
251         from app.tax import estimate_health_insurance, estimate_income_tax, expense_rate_for
252
253         annual_business, annual_employment, platform_types = load_annual_income(db, user_id)
254         if annual_business > 0 or annual_employment > 0:
255             _tax = estimate_income_tax(annual_business, expense_rate_for(platform_types))
256             _health = estimate_health_insurance(annual_business, has_employment_income=annual_employment > 0)
257             tax_reserve = _tax["monthly_reserve"]
258             insurance_reserve = _health["total_monthly"]
259         else:
260             tax_reserve = round(monthly_avg_income * TAX_RESERVE_RATE)
261             insurance_reserve = round(monthly_avg_income * INSURANCE_RESERVE_RATE)
262     except Exception:
263         logger.exception("[cashflow] 세금·건보료 추정에 실패해 간이 비율로 되돌립니다.")

```

```

264         tax_reserve = round(monthly_avg_income * TAX_RESERVE_RATE)
265         insurance_reserve = round(monthly_avg_income * INSURANCE_RESERVE_RATE)
266         available_cash = current_balance - tax_reserve - insurance_reserve
267
268         # 다음 정산 주기(약 30일) 동안 추가로 더 모아줘야 할 권장액 - 변동성 클수록 더 넉넉히
269         recommended_reserve = round((tax_reserve + insurance_reserve) * variability_multiplier)
270
271         # --- Risk Shield: 일별 시뮬레이션 ---
272         min_safety_balance = _get_min_safety_balance(db, user_id)
273         horizon_end = demo_now + timedelta(days=SIMULATION_HORIZON_DAYS)
274
275         future_income = _project_future_income(income_df, demo_now, horizon_end)
276         future_expenses = _project_recurring_expenses(tx_df, demo_now, horizon_end)
277
278         # 정기결제로 안 잡히는 나머지(변동비)는 최근 지출 속도로 매일 조금씩 빠져나간다고 가정
279         recent_window_days = 60
280         recent_tx = tx_df[tx_df["timestamp"] >= demo_now - timedelta(days=recent_window_days)] if not tx_df.empty else t
x_df
281         recent_total = float(recent_tx["amount"].sum()) if not recent_tx.empty else 0.0
282         recurring_total_recent = 0.0
283         if future_expenses:
284             recurring_merchants = {e["merchant_name"] for e in future_expenses}
285             recurring_total_recent = float(
286                 recent_tx[recent_tx["merchant_name"].isin(recurring_merchants)]["amount"].sum()
287             ) if not recent_tx.empty else 0.0
288         variable_daily_rate = max(0.0, (recent_total - recurring_total_recent)) / recent_window_days
289
290         daily_points: list[DailyBalancePoint] = []
291         balance = current_balance
292         minimum_balance_date: Optional[date] = None
293         running_min = balance
294
295         for offset in range(1, SIMULATION_HORIZON_DAYS + 1):
296             day = (demo_now + timedelta(days=offset)).date()
297             day_start = datetime.combine(day, datetime.min.time())
298             day_end = day_start + timedelta(days=1)
299             events_today = []
300
301             for inc in future_income:
302                 if day_start <= inc["date"] < day_end:
303                     balance += inc["amount"]
304                     events_today.append(f"{inc['platform_name']} 수입 +{inc['amount']:,.0f}원")
305
306             for exp in future_expenses:
307                 if day_start <= exp["date"] < day_end:
308                     balance -= exp["amount"]
309                     events_today.append(f"{exp['merchant_name']} 정기결제 -{exp['amount']:,.0f}원")
310
311             balance -= variable_daily_rate
312
313             daily_points.append(DailyBalancePoint(day=day, balance=round(balance), events=events_today))
314             if balance < running_min:
315                 running_min = balance
316             if balance < min_safety_balance and minimum_balance_date is None:
317                 minimum_balance_date = day
318
319         # --- 위험도 판정 ---
320         risk_reasons: list[str] = []
321         if cv >= 0.4:
322             risk_reasons.append(f"최근 수입 변동성 증가 (변동계수 {cv:.2f})")
323         if not tx_df.empty:
324             fixed_ratio = (recurring_total_recent / recent_total) if recent_total else 0.0
325             if fixed_ratio >= 0.45:
326                 risk_reasons.append(f"고정비 비율 {fixed_ratio*100:.0f}%")
327         if available_cash < recommended_reserve:
328             risk_reasons.append("세금·건보료 준비금 부족")
329         if minimum_balance_date is not None:
330             days_until = (minimum_balance_date - demo_now.date()).days

```

```

331         risk_reasons.append(f"{days_until}일 후 최소 안전잔액 이하로 예상")
332
333     if minimum_balance_date is not None and running_min < 0:
334         risk_level = "위험"
335     elif minimum_balance_date is not None or available_cash < recommended_reserve:
336         risk_level = "주의"
337     else:
338         risk_level = "안전"
339         if not risk_reasons:
340             risk_reasons.append("최근 현금흐름이 안정적이에요")
341
342     return {
343         "user_id": user_id,
344         "current_balance": round(current_balance),
345         "balance_source": balance_source,
346         "tax_reserve": tax_reserve,
347         "insurance_reserve": insurance_reserve,
348         "available_cash": round(available_cash),
349         "minimum_balance_date": minimum_balance_date.isoformat() if minimum_balance_date else None,
350         "risk_level": risk_level,
351         "risk_reasons": risk_reasons,
352         "recommended_reserve": recommended_reserve,
353         "min_safety_balance": min_safety_balance,
354         "income_variability": round(cv, 3),
355         "daily_balances": [
356             {"date": p.day.isoformat(), "balance": p.balance, "events": p.events} for p in daily_points
357         ],
358         "calculated_at": demo_now.isoformat(),
359     }
360
361
362 def compute_scenarios(db: Session, user_id: str) -> Optional[dict]:
363     """기본/임금지연/수입감소 3개 시나리오의 일별 잔액 시계열. compute_shield()와
364     같은 시뮬레이션 로직을 재사용하되, 입력을 살짝 비틀어서 3번 돌립니다."""
365     from app.demo_clock import get_demo_now
366
367     demo_now = get_demo_now(db)
368     income_df = _load_income_events(db, user_id)
369     tx_df = _load_transactions(db, user_id)
370     if income_df.empty and tx_df.empty:
371         return None
372
373     current_balance, _ = _get_current_balance(db, user_id, income_df, tx_df)
374     horizon_end = demo_now + timedelta(days=SIMULATION_HORIZON_DAYS)
375
376     def _simulate(income_delay_days: int, income_scale: float) -> list[dict]:
377         future_income = _project_future_income(income_df, demo_now, horizon_end)
378         for inc in future_income:
379             inc["date"] = inc["date"] + timedelta(days=income_delay_days)
380             inc["amount"] = inc["amount"] * income_scale
381         future_expenses = _project_recurring_expenses(tx_df, demo_now, horizon_end)
382
383         recent_window_days = 60
384         recent_tx = tx_df[tx_df["timestamp"] >= demo_now - timedelta(days=recent_window_days)] if not tx_df.empty else tx_df
385         recent_total = float(recent_tx["amount"].sum()) if not recent_tx.empty else 0.0
386         recurring_merchants = {e["merchant_name"] for e in future_expenses}
387         recurring_total_recent = float(
388             recent_tx[recent_tx["merchant_name"].isin(recurring_merchants)]["amount"].sum()
389         ) if not recent_tx.empty and recurring_merchants else 0.0
390         variable_daily_rate = max(0.0, (recent_total - recurring_total_recent)) / recent_window_days
391
392         balance = current_balance
393         points = []
394         for offset in range(1, SIMULATION_HORIZON_DAYS + 1):
395             day = (demo_now + timedelta(days=offset)).date()
396             day_start = datetime.combine(day, datetime.min.time())
397             day_end = day_start + timedelta(days=1)

```

```

398         for inc in future_income:
399             if day_start <= inc["date"] < day_end:
400                 balance += inc["amount"]
401         for exp in future_expenses:
402             if day_start <= exp["date"] < day_end:
403                 balance -= exp["amount"]
404         balance -= variable_daily_rate
405         points.append({"date": day.isoformat(), "balance": round(balance)})
406     return points
407
408     return {
409         "user_id": user_id,
410         "calculated_at": demo_now.isoformat(),
411         "scenarios": {
412             "기본": _simulate(income_delay_days=0, income_scale=1.0),
413             "입금지연": _simulate(income_delay_days=10, income_scale=1.0),
414             "수입감소": _simulate(income_delay_days=0, income_scale=0.7),
415         },
416     }
417

```

nfinity/app/database.py

38줄

```

1  """
2  DB / Redis 연결 설정.
3  - DATABASE_URL, REDIS_URL은 .env 파일에서 읽어옵니다.
4  - 로컬에서 도커 없이 테스트할 때는 환경변수를 sqlite 등으로 바꿔서 써도 됩니다.
5  """
6  import os
7
8  from dotenv import load_dotenv
9  from sqlalchemy import create_engine
10 from sqlalchemy.orm import declarative_base, sessionmaker
11
12 load_dotenv()
13
14 DATABASE_URL = os.getenv(
15     "DATABASE_URL", "postgresql://sidegig:sidegig_pw@localhost:5432/sidegig"
16 )
17 REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")
18
19 engine = create_engine(DATABASE_URL, pool_pre_ping=True)
20 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
21 Base = declarative_base()
22
23
24 def get_db():
25     """FastAPI Depends()로 주입해서 쓰는 DB 세션 제너레이터."""
26     db = SessionLocal()
27     try:
28         yield db
29     finally:
30         db.close()
31
32
33 def get_redis():
34     """지연 import: redis 서버가 없는 환경(로컬 테스트)에서도 앱이 죽지 않도록."""
35     import redis
36
37     return redis.from_url(REDIS_URL, decode_responses=True)
38

```

```

1  """
2  데모 "현재 시각" 기준점 (9/2 발견한 버그를 고치면서 추가)
3
4  문제: budgets.py/redis_client.py/seed_demo_personas.py가 전부 실제 서버 시각
5  (datetime.utcnow() / date.today())을 "이번 달"의 기준으로 썼습니다. 그런데 이 앱의
6  mock_transactions.csv는 2026-05-25 ~ 2026-08-22 데이터만 있습니다 - 즉 실제 달력이
7  8월을 넘아가는 순간(정확히는 심사 기간인 9/7~9/11 내내), "이번 달 실제로 쓴 돈"을 실서버
8  시각 기준으로 계산하면 이번 달(9월) 거래가 하나도 없으니 무조건 0원이 나옵니다.
9  직접 재현해서 확인함: 9/2에 seed_demo_personas.py를 돌려보니 5명 전원 "이번 달 실지출
10 합계 0원"이 나왔고, 그 상태로는 예산 화면이 전부 0%/미사용으로 보여서 리뷰 때 그렇게
11 공들인 예산통제 기능이 심사 당일엔 텅 빈 것처럼 보이는 심각한 문제였습니다.
12
13 고친 방식: "지금"을 실서버 시각이 아니라 "이 mock 데이터셋에 있는 가장 최근 거래
14 시각"으로 정의합니다(get_demo_now). 데이터가 8/22에 끝나니 데모 기준 "오늘"은 항상
15 8/22이고, 심사위원이 9/7에 열든 9/11에 열든 결과가 똑같이 재현됩니다 - 실서버 시각에
16 좌우되지 않습니다. DB 쿼리 결과는 프로세스 안에서 캐싱합니다(mock 데이터는 안 바뀌므로
17 매 요청마다 다시 조회할 필요가 없습니다).
18 """
19 from __future__ import annotations
20
21 from datetime import datetime
22
23 from sqlalchemy import text
24 from sqlalchemy.orm import Session
25
26 _cached_demo_now: datetime | None = None
27
28
29 def get_demo_now(db: Session) -> datetime:
30     """이 mock 데이터셋 기준 '지금'. transactions 테이블의 가장 최근 timestamp를 씁니다."""
31     global _cached_demo_now
32     if _cached_demo_now is not None:
33         return _cached_demo_now
34
35     row = db.execute(text("SELECT MAX(timestamp) AS latest FROM transactions")).mappings().first()
36     latest = row["latest"] if row else None
37     # 거래가 하나도 없는 극단적인 경우(빈 DB)에는 실서버 시각으로 대체합니다.
38     _cached_demo_now = latest if latest is not None else datetime.utcnow()
39     return _cached_demo_now
40

```

```

1  """
2  데모 계정 접근 제한 (9/5 추가)
3
4  배경
5  ----
6  이 서비스의 API는 조회할 대상을 `user_id`로 받습니다(경로 또는 쿼리 파라미터). 로그인이
7  없는 공개 데모라 지금까지는 누구든 `user_id`만 바꿔서 다른 사람의 예산·리스크·수입
8  데이터를 꺼낼 수 있었습니다 - 전형적인 IDOR입니다. README에도 "의도적으로 남겨둔
9  한계"로 적어뒀지만, 금융보안원이 주최하는 공모전에서 그대로 두기에는 눈에 띄는 구멍입니다.
10
11  무엇을 하나
12  -----
13  이 배포는 시연용이라 공개해야 할 데이터가 시드된 페르소나 3명뿐입니다. 그래서 요청에
14  담긴 `user_id`가 그 3명이 아니면 서버가 403으로 막습니다. mock 데이터에는 이 3명 말고도
15  35명의 시드 유저가 들어있는데, 그 데이터도 이제 외부에서 꺼낼 수 없습니다.
16
17  이게 진짜 인가(authorization)는 아니라는 점은 분명히 해둡니다. 실서비스로 가면 세션
18  로그인을 붙이고 "요청자 본인의 user_id만 조회 가능"을 서버가 강제해야 합니다. 다만 그
19  구조로 가기 전이라도, 공개된 데모가 시드 데이터 전체를 열어두지는 않게 하는 최소한의
20  방어선입니다.
21  """
22  import re
23
24  from starlette.middleware.base import BaseHTTPMiddleware
25  from starlette.responses import JSONResponse
26
27  # 경로에 박힌 UUID를 찾기 위한 패턴 (예: /api/v1/shield/<uuid>/scenarios)
28  _UUID_RE = re.compile(
29      r"[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}"
30  )
31
32  # 경로의 UUID가 user_id가 아닌 라우트들 (여기서는 income_sources.source_id).
33  _NON_USER_ID_PATH_PREFIXES = ("/api/v1/income/sources/",)
34
35  _FORBIDDEN = JSONResponse(
36      status_code=403,
37      content={
38          "error": {
39              "code": 403,
40              "message": "이 시연 배포는 공개된 데모 계정의 데이터만 조회할 수 있습니다.",
41          }
42      },
43  )
44
45
46  def allowed_user_ids() -> set[str]:
47      from app.demo_personas import DEMO_PERSONAS
48
49      return {p["user_id"] for p in DEMO_PERSONAS}
50
51
52  class DemoUserGuardMiddleware(BaseHTTPMiddleware):
53      """/api/v1/* 요청에 담긴 user_id가 데모 페르소나인지 확인합니다.
54
55      경로의 UUID와 쿼리 파라미터 `user_id`를 검사합니다. POST 본문에 user_id를 담은
56      엔드포인트(/risk/assess)는 본문을 여기서 읽으면 라우터가 다시 읽지 못하므로,
57      그쪽은 라우터에서 직접 확인합니다(app/routers/risk.py).
58      """
59
60      async def dispatch(self, request, call_next):
61          path = request.url.path
62          if path.startswith("/api/v1/"):
63              allowed = allowed_user_ids()

```

```

64
65     qs_user = request.query_params.get("user_id")
66     if qs_user and qs_user not in allowed:
67         return _FORBIDDEN
68
69     # 경로에 박힌 UUID가 전부 user_id인 건 아닙니다. /income/sources/{source_id}/connect
70     # 처럼 다른 자원의 id를 경로에 쓰는 라우트가 있어서, 그것까지 user_id로 보고 막으면
71     # 정상 요청이 403으로 죽습니다(실제로 "연결하기" 버튼이 조용히 실패했습니다).
72     # 이런 라우트는 경로 UUID 검사를 건너뛰고 쿼리의 user_id만 봅니다 - 라우터 쿼리가
73     # 이미 `WHERE source_id = :sid AND user_id = :uid`로 소유자까지 함께 확인하므로,
74     # 남의 source_id를 넣어도 자기 데이터가 아니면 404가 납니다.
75     if not any(path.startswith(p) for p in _NON_USER_ID_PATH_PREFIXES):
76         for found in _UUID_RE.findall(path):
77             if found not in allowed:
78                 return _FORBIDDEN
79
80     return await call_next(request)
81

```

```

1  """
2  데모 페르소나 (8/30 추가)
3
4  공모전 MVP는 "본인 데이터를 입력해야 돌아가는 서비스"일 필요가 없습니다 - 데이콘
5  운영진이 게시판 공식 답변에서 "개인화 기능을 가상(더미) 데이터로 구현·시연해도 된다.
6  다만 가상 데이터임을 화면에 안내하는 걸 권장한다"고 확인했습니다. 그래서 로그인/개인정보
7  입력 없이, 이야기가 뚜렷한 시드 페르소나 중 하나를 골라 "그 사람의 계정을 잠깐
8  들여다보는" 데모 UX로 갑니다.
9
10 9/5 업데이트 - 페르소나를 5명에서 3명으로 줄였습니다. 심사위원이 한 팀에 쓰는 시간은
11 길지 않아서, 서로 다른 축을 대표하는 최소 인원만 남기는 편이 낫다고 판단했습니다:
12 김민철(위험 신호가 적은 대조군 겸 이 서비스의 상징적 타깃인 배달 N잡러),
13 박지수(현금흐름 위기 - Risk Shield 주인공, 기본 진입), 이하늘(이상거래 CRITICAL).
14 빠진 최유진·정다운의 시드 데이터가 DB에 남아 있어도 이 목록에 없으면 화면·API 어디에도
15 노출되지 않습니다.
16
17 current_balance는 Risk Shield 시뮬레이션의 시작 잔액입니다(app/cashflow.py 참고).
18 실서비스라면 오픈뱅킹/마이데이터로 채울 자리라, 데모에서는 페르소나 스토리에 맞는
19 값을 시드로 넣습니다.
20
21 user_id는 seed_data.py가 이미 만들어둔 진짜 유저이고, 아래 5명은 story_flag로 캐릭터가
22 뚜렷한 "간판" 인물들입니다. limits는 scripts/seed_demo_personas.py가 budgets 테이블에
23 넣는 카테고리별 한도값이고(지출 자체는 실제 mock_transactions.csv 합계), tagline은
24 프론트 페르소나 선택 화면에 보여줄 한 줄 소개입니다.
25
26 limits 값은 실제 이번 달 지출 합계(scripts/seed_demo_personas.py로 직접 집계해서 확인한
27 값)를 보고 "이 페르소나의 스토리에 맞는 한도"로 역산해서 정한 것입니다 - 처음엔 대충
28 잡았다가, 최유진처럼 실제 지출이 매우 큰 페르소나는 한도도 같이 안 키우면 쇼핑 한 카테고리만
29 5000%대로 튀어서(버그처럼 보임) 다시 조정했습니다. 지출 자체는 절대 손대지 않았고, 한도만
30 그 지출 규모에 맞춰 "정상/위기/초과" 스토리가 실제로 그렇게 보이도록 맞췄습니다.
31
32 이 파일이 "진실의 원천"이라 시딩 스크립트(scripts/seed_demo_personas.py)와 API
33 (app/routers/demo.py)가 둘 다 여기서 가져다 씁니다 - 목록이 두 군데서 따로 관리되며
34 어긋나는 걸 막기 위함입니다.
35
36 story/tagline은 실제 risk_events 결과와 반드시 맞아야 합니다. 전체 38명 시드 유저를
37 룰 엔진 + 이상탐지로 전수 리플레이해본 결과(거래 이력이 있는 유저 기준) 가장 낮은
38 케이스도 HIGH(91점)였고, "위험 신호가 전혀 없는" 유저는 이 mock 데이터셋에 존재하지
39 않았습니다. 그래서 "정상/안정적"이라고 적어둔 페르소나라도 실제로는 MEDIUM 이상의
40 점수가 나올 수 있다는 전제로 tagline을 씁니다 - 김민철은 애초 "특별한 위험 신호 없는"
41 이라 써놔었는데 실제 최신 점수가 68점(MEDIUM)이라 "애매한 결제 한두 건 빼면"으로
42 완화했고, 이하늘은 "안정적인 지출"이라 써놔었는데 실제로는 100점(CRITICAL, 유흥업소/
43 금은장/해외 Steam 고액결제로 R001·R002·R003·R005·R007 룰이 동시에 걸린 케이스)이라
44 야에 반대로 "확인이 필요한 이상거래" 케이스로 스토리를 바꿨습니다. (검증: python3
45 /tmp/risk_survey.py - 저장소 밖 1회성 분석 스크립트, 저장소에는 포함 안 함)
46 """
47
48 DEMO_PERSONAS = [
49     {
50         "user_id": "c5dd4004-498c-4409-9472-7a0e457ff8f1",
51         "name": "김민철",
52         "job": "배달 투잡러",
53         "story": "mostly_normal",
54         "tagline": "본업 + 배달로 버는 N잡러, 애매한 결제 한두 건 빼면 대체로 평범한 한 달",
55         "avatar_emoji": "🍕",
56         "limits": {"식비": 220000, "교통": 180000, "쇼핑": 550000, "업무용": 1000000, "의료/건강": 80000, "기타": 1000
57     },
58     {
59         "user_id": "6d422b5f-1b23-4312-bbfc-4ef97e5fc845",
60         "name": "박지수",
61         "job": "프리랜서 마케터",

```

```

63     "story": "cashflow_crisis",
64     "tagline": "업무용 지출이 몰려 이번 달 자금 흐름이 빠듯해진 프리랜서",
65     "avatar_emoji": "👨‍💻",
66     "limits": {"식비": 180000, "교통": 160000, "쇼핑": 700000, "업무용": 1700000, "의료/건강": 60000, "기타": 4800
00},
67     # 기본 진입 페르소나. 지금은 잔고가 남아있지만 고정비·정기결제가 이어지면서
68     # 시뮬레이션 중반에 최소 안전잔액을 뚫도록 잡은 값입니다("자금은 괜찮은데
69     # N일 후 위험"이라는 Risk Shield의 핵심 메시지가 실제로 보이는 케이스).
70     "current_balance": 1_450_000,
71 },
72 {
73     "user_id": "131b9d96-aa5a-4938-844d-2d08b7a2df9a",
74     "name": "이하늘",
75     "job": "IT 개발자",
76     "story": "fraud_risk",
77     "tagline": "평소와 다른 고액 결제가 잇달아 감지된 개발자, 본인 결제인지 확인이 필요한 케이스",
78     "avatar_emoji": "👨‍💻",
79     "limits": {"식비": 250000, "교통": 100000, "쇼핑": 400000, "업무용": 4000000, "의료/건강": 80000, "기타": 5000
00},
80     "current_balance": 4_800_000,
81 },
82 ]
83
84 CATEGORIES = ["식비", "교통", "쇼핑", "업무용", "의료/건강", "기타"]
85 PERIOD = "monthly"
86
87
88 def get_persona(user_id: str) -> dict | None:
89     return next((p for p in DEMO_PERSONAS if p["user_id"] == user_id), None)
90

```

```

1  """
2  데모 상태 자동 복원 (9/5 추가)
3
4  왜 필요한가
5  -----
6  이 배포는 심사 기간 동안 여러 사람이 차례로 열어보는 공개 데모인데, 화면의 주요 동작이
7  실제로 서버 상태를 바꿉니다. 그래서 앞서 사람이 만져놓은 상태 그대로 다음 사람이 보게 되고,
8  볼수록 데모가 약해집니다. 실제로 테스트 중에 두 가지가 확인됐습니다.
9
10 1. "연결하기"를 누르면 그 플랫폼이 영구히 연결됩니다. 박지수의 미연결 플랫폼을 연결하면
11   수입이 늘어 현금흐름이 '주의'에서 '안전'으로 바뀌고, 45일 내 잔고 부족 예상도 사라집니다
12   - 이 데모에서 가장 보여주고 싶은 장면이 사라지는 셈입니다. 동시에 "연결하면 예측이
13   정확해진다"는 동선 자체도 없어집니다.
14 2. "거래 위험도 검사"는 결과를 risk_events에 기록합니다(그게 이 기능의 장점이기도 합니다).
15   그런데 몇 번만 눌러도 최근 10건이 전부 테스트 평가로 채워져서, 원래 시드된 이상거래
16   (해외 결제·심야 고액 등)가 목록 밖으로 밀려납니다.
17
18 무엇을 하나
19 -----
20 주기적으로(기본 30분) 아래 둘을 시드 상태로 되돌립니다.
21
22 - income_sources.connected 를 scripts/seed_demo_income.py의 설정값으로 복원
23 - risk_events 중 transaction_id가 NULL인 행 삭제 - 실제 거래에 붙은 시드 이벤트는 모두
24   transaction_id를 갖고 있고, 화면에서 즉석 검사로 만들어진 평가만 NULL입니다.
25 - user_settings.min_safety_balance 를 NULL(=기본값)로 복원 - PUT /shield/{uid}/settings로
26   누군가 바꾼 최소 안전잔액이 다음 관람자에게 그대로 남지 않도록.
27 - Redis 예산 지출 카운터를 시드값으로 복원 - POST /budgets/record-spending은 Redis에
28   INCRBYFLOAT로 누적만 하고 되돌리지 않아서(TTL 40일), 몇 번만 눌러도 예산 사용률이
29   실제 거래와 무관하게 부풀려진 채 다음 관람자에게 남습니다. 시드와 같은 방식으로
30   (해당 키 삭제 후 실제 거래 합계로 다시 채움) 정확한 값으로 되돌립니다.
31
32 주기를 30분으로 잡은 이유: 한 사람이 둘러보는 동안(보통 몇 분)에는 자기가 만든 변화가
33 그대로 남아 있어야 조작한 보람이 있고, 그 사람이 떠난 뒤 다음 사람은 온전한 데모를
34 봐야 하기 때문입니다.
35 """
36 import logging
37
38 from sqlalchemy import text
39
40 logger = logging.getLogger("nfinity.demo_reset")
41
42
43 def reset_demo_state() -> dict:
44     """연결 상태와 즉석 평가 기록을 시드 상태로 되돌립니다. 실패해도 예외를 던지지 않습니다."""
45     from app.database import SessionLocal
46     from app.demo_personas import DEMO_PERSONAS
47
48     try:
49         from scripts.seed_demo_income import INCOME_PLATFORMS
50     except Exception:
51         logger.exception("[demo_reset] 시드 설정을 불러오지 못해 복원을 건너뛰니다.")
52         return {"restored": 0, "deleted_events": 0}
53
54     demo_ids = {p["user_id"] for p in DEMO_PERSONAS}
55     restored = 0
56     deleted = 0
57     db = SessionLocal()
58     try:
59         for uid, platforms in INCOME_PLATFORMS.items():
60             if uid not in demo_ids:
61                 continue
62             for platform in platforms:
63                 name, connected_seed = platform[0], platform[6]

```

```

64         result = db.execute(
65             text(
66                 "UPDATE income_sources SET connected = :want, "
67                 "connected_at = CASE WHEN :want THEN COALESCE(connected_at, now()) ELSE NULL END "
68                 "WHERE user_id = :uid AND platform_name = :name AND connected <> :want"
69             ),
70             {"want": bool(connected_seed), "uid": uid, "name": name},
71         )
72         restored += result.rowcount or 0
73
74     # 즉석 검사로 생긴 평가만 지웁니다(시드 이벤트는 transaction_id가 있습니다).
75     res = db.execute(text("DELETE FROM risk_events WHERE transaction_id IS NULL"))
76     deleted = res.rowcount or 0
77
78     # 화면에서 바뀌든 최소 안전잔액을 기본값으로 되돌립니다(NULL이면 get_shield가
79     # DEFAULT_MIN_SAFETY_BALANCE로 폴백). current_balance는 시드값이라 건드리지 않습니다.
80     for uid in demo_ids:
81         db.execute(
82             text("UPDATE user_settings SET min_safety_balance = NULL WHERE user_id = :uid"),
83             {"uid": uid},
84         )
85
86     db.commit()
87     if restored or deleted:
88         logger.info("[demo_reset] 연결 상태 %d건 복원, 즉석 평가 %d건 정리", restored, deleted)
89 except Exception:
90     db.rollback()
91     logger.exception("[demo_reset] 복원 중 오류가 발생했습니다.")
92 finally:
93     db.close()
94
95 # Redis 예산 지출 복원은 별도 트랜잭션/연결이라 위 DB 작업과 분리해서, 하나가 실패해도
96 # 다른 하나는 되돌아가도록 합니다(시드 스크립트의 로직을 그대로 재사용).
97 _reset_demo_spending()
98
99 return {"restored": restored, "deleted_events": deleted}
100
101
102 def _reset_demo_spending() -> None:
103     """데모 페르소나의 Redis 예산 지출 카운터를 시드 상태(실제 거래 합계)로 되돌립니다.
104     seed_demo_personas의 로직을 그대로 재사용합니다 - 먼저 해당 budget:* 키를 삭제하고,
105     실제 mock 거래를 카테고리별로 합산해 다시 채우므로 재호출해도 값이 정확합니다."""
106     from app.database import SessionLocal
107
108     try:
109         from scripts.seed_demo_personas import _reset_demo_redis_keys, seed_budgets_and_spending
110     except Exception:
111         logger.exception("[demo_reset] 예산 시드 로직을 불러오지 못해 지출 복원을 건너웁니다.")
112         return
113
114     try:
115         _reset_demo_redis_keys()
116         db = SessionLocal()
117         try:
118             seed_budgets_and_spending(db)
119         finally:
120             db.close()
121     except Exception:
122         logger.exception("[demo_reset] Redis 예산 지출 복원 중 오류가 발생했습니다.")
123

```

```

1  """
2  Gemini 호출 공통 계층 (9/6 신규)
3
4  왜 만들었나
5  -----
6  두 곳(업무경비 분류·AI 브리핑)이 각자 Gemini를 부르다 보니, 모델 하나가 죽으면 서비스 전체의
7  AI가 조용히 멈춥니다. 실제로 두 번 겪었습니다.
8
9  1. `gemini-2.0-flash`가 서비스 종료되어 모든 호출이 404. 응답 모양이 규칙 폴백과 같아서
10   화면상으로는 정상으로 보였고, /health도 "키가 있으니 gemini"라고만 보고했습니다.
11  2. 최신 모델로 바꾼 직후에는 503(high demand) - 일시적 용량 문제로 호출이 실패했습니다.
12
13  그래서 (a) 모델을 여러 개 순서대로 시도하고, (b) 마지막 호출이 성공했는지를 기록해서
14  /health가 사실대로 보고하게 합니다. 심사 기간에 특정 모델이 붐벼도 다음 모델로 넘어가고,
15  전부 실패하면 각 호출부가 가진 규칙/템플릿 폴백으로 내려갑니다.
16  """
17  import json
18  import logging
19  import os
20  import time
21
22  logger = logging.getLogger("nfinity.gemini")
23
24  # 앞에서부터 시도합니다. 환경변수 GEMINI_MODEL을 주면 그 모델을 최우선으로 씁니다.
25  # 9/6 진단(/api/v1/validation/gemini)으로 확인한 것:
26  #   gemini-2.5-flash → 404 (이미 서비스 종료) ... 체인에서 제거
27  #   gemini-3.6-flash → 429 (무료 할당량 소진)
28  #   gemini-flash-latest → 503 (일시 과부하)
29  # 종료된 모델을 계속 시도하는 건 실패 시간만 늘리므로 빼고, 경량 모델을 뒤에 둡니다.
30  # 경량 모델은 할당량이 따로 잡히는 경우가 있어 주 모델이 429일 때 살아있을 수 있습니다.
31  # 9/6 2차 진단 결과: gemini-3.6-flash-lite도 404(존재하지 않는 이름)였고,
32  # gemini-flash-lite-latest만 정상 응답했습니다. 주 모델들이 할당량(429)에 걸린 동안에도
33  # 경량 모델은 살아있어서, 마지막 폴백으로 두면 AI 결과를 계속 보여줄 수 있습니다.
34  _DEFAULT_CHAIN = [
35      "gemini-3.6-flash",
36      "gemini-flash-latest",
37      "gemini-flash-lite-latest",
38  ]
39
40  # 마지막 호출 결과 - /health가 읽습니다.
41  LAST_CALL_OK = None      # None=아직 호출 전 / True / False
42  LAST_CALL_ERROR = ""
43  LAST_MODEL_USED = ""
44
45  # 차단기(circuit breaker). 한 번 실패하면 이 시각까지는 아예 시도하지 않습니다.
46  #
47  # 왜 필요한가: 업무경비 분류는 거래 한 건마다 이 함수를 부르고(화면 한 장에 최대 35건),
48  # 실패하면 모델 체인을 3개까지 순서대로 시도합니다. Gemini가 503을 내는 동안에는
49  # 화면 한 번 그리는 데 실패한 네트워크 왕복이 백 번 가까이 쌓여 응답이 수십 초로 늘어나고,
50  # 실패는 캐시되지 않아 새로고침해도 그대로 반복됩니다. 실제로 이 상태에서 Gig Score와
51  # 업무경비 카드가 "불러오는 중"에서 멈추는 것을 확인했습니다.
52  # 한 번 실패하면 잠시 규칙 기반으로만 답하고, 쿨다운이 지난 뒤 다시 시도합니다.
53  COOLDOWN_SECONDS = int(os.environ.get("GEMINI_COOLDOWN_SECONDS", "120"))
54  _cooldown_until = 0.0
55
56
57  class GeminiCoolingDown(RuntimeError):
58      """직전 실패로 쿨다운 중이라 호출을 건너뛴 경우."""
59
60
61  def model_chain() -> list[str]:
62      preferred = (os.environ.get("GEMINI_MODEL") or "").strip()
63      chain = [m for m in _DEFAULT_CHAIN]

```

```

64     if preferred:
65         chain = [preferred] + [m for m in chain if m != preferred]
66     return chain
67
68
69 def available() -> bool:
70     """패키지와 키가 모두 있는지 (실제 호출이 되는지는 별개)."""
71     if not os.environ.get("GEMINI_API_KEY"):
72         return False
73     try:
74         from google import genai # noqa: F401
75     except ImportError:
76         return False
77     return True
78
79
80 def generate_json(system_instruction: str, contents: str, temperature: float = 0.0) -> dict:
81     """JSON 응답을 요구하는 호출. 모델 체인을 순서대로 시도하고, 전부 실패하면 예외를 뱉는다."""
82     global LAST_CALL_OK, LAST_CALL_ERROR, LAST_MODEL_USED, _cooldown_until
83
84     now = time.monotonic()
85     if now < _cooldown_until:
86         raise GeminiCoolingDown(
87             "직전 호출 실패로 " + str(int(_cooldown_until - now)) + "초간 규칙 기반으로 동작합니다."
88         )
89
90     from google import genai
91     from google.genai import types
92
93     last_exc = None
94     try:
95         client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
96     except Exception as exc: # 클라이언트 생성 실패도 "호출 실패"로 기록해야 health가 사실대로 보고합니다
97         LAST_CALL_OK, LAST_CALL_ERROR, LAST_MODEL_USED = False, (type(exc).__name__ + ": " + str(exc))[:200], ""
98         raise
99     for model in model_chain():
100         try:
101             resp = client.models.generate_content(
102                 model=model,
103                 contents=contents,
104                 config=types.GenerateContentConfig(
105                     system_instruction=system_instruction,
106                     response_mime_type="application/json",
107                     temperature=temperature,
108                 ),
109             )
110             parsed = json.loads(resp.text)
111             LAST_CALL_OK, LAST_CALL_ERROR, LAST_MODEL_USED = True, "", model
112             _cooldown_until = 0.0
113             return parsed
114         except Exception as exc:
115             last_exc = exc
116             logger.warning("[gemini] %s 호출 실패: %s", model, str(exc)[:160])
117
118     LAST_CALL_OK = False
119     LAST_CALL_ERROR = (type(last_exc).__name__ + ": " + str(last_exc))[:200] if last_exc else "unknown"
120     LAST_MODEL_USED = ""
121     _cooldown_until = time.monotonic() + COOLDOWN_SECONDS
122     raise last_exc if last_exc else RuntimeError("Gemini 호출 실패")
123
124
125 def status() -> str:
126     """/health용 한 줄 상태."""
127     if not os.environ.get("GEMINI_API_KEY"):
128         return "fallback (GEMINI_API_KEY 없음)"
129     try:
130         from google import genai # noqa: F401
131     except ImportError:

```

```
132         return "fallback (google-genai 패키지 없음)"
133     if LAST_CALL_OK is True:
134         return "gemini (" + LAST_MODEL_USED + ")"
135     if LAST_CALL_OK is False:
136         remain = int(max(0.0, _cooldown_until - time.monotonic()))
137         suffix = " . " + str(remain) + "초 후 재시도" if remain else ""
138         return "fallback (호출 실패: " + LAST_CALL_ERROR[:80] + suffix + ")"
139     return "gemini (대기 중 - 아직 호출 없음)"
140
```

```

1  """
2  배포된 데모를 깨워두는 자가 핑 (9/5 추가)
3
4  왜 필요한가
5  -----
6  Render 무료 플랜의 웹 서비스는 15분 동안 들어오는 요청이 없으면 컨테이너를 내리고,
7  그 다음 첫 요청은 다시 뜰 때까지 50초 넘게 걸립니다. 공모전 MVP 심사는
8  2026-09-07 11:00 ~ 09-11 23:59 사이에 이뤄지는데, 심사위원이 링크를 눌렀을 때 흰 화면을
9  한참 보게 되면 인상이 나빠지고, 접속 불가로 오해하면 결격 사유가 될 수도 있습니다.
10
11  어떻게
12  -----
13  앱이 뜬 뒤 백그라운드 태스크로 KEEPALIVE_INTERVAL_SECONDS마다 자기 자신의 공개 URL
14  (/health)을 한 번 호출합니다. 스피ندا운은 "외부에서 들어오는 요청이 없을 때" 일어나므로,
15  프로세스가 살아있는 동안 스스로 트래픽을 만들어 주면 잠들지 않습니다.
16
17  - 공개 URL은 Render가 넣어주는 RENDER_EXTERNAL_URL 환경변수에서 읽습니다. 이 값이 없는
18  환경(로컬 개발, docker-compose)에서는 아무 것도 하지 않습니다.
19  - 요청 실패는 무시하고 다음 주기에 다시 시도합니다. 이 기능이 실패해도 서비스 자체에는
20  영향이 없어야 하기 때문입니다.
21  - urllib만 씁니다(의존성 추가 없음). 동기 호출이라 이벤트 루프를 막지 않도록 스레드로
22  넘깁니다.
23
24  한계: 배포·재시작 직후처럼 프로세스가 아예 내려간 상태에서는 스스로 깨어날 수 없습니다.
25  그때는 누군가 한 번 접속하면 다시 살아나고, 이후로는 이 루프가 계속 유지합니다.
26  """
27  import asyncio
28  import logging
29  import os
30  import urllib.request
31
32  logger = logging.getLogger("nfinity.keepalive")
33
34  # 15분 스피ندا운 기준으로 여유를 둡니다.
35  KEEPALIVE_INTERVAL_SECONDS = int(os.environ.get("KEEPALIVE_INTERVAL_SECONDS", "300"))
36
37
38  def _public_health_url() -> str | None:
39      base = (os.environ.get("RENDER_EXTERNAL_URL") or "").strip().rstrip("/")
40      if not base:
41          return None
42      return base + "/health"
43
44
45  # 데모 상태 복원 주기(초).
46  #
47  # 처음엔 30분으로 잡았는데, 그 사이에 누군가 "연결하기"를 누르면 다음 사람이 다른 수치를
48  # 보게 됩니다. 실제로 플랫폼 하나가 연결된 채로 남아 잔고 부족 예상일이 사라지고(9/26 → 없음)
49  # 세금이 환급에서 납부로 뒤집혀, 기능명세서에 적어둔 확인 절차의 예상 결과와 화면이
50  # 어긋나는 상황이 발생했습니다. 관람자가 자기 조작 결과를 확인할 시간은 남기되
51  # 어긋난 상태가 오래가지 않도록 5분으로 줄였습니다. 관람자가 자기 조작 결과를 확인하기에는
52  # 충분하고(연결 즉시 화면이 다시 계산되어 바로 보임), 다음 사람이 어긋난 값을 볼 여지는 최소화됩니다.
53  DEMO_RESET_INTERVAL_SECONDS = int(os.environ.get("DEMO_RESET_INTERVAL_SECONDS", "300"))
54
55
56  async def _loop(url: str):
57      elapsed = 0
58      while True:
59          await asyncio.sleep(KEEPALIVE_INTERVAL_SECONDS)
60          elapsed += KEEPALIVE_INTERVAL_SECONDS
61          try:
62              await asyncio.to_thread(_fetch, url)
63          except Exception as exc: # 실패해도 서비스에 영향이 없어야 하므로 로그만 남깁니다.

```

```

64         logger.warning("[keepalive] 자가 핑 실패: %s", exc)
65
66     if elapsed >= DEMO_RESET_INTERVAL_SECONDS:
67         elapsed = 0
68         try:
69             from app.demo_reset import reset_demo_state
70
71             await asyncio.to_thread(reset_demo_state)
72         except Exception as exc:
73             logger.warning("[keepalive] 데모 상태 복원 실패: %s", exc)
74
75         # 생성형 AI 결과 캐시가 비어 있으면 조용히 채워둡니다.
76         #
77         # 무료 Redis는 재시작 시 내용이 사라질 수 있고, Gemini 무료 할당량은 하루 단위로
78         # 소진·회복됩니다. 둘이 겹치면 심사위원이 열었을 때 AI 브리핑이 템플릿 문장으로
79         # 보일 수 있어서, 주기적으로 비어 있는 것만 한 건씩 미리 만들어 둡니다.
80         # 한 번에 하나만 호출해 할당량을 아끼고, 실패하면 다음 주기에 다시 시도합니다.
81         try:
82             await asyncio.to_thread(_warm_one_brief)
83         except Exception as exc:
84             logger.warning("[keepalive] 브리핑 예열 실패: %s", exc)
85
86
87     def _warm_one_brief() -> None:
88         """캐시가 없는 데모 페르소나 브리핑을 하나만 생성해 캐시에 넣습니다."""
89         from app.database import SessionLocal
90         from app.demo_personas import DEMO_PERSONAS
91         from app.routers.brief import get_brief
92
93         db = SessionLocal()
94         try:
95             for persona in DEMO_PERSONAS:
96                 try:
97                     result = get_brief(persona["user_id"], db=db)
98                 except Exception:
99                     continue
100                 if result.get("source") == "gemini":
101                     continue # 이미 캐시에 있음
102                 # 아직 템플릿이면 방금 호출에서 캐시가 채워졌을 수도 있으니 여기서 멈춥니다.
103                 logger.info("[keepalive] %s 브리핑 예열 시도 (source=%s)", persona["name"], result.get("source"))
104                 return
105         finally:
106             db.close()
107
108
109     def _fetch(url: str):
110         with urllib.request.urlopen(url, timeout=30) as resp:
111             resp.read(64)
112
113
114     def start(loop_factory=asyncio.ensure_future):
115         """앱 시작 시 호출. 공개 URL을 모르는 환경에서는 조용히 넘어갑니다."""
116         url = _public_health_url()
117         if not url:
118             logger.info("[keepalive] RENDER_EXTERNAL_URL이 없어 자가 핑을 켜지 않습니다.")
119             return None
120         logger.info("[keepalive] %d초마다 %s 를 호출해 인스턴스를 깨워둡니다.", KEEPALIVE_INTERVAL_SECONDS, url)
121         return loop_factory(_loop(url))
122

```

```

1  """
2  FastAPI 앱 진입점.
3  실행: docker-compose up --build -d (또는 uvicorn app.main:app --reload)
4  확인: http://localhost:8000/docs 에서 Swagger UI로 API를 직접 눌러볼 수 있습니다.
5
6  8/30 보안 점검 업데이트:
7  - CORS: 프론트(정적 HTML/JS)가 이 API를 다른 origin에서 부를 수 있도록 허용 목록을
8  추가했습니다. 기본값은 로컬 개발용 origin들만 허용하고, ``*(전체 허용)는 절대
9  쓰지 않습니다 - 금융 데이터를 다루는 API라 필요합니다. 배포 시엔 .env의
10  CORS_ALLOWED_ORIGINS에 실제 프론트 배포 주소를 콤마로 추가하세요.
11  - 각 라우터(risk/budgets/gig-score/expense/demo)에는 app/security.py의 verify_api_key가
12  붙어있습니다. 자세한 이유는 그 파일 docstring 참고.
13
14  8/30 데모 프론트 연동:
15  - 루트(`/`)는 이제 JSON 상태 응답 대신 frontend/ 폴더의 데모 웹앱(index.html =
16  페르소나 선택 화면)을 그대로 서빙합니다. 헬스체크는 `/health`가 대신합니다.
17
18  9/2 세 번째 업데이트(보안 강화 - 금융보안원 주최 공모전이라 특별히 신경써달라는
19  요청으로 추가):
20  - SecurityHeadersMiddleware(app/security_headers.py): CSP, X-Frame-Options,
21  X-Content-Type-Options 등 브라우저 방어용 응답 헤더를 모든 응답에 붙입니다.
22  - RateLimitMiddleware(app/rate_limit.py): /api/v1/* 요청을 IP당 분당 제한해서
23  무차별 스크래이핑/무차별 대입을 최소한이라도 막습니다.
24  - 미들웨어는 나중에 등록한 게 바깥쪽(먼저 실행)이라, SecurityHeadersMiddleware를
25  가장 나중에 등록해서 CORS 에러/429 응답을 포함한 "모든" 응답에 보안 헤더가
26  붙도록 했습니다.
27  """
28  import os
29  from pathlib import Path
30
31  from fastapi import FastAPI, Request
32  from fastapi.exceptions import RequestValidationError
33  from fastapi.middleware.cors import CORSMiddleware
34  from fastapi.responses import JSONResponse
35  from fastapi.staticfiles import StaticFiles
36  from starlette.exceptions import HTTPException as StarletteHTTPException
37
38  from app.demo_guard import DemoUserGuardMiddleware
39  from app.keepalive import start as start_keepalive
40  from app.rate_limit import RateLimitMiddleware
41  from app.redis_client import ping as redis_ping
42  from app.routers import (
43      brief, budgets, demo, expense, gig_score, guidance, income, risk, shield, simulate, tax,
44      validation,
45  )
46  from app.security_headers import (
47      SecurityHeadersMiddleware,
48      build_security_headers,
49      is_https_request,
50  )
51  from app.startup_seed import run_startup_seed
52
53  app = FastAPI(
54      title="Nfinity API",
55      description="N잡러를 위한 리스크방어 & 예산통제 플랫폼 백엔드",
56      version="0.2.0",
57  )
58
59  _DEFAULT_DEV_ORIGINS = "http://localhost:3000,http://127.0.0.1:3000,http://localhost:5500,http://127.0.0.1:5500,http://localhost:8000,http://127.0.0.1:8000"
60  _cors_origins = [
61      o.strip()
62      for o in os.environ.get("CORS_ALLOWED_ORIGINS", _DEFAULT_DEV_ORIGINS).split(",")

```

```

63     if o.strip()
64 ]
65 app.add_middleware(
66     CORSMiddleware,
67     allow_origins=_cors_origins,
68     allow_credentials=False,
69     # 9/2 세 번째 업데이트: Risk Shield의 최소 안전잔액 설정(PUT /api/v1/shield/{uid}/settings)이
70     # 추가되면서 PUT도 허용해야 합니다.
71     allow_methods=["GET", "POST", "PUT"],
72     allow_headers=["Content-Type", "X-API-Key"],
73 )
74 # 9/5: 공개 데모라 시드된 페르소나 외의 user_id 조회를 서버에서 막습니다(app/demo_guard.py).
75 app.add_middleware(DemoUserGuardMiddleware)
76 app.add_middleware(RateLimitMiddleware)
77 app.add_middleware(SecurityHeadersMiddleware)
78
79 app.include_router(risk.router)
80 app.include_router(shield.router)
81 app.include_router(budgets.router)
82 app.include_router(gig_score.router)
83 app.include_router(expense.router)
84 app.include_router(income.router)
85 app.include_router(demo.router)
86 app.include_router(tax.router)
87 app.include_router(brief.router)
88 app.include_router(validation.router)
89 app.include_router(guidance.router)
90 app.include_router(simulate.router)
91
92
93 @app.on_event("startup")
94 def _seed_on_startup():
95     """9/2 추가: 배포 직후 DB가 비어있으면(관리형 Postgres에 처음 연결하는 경우 등)
96     스키마 생성 + 데모 데이터 시딩을 앱이 스스로 처리합니다. app/startup_seed.py 참고 -
97     이미 데이터가 있으면 아무 것도 하지 않고 즉시 반환하므로 재시작마다 반복 실행돼도
98     안전합니다."""
99     run_startup_seed()
100     # 9/5: Render 무료 플랜의 15분 스펠다운으로 심사 중 첫 접속이 50초 넘게 걸리는 걸
101     # 막기 위해, 앱이 스스로 자기 /health를 주기적으로 호출합니다(app/keepalive.py).
102     start_keepalive()
103     # 배포/재시작 시점에도 한 번 정리해서, 앞선 관람자가 만져놓은 상태로 시작하지 않게 합니다.
104     try:
105         from app.demo_reset import reset_demo_state
106
107         reset_demo_state()
108     except Exception: # 복원 실패가 서버 기동을 막으면 안 됩니다.
109         pass
110
111
112 # =====
113 # 공통 에러 응답 포맷 통일 (8/26 백엔드 구축 데이)
114 #
115 # 이전에는 라우터마다 FastAPI 기본 에러 형식({"detail": "..."})을 그대로 썼는데,
116 # 여기서 한 군데로 모아서 모든 API가 항상 같은 모양의 에러를 반환하도록 통일했습니다.
117 # 프론트(소현)는 이제 어떤 API를 호출하든 에러 처리 코드를 하나만 만들면 됩니다.
118 #
119 # {"error": {"code": 404, "message": "..."}}
120 # =====
121
122
123 @app.exception_handler(StarletteHTTPException)
124 async def http_exception_handler(request: Request, exc: StarletteHTTPException):
125     return JSONResponse(
126         status_code=exc.status_code,
127         content={"error": {"code": exc.status_code, "message": exc.detail}},
128     )
129
130

```

```

131 @app.exception_handler(RequestValidationError)
132 async def validation_exception_handler(request: Request, exc: RequestValidationError):
133     """요청 body/쿼리 형식이 잘못됐을 때(예: amount에 문자열을 보낸 경우) 나오는 에러."""
134     return JSONResponse(
135         status_code=422,
136         content={
137             "error": {
138                 "code": 422,
139                 "message": "입력값이 올바르지 않습니다.",
140                 "detail": exc.errors(),
141             }
142         },
143     )
144
145
146 @app.exception_handler(Exception)
147 async def unhandled_exception_handler(request: Request, exc: Exception):
148     """9/5 추가: 위 두 핸들러가 잡지 못한 "예상 못 한" 예외(예: 잘못된 쿼리로 DB가 던지는
149     오류)까지 항상 같은 {"error": {...}} 포맷으로 내보냅니다. 이전에는 이런 500이 Starlette
150     기본 처리로 빠져서 (1) 응답 포맷이 맨 'Internal Server Error' 문자열이었고 (2)
151     SecurityHeadersMiddleware가 실행되지 않아 CSP 등 보안 헤더가 통째로 빠졌습니다. 실제
152     원인/스택트레이스는 서버 로그로만 남기고, 응답 본문에는 노출하지 않습니다."""
153     resp = JSONResponse(
154         status_code=500,
155         content={"error": {"code": 500, "message": "서버 내부 오류가 발생했습니다."}},
156     )
157     for key, value in build_security_headers(is_https_request(request)).items():
158         resp.headers[key] = value
159     return resp
160
161
162 @app.get("/health", tags=["health"])
163 def health():
164     """DB는 요청이 오면 바로 알 수 있지만, Redis는 안 쓰는 API도 있어서 여기서 미리 확인합니다.
165
166     9/5 추가 - 업무경비 분류가 실제 Gemini를 부르는지, 규칙 기반 폴백으로 도는지도 같이
167     돌려줍니다. 둘의 응답 모양이 같아서 밖에서는 구분할 방법이 없었는데, 키를 넣었는데도
168     폴백으로 도는 상황(패키지 미설치, 키 오타 등)을 눈치채지 못하면 "AI 서비스"라고 해놓고
169     규칙 엔진만 돌리는 셈이 됩니다.
170     """
171     try:
172         from app.gemini_client import status as gemini_status
173
174         expense_ai = gemini_status()
175     except Exception as exc: # 진단이 헬스체크를 깨뜨리면 안 됩니다.
176         expense_ai = "unknown (" + type(exc).__name__ + ")"
177
178     return {
179         "status": "healthy",
180         "redis": "connected" if redis_ping() else "unreachable",
181         "expense_ai": expense_ai,
182     }
183
184
185 # =====
186 # 데모 프론트엔드 서빙 (8/30)
187 #
188 # 별도의 프론트 서버 없이, 이 FastAPI 앱 하나가 API(/api/v1/*)와 데모 웹앱을 함께
189 # 서빙합니다 - 공모전 MVP가 "실행 가능한 배포 URL 하나"를 요구하기 때문에, 한 프로세스로
190 # 배포가 끝나는 게 가장 실수할 여지가 적습니다. 반드시 다른 라우터를 전부 등록한
191 # "다음"에 마운트해야 합니다 - StaticFiles(html=True)는 "/" 아래 모든 경로를 잡아버려서,
192 # 먼저 마운트하면 /api/v1/* 같은 API 라우트가 전부 가려집니다.
193 # =====
194 _FRONTEND_DIR = Path(__file__).resolve().parent.parent / "frontend"
195 if _FRONTEND_DIR.is_dir():
196     app.mount("/", StaticFiles(directory=str(_FRONTEND_DIR), html=True), name="frontend")
197

```

```

1  """
2  Pydantic 모델 정의
3  - DB 테이블(sql/init.sql)과 1:1로 맞춰서 만들었습니다.
4  - 이 파일의 필드명을 B(소현)/C(하은)와 공유해서 mock_transactions.csv, 카테고리 매핑 결과가
5    여기 이름과 정확히 일치하도록 맞춰주세요. (이게 "데이터 계약"입니다)
6  """
7  from __future__ import annotations
8
9  from datetime import datetime
10 from enum import Enum
11 from typing import Optional
12
13 from pydantic import BaseModel, Field
14
15
16 class Severity(str, Enum):
17     LOW = "LOW"
18     MEDIUM = "MEDIUM"
19     HIGH = "HIGH"
20     CRITICAL = "CRITICAL"
21
22
23 class RuleAction(str, Enum):
24     ALLOW = "ALLOW"
25     FLAG_FOR_REVIEW = "FLAG_FOR_REVIEW"
26     REQUIRE_CONFIRMATION = "REQUIRE_CONFIRMATION"
27     BLOCK = "BLOCK"
28
29
30 class Transaction(BaseModel):
31     """하나의 결제/거래 건. transactions 테이블과 대응됩니다."""
32
33     transaction_id: Optional[str] = None
34     user_id: str
35     amount: float
36     merchant_id: Optional[str] = None
37     merchant_name: Optional[str] = None
38     mcc_code: Optional[str] = None
39     category: Optional[str] = None
40     timestamp: datetime
41     latitude: Optional[float] = None
42     longitude: Optional[float] = None
43     country: str = "KR"
44     device_id: Optional[str] = None
45
46
47 class UserRiskProfile(BaseModel):
48     """유저별 리스크 판단 기준선. user_risk_profiles 테이블과 대응됩니다."""
49
50     user_id: str
51     avg_transaction_amount: float = 0
52     std_transaction_amount: float = 0
53     single_transaction_limit: float = 500_000
54     daily_limit: float = 1_000_000
55     avg_daily_transactions: float = 0
56     allowed_countries: list[str] = Field(default_factory=lambda: ["KR"])
57
58
59 class RuleResult(BaseModel):
60     """룰 엔진이 규칙 하나에 위반되었을 때 반환하는 결과."""
61
62     rule_id: str
63     severity: Severity

```

```

64     description: str
65     action: RuleAction
66
67
68 class RiskAssessment(BaseModel):
69     """리스크 평가 최종 결과. risk_events 테이블과 대응됩니다."""
70
71     user_id: str
72     transaction_id: Optional[str] = None
73     score: int
74     severity: Severity
75     action: RuleAction
76     triggered_rules: list[RuleResult] = Field(default_factory=list)
77
78
79 # =====
80 # 예산통제 (budgets) - 8/26 백엔드 구축 데이에 새로 추가
81 # =====
82
83
84 class BudgetCreate(BaseModel):
85     """예산 설정/변경 요청. POST /api/v1/budgets 의 입력."""
86
87     user_id: str
88     category: str
89     period: str = "monthly" # weekly / monthly / quarterly
90     limit_amount: float
91
92
93 class BudgetUsage(BaseModel):
94     """카테고리 하나의 현재 사용 현황. GET /api/v1/budgets/usage 의 출력."""
95
96     user_id: str
97     category: str
98     period: str
99     spent_amount: float
100    limit_amount: float
101    usage_rate: float # 0.0 ~ 1.0+ (1.0 넘으면 예산 초과)
102    # 8/30 추가 (소현님 recurring_detector.py의 check_crossed_thresholds() 사용).
103    # 이미 넘은 임계값들(%). 예: [50, 80] = 50%/80%는 넘었고 90% 이상은 아직 안 넘음.
104    crossed_thresholds: list[int] = Field(default_factory=list)
105
106
107 class RecurringPayment(BaseModel):
108     """감지된 정기결제 한 건. BudgetForecast.recurring_payments 안에 들어갑니다."""
109
110     merchant_name: str
111     expected_amount: int
112     expected_date: str # YYYY-MM-DD
113
114
115 class BudgetForecast(BaseModel):
116     """월말 예상 지출. GET /api/v1/budgets/forecast 의 출력.
117
118     8/30: 소현님이 만든 정기결제 감지 로직(data_pipeline/recurring_detector.py)을
119     반영해서, "지금까지 쓴 돈/지난 날짜 수 * 이번달 전체 날짜 수"(base_predicted_total)에
120     이번 달 안에 아직 안 나온 정기결제 예상액(recurring_addition)을 더합니다.
121     """
122
123     user_id: str
124     category: str
125     current_spent: float
126     predicted_total: float # 정기결제까지 반영한 최종 예측치
127     base_predicted_total: float # 참고용: 정기결제 반영 전(기존 단순 선형 예측) 값
128     predicted_overage: float # 0이면 초과 예상 없음 (predicted_total 기준)
129     confidence: float # 0.0 ~ 1.0, 월초일수록 낮음
130     recurring_payments: list[RecurringPayment] = Field(default_factory=list)
131

```

```

132
133 # =====
134 # Gig-Score - 8/26 백엔드 구축 데이에 새로 추가 (스텝)
135 # =====
136
137
138 class GigScoreResponse(BaseModel):
139     """GET /api/v1/gig-score/{user_id} 의 출력.
140
141     지금은 임시 산식입니다. 세금 파킹 성실도·업무경비 비율 등
142     실제 지표로 8/30-31에 교체될 예정입니다.
143     """
144
145     user_id: str
146     score: int # 0 ~ 1000
147     components: dict[str, float]
148     message: str = ""
149
150
151 # =====
152 # AI 절세장부(경비분류) - 8/28 계약 확정판
153 # (조소현님이 실제 프롬프트/분류 로직을 완성해서 전달, data_pipeline/expense_classifier.py로 연동)
154 # ★ 필드명은 프론트(하은)와 이미 합의된 것이라 임의로 바꾸면 안 됩니다.
155 # =====
156
157
158 class ExpenseQueueItem(BaseModel):
159     """Tinder형 스와이프 카드 한 장에 해당하는 거래 한 건."""
160
161     id: int
162     merchant: str
163     amount: int
164     ai_tag: str # 예: "업무미팅", "SW구독", "개인지출", "미분류"
165     prob: int # 0 ~ 100 (업무 경비로 인정될 확률, %)
166
167
168 class ExpenseClassifyResponse(BaseModel):
169     """GET /api/v1/expense/classify 의 출력. 화면(스와이프 큐 + 환급액 카운터)에 그대로 매핑됩니다."""
170
171     transactions_queue: list[ExpenseQueueItem]
172     current_estimated_refund: int # prob >= APPROVAL_THRESHOLD(50)인 건들의 예상 환급액 합계(원)
173
174
175 # =====
176 # 수입 플랫폼 연결 (income) - 9/2 추가
177 # 실제 계좌/마이데이터 연동 대신, "연결하기"를 누르면 이미 있던 데모 데이터가
178 # 드러나는 방식으로 그 흐름을 재연합니다. app/demo_income.py 참고.
179 # =====
180
181
182 class IncomeSource(BaseModel):
183     """플랫폼/계좌 하나. income_sources 테이블과 대응됩니다."""
184
185     source_id: str
186     platform_name: str
187     platform_type: str # 배달 / 프리랜서 / 콘텐츠 / 커머스 / 본업
188     icon_emoji: str
189     connected: bool
190     connected_at: Optional[datetime] = None
191     # connected=false인 동안은 요약에서 제외되지만, 몇 건이 기다리고 있는지는 미리 보여줍니다
192     # ("연결하면 3건이 더 보여요" 같은 동기부여용 - 금액 자체는 연결 전엔 숨깁니다).
193     pending_event_count: int = 0
194
195
196 class IncomeConnectResponse(BaseModel):
197     """POST /api/v1/income/sources/{source_id}/connect 의 출력."""
198
199     source_id: str

```

```

200     platform_name: str
201     connected: bool
202     newly_connected: bool # 이번 호출로 처음 연결됐으면 true, 이미 연결돼있던 거면 false
203     revealed_event_count: int # 이번에 새로 드러난(=이미 있던) 수입 이벤트 건수
204
205
206 class IncomeEvent(BaseModel):
207     """수입 이벤트 하나. income_events 테이블과 대응됩니다."""
208
209     platform_name: str
210     icon_emoji: str
211     amount: float
212     memo: Optional[str] = None
213     status: str # 정산완료 / 정산예정
214     settled_at: datetime
215
216
217 class IncomeByPlatform(BaseModel):
218     """플랫폼별 이번 기간 수입 합계 (연결된 플랫폼만)."""
219
220     platform_name: str
221     platform_type: str
222     icon_emoji: str
223     total_amount: float
224     event_count: int
225
226
227 class IncomeSummary(BaseModel):
228     """GET /api/v1/income/summary 의 출력 - '한 장 요약' 홈 카드가 그대로 쓰는 모양."""
229
230     user_id: str
231     period_label: str # 예: "이번 주" / "이번 달"
232     total_settled: float # 연결된 플랫폼의 정산완료 합계
233     total_upcoming: float # 연결된 플랫폼의 정산예정 합계
234     by_platform: list[IncomeByPlatform] = Field(default_factory=list)
235     unconnected_source_count: int # 아직 연결 안 한 플랫폼 수 (있으면 연결 유도 배너용)
236
237
238 class IncomeDisconnectResponse(BaseModel):
239     """POST /api/v1/income/sources/{source_id}/disconnect 의 출력.
240     기획서 8번 "사용자가 연결 해제할 수 있는 화면" 요구사항 반영 (9/2 세 번째 업데이트)."""
241
242     source_id: str
243     platform_name: str
244     connected: bool # 항상 false
245
246
247 # =====
248 # Risk Shield / Dynamic Parking (9/2 세 번째 업데이트)
249 # app/cashflow.py의 계산 결과를 그대로 담는 응답 모델입니다.
250 # 필드명은 기획서 7-2 데모 API 계약을 그대로 따릅니다.
251 # =====
252
253
254 class DailyBalancePoint(BaseModel):
255     date: str
256     balance: float
257     events: list[str] = Field(default_factory=list)
258
259
260 class ShieldResponse(BaseModel):
261     user_id: str
262     current_balance: float
263     # "user_set" = 사용자가 알려준 잔액에서 출발한 시뮬레이션,
264     # "estimated" = 최근 30일 수입·지출로 앱이 추정한 값 (app/cashflow.py 참고)
265     balance_source: str = "estimated"
266     tax_reserve: float
267     insurance_reserve: float

```

```

268     available_cash: float
269     minimum_balance_date: Optional[str] = None
270     risk_level: str # 안전 / 주의 / 위험
271     risk_reasons: list[str] = Field(default_factory=list)
272     recommended_reserve: float
273     min_safety_balance: float
274     income_variability: float
275     daily_balances: list[DailyBalancePoint] = Field(default_factory=list)
276     calculated_at: str
277     disclaimer: str = (
278         "이 잔액과 예측은 연결된 수입·지출 데이터를 바탕으로 한 참고용 추정치이며, "
279         "실제 계좌 잔액이나 확정 세액·보험료가 아닙니다. 세무 신고, 보험료 고지, "
280         "신용·대출 심사에 사용되지 않습니다."
281     )
282
283
284 class ScenarioSeries(BaseModel):
285     date: str
286     balance: float
287
288
289 class ScenarioResponse(BaseModel):
290     user_id: str
291     calculated_at: str
292     scenarios: dict[str, list[ScenarioSeries]]
293
294
295 class SafetyBalanceUpdate(BaseModel):
296     min_safety_balance: float = Field(ge=0)
297
298
299 class SafetyBalanceResponse(BaseModel):
300     user_id: str
301     min_safety_balance: float
302     is_default: bool # 사용자가 직접 설정했으면 false, 기본값을 쓰고 있으면 true
303

```

```

1  """
2  기본 Rate Limiting (9/2 세 번째 업데이트 - 금융 데이터를 다루는 API라 스크립트로
3  전체 유저 데이터를 긁어가거나(스크래핑), API_ACCESS_KEY를 무차별 대입으로 맞춰보는
4  공격을 최소한이라도 막기 위해 추가했습니다.
5
6  구현 방식과 한계를 솔직하게 적어둡니다:
7  - 이 앱은 지금 단일 프로세스(uvicorn 워커 1개)로만 배포됩니다(Dockerfile CMD에
8    --workers 옵션이 없음). 그래서 메모리(dict) 안에 "IP별 최근 요청 시각"을 그냥
9    들고 있는 가장 단순한 방식으로도 충분히 동작합니다.
10 - 만약 나중에 --workers 2 이상으로 멀티프로세스 배포를 한다면, 이 방식은 프로세스마다
11   카운터가 따로 놀아서 실제 허용량이 워커 수만큼 늘어나 버립니다 - 그때는 Redis
12   (이미 이 프로젝트에 떠 있음) 같은 공유 저장소 기반으로 바꿔야 합니다. 지금 범위에선
13   단일 프로세스 배포가 맞아서 이 정도로 충분하다고 판단했습니다.
14 - 클라이언트 IP는 리버스 프록시(대부분의 PaaS가 이 앞에 하나 있음) 뒤에 있으면
15   X-Forwarded-For 헤더의 첫 값을 신뢰합니다. 이건 "믿을 수 있는 프록시가 이 헤더를
16   올바르게 채워준다"는 전제가 있어야 의미가 있고, 직접 인터넷에 노출된 배포(프록시 없음)
17   에서는 이 헤더를 요청자가 마음대로 조작할 수 있으므로 그 경우엔 request.client.host를
18   씁니다.
19 - 정적 프론트(HTML/JS/CSS)는 제한하지 않습니다 - 페이지 하나 로드에도 여러 파일을
20   받아오는데, 그것까지 세면 정상 사용자도 금방 걸립니다. /api/v1/* 요청만 셉니다.
21
22 기본값: IP당 분당 120회. 데모 페이지 하나가 보통 3~5개 API를 호출하니 넉넉하지만,
23 스크립트로 반복 호출하는 건 확실히 막습니다. 필요하면 아래 상수만 바꾸면 됩니다.
24 """
25 from __future__ import annotations
26
27 import time
28 from collections import defaultdict, deque
29
30 from starlette.middleware.base import BaseHTTPMiddleware
31 from starlette.requests import Request
32 from starlette.responses import JSONResponse, Response
33
34 WINDOW_SECONDS = 60
35 MAX_REQUESTS_PER_WINDOW = 120
36 _RATE_LIMITED_PREFIX = "/api/v1/"
37
38 # IP -> 최근 WINDOW_SECONDS 안에 들어온 요청 타임스탬프들
39 _hits: dict[str, deque[float]] = defaultdict(deque)
40
41
42 def _client_ip(request: Request) -> str:
43     forwarded = request.headers.get("x-forwarded-for")
44     if forwarded:
45         return forwarded.split(",")[0].strip()
46     return request.client.host if request.client else "unknown"
47
48
49 class RateLimitMiddleware(BaseHTTPMiddleware):
50     async def dispatch(self, request: Request, call_next) -> Response:
51         if not request.url.path.startswith(_RATE_LIMITED_PREFIX):
52             return await call_next(request)
53
54         ip = _client_ip(request)
55         now = time.monotonic()
56         window = _hits[ip]
57
58         while window and now - window[0] > WINDOW_SECONDS:
59             window.popleft()
60
61         if len(window) >= MAX_REQUESTS_PER_WINDOW:
62             return JSONResponse(
63                 status_code=429,

```

```
64         content={
65             "error": {
66                 "code": 429,
67                 "message": "요청이 너무 많습니다. 잠시 후 다시 시도해주세요.",
68             }
69         },
70         headers={"Retry-After": str(WINDOW_SECONDS)},
71     )
72
73     window.append(now)
74     return await call_next(request)
75
```

```

1  """
2  Redis 연동 헬퍼.
3
4  지금까지는 database.py에 get_redis() 함수만 있고 실제로 쓰는 곳이 없었습니다
5  ("README에 '지금은 연결만 해두고 아직 안 씀'이라고 적혀 있던 그 부분").
6  이 파일부터 실제로 Redis를 씁니다 - 예산통제 기능에서 "이번 달 카테고리별로
7  얼마 썼는지"를 실시간으로 집계할 때 씁니다.
8
9  왜 굳이 Redis를 쓰나요?
10 - PostgreSQL에서 매번 `SELECT SUM(amount) FROM transactions WHERE ...`로 합계를 구해도
11   되긴 하지만, 거래가 쌓일수록 매번 다시 계산하는 게 느려집니다.
12 - Redis는 "숫자를 계속 더해나가는" 작업(INCRBYFLOAT)에 최적화된 아주 빠른 저장소라서,
13   거래가 들어올 때마다 누적값에 더하기만 하면 되고 조회도 즉시 됩니다.
14 - 대신 Redis 값은 "언제든 날아가도 되는 캐시"로 취급합니다. 진짜 원본 데이터는
15   PostgreSQL의 transactions 테이블에 그대로 남아있습니다.
16  """
17  from __future__ import annotations
18
19  import os
20  from datetime import datetime
21  from typing import Optional
22
23  import redis
24
25  REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")
26
27  _redis_client: Optional[redis.Redis] = None
28
29
30  def get_redis_client() -> redis.Redis:
31      """Redis 연결은 앱 시작할 때 한 번만 만들고 재사용합니다 (매번 새로 연결하면 느려짐)."""
32      global _redis_client
33      if _redis_client is None:
34          _redis_client = redis.from_url(REDIS_URL, decode_responses=True)
35      return _redis_client
36
37
38  def _period_key(period: str, dt: Optional[datetime] = None) -> str:
39      """'2026-08' 같은, 기간을 나타내는 문자열을 만듭니다. (월간/주간/일간)"""
40      dt = dt or datetime.utcnow()
41      if period == "monthly":
42          return dt.strftime("%Y-%m")
43      if period == "weekly":
44          iso = dt.isocalendar()
45          return f"{iso.year}-W{iso.week:02d}"
46      return dt.strftime("%Y-%m-%d") # daily 등 그 외는 일 단위로
47
48
49  def budget_key(user_id: str, category: str, period: str = "monthly", dt: Optional[datetime] = None) -> str:
50      """
51      Redis에 값을 저장할 때 쓰는 '이름표'입니다.
52      예: budget:u_001:식비:monthly:2026-08
53      이렇게 유저+카테고리+기간을 다 합쳐서 키로 만들면, 서로 다른 유저/카테고리/달의
54      값이 섞이지 않습니다.
55      """
56      return f"budget:{user_id}:{category}:{period}:{_period_key(period, dt)}"
57
58
59  def add_spending(
60      user_id: str, category: str, amount: float, period: str = "monthly", now: Optional[datetime] = None
61  ) -> float:
62      """
63      거래 하나가 발생할 때마다 호출합니다. 누적 지출에 amount를 더하고,

```

```

64     더한 뒤의 새 누적값을 반환합니다.
65
66     INCRBYFLOAT는 "현재 값에 얼마를 더해라"를 원자적으로(다른 요청과 겹쳐도 안전하게)
67     처리해주는 Redis 명령어입니다.
68
69     now: 9/2에 추가된 파라미터입니다. "이번 달"을 실서버 시각이 아니라 데모 데이터
70     기준 시각(app/demo_clock.py의 get_demo_now)으로 맞추기 위한 것입니다 - 실서버
71     시각을 그대로 쓰면 mock 데이터가 끝난 달이 지나가는 순간 이번 달 누적치가 항상
72     0이 되어버리는 버그가 있었습니다. 안 넘기면 기존처럼 실서버 시각을 씁니다.
73     """
74     client = get_redis_client()
75     key = budget_key(user_id, category, period, now)
76     new_total = client.incrbyfloat(key, amount)
77     # 기간이 끝나면(예: 다음 달이 되면) 이 값은 더 이상 의미가 없으니, 40일 뒤 자동 삭제
78     # 되도록 만료시간(TTL)을 걸어둡니다. 안 걸어두면 Redis에 계속 쌓입니다.
79     client.expire(key, 60 * 60 * 24 * 40)
80     return float(new_total)
81
82
83 def get_spending(user_id: str, category: str, period: str = "monthly", now: Optional[datetime] = None) -> float:
84     """지금까지 누적된 지출을 조회합니다. 값이 없으면(=아직 지출 없음) 0을 반환합니다.
85     now: add_spending과 동일 - 데모 기준 시각을 넘기면 그 기준으로 키를 찾습니다."""
86     client = get_redis_client()
87     key = budget_key(user_id, category, period, now)
88     val = client.get(key)
89     return float(val) if val else 0.0
90
91
92 def ping() -> bool:
93     """Redis가 살아있는지 확인 (헬스체크용)."""
94     try:
95         return get_redis_client().ping()
96     except redis.exceptions.RedisError:
97         return False
98

```

```

1  """
2  AI 브리핑 API (9/6 신규) - GET /api/v1/brief/{user_id}
3
4  무엇을 하나
5  -----
6  이 사용자의 현금흐름·세금·예산 계산 결과를 한 문단으로 풀어 설명하고, 지금 할 일을 한 줄로
7  제안합니다. 화면 맨 위에서 "그래서 내 상황이 어떨다는 건데?"에 답하는 자리입니다.
8
9  왜 생성형 AI인가
10 -----
11 숫자는 이미 규칙과 통계로 정확히 계산됩니다(app/cashflow.py, app/tax.py). 문제는 그 숫자들이
12 서로 얹혀 있을 때 사람이 인과를 읽어내기 어렵다는 것입니다 - "잔고는 145만원인데 세금·건보료
13 빼면 107만원이고, 다음 정산은 8일 뒤인데 그 사이 고정비가 나가서 9월 26일에 바닥난다"를
14 스스로 이어붙여야 합니다. 이 연결을 문장으로 만들어주는 게 생성형 모델이 실제로 잘하는 일이라
15 여기에 씁니다.
16
17 환각 방지
18 -----
19 모델에는 **이미 계산이 끝난 수치만** 넘기고, 새로운 숫자를 만들어내거나 조언을 지어내지 말라고
20 지시합니다. 호출이 실패하거나 키가 없으면 같은 수치로 만든 템플릿 문장으로 조용히 대체해서,
21 이 기능 때문에 화면이 비는 일이 없게 합니다.
22 """
23 import json
24 import logging
25 import os
26 from functools import lru_cache
27
28 from fastapi import APIRouter, Depends, HTTPException
29 from sqlalchemy.orm import Session
30
31 from app.cashflow import compute_shield
32 from app.database import get_db
33 from app.security import verify_api_key
34
35 logger = logging.getLogger("nfinity.brief")
36 router = APIRouter(prefix="/api/v1/brief", tags=["brief"], dependencies=[Depends(verify_api_key)])
37
38 SYSTEM_PROMPT = """너는 N잡러(여러 플랫폼에서 수입을 얻는 사람)의 자금 상황을 설명해주는 금융 비서다.
39 주어진 수치만 사용해 상황을 설명하고, 지금 할 일을 하나만 제안한다.
40
41 규칙:
42 - 주어지지 않은 숫자나 사실을 절대 만들어내지 마라.
43 - 2~3문장, 존댓말, 담백하게. 과장·불안 조성 금지.
44 - 금액은 만원 단위로 반올림해 읽기 쉽게 (예: 145만원).
45 - 확정된 세액·보험료가 아니라 추정치라는 점을 필요할 때만 짧게 덧붙인다.
46 - 출력은 JSON: {"summary": "...", "action": "..."} (action은 25자 이내 한 줄 권고)"""
47
48
49 def _facts(db: Session, user_id: str) -> tuple[dict, dict]:
50     shield = compute_shield(db, user_id)
51     if shield is None:
52         raise HTTPException(status_code=404, detail="아직 계산할 수입·지출 데이터가 없습니다.")
53
54     facts = {
55         "현재잔액": shield["current_balance"],
56         "세금건보료_제외_가용액": shield["available_cash"],
57         "최소안전잔액": shield["min_safety_balance"],
58         "잔고부족_예상일": shield["minimum_balance_date"],
59         "현금흐름_위험도": shield["risk_level"],
60         "위험사유": shield["risk_reasons"],
61         "수입_변동계수": shield["income_variability"],
62     }
63     try:

```

```

64         from app.routers.tax import estimate as tax_estimate
65
66         tax = tax_estimate(user_id, db)
67         it, hi = tax["income_tax"], tax["health_insurance"]
68         facts["연_사업소득_환산"] = tax["annual_business_income"]
69         facts["5월_종합소득세_정산액"] = it["balance_due"] # 음수면 환급
70         facts["건강보험료_월"] = hi["total_monthly"]
71         facts["건강보험_가입자유형"] = hi["subscriber_type"]
72         if hi.get("warning"):
73             facts["건강보험_경고"] = hi["warning"]
74     except HTTPException:
75         pass
76     except Exception:
77         logger.exception("[brief] 세금 추정 조회 실패")
78     return facts, shield
79
80
81 def _fallback(facts: dict) -> dict:
82     """키가 없거나 호출이 실패했을 때 같은 수치로 만드는 템플릿 문장."""
83     man = lambda v: format(int(round((v or 0) / 10000)), ",") + "만원"
84     short = facts.get("잔고부족_예상일")
85     if short:
86         summary = (
87             "현재 잔액 " + man(facts.get("현재잔액")) + " 중 세금·건보료를 빼면 "
88             + man(facts.get("세금건보료_제외_가용액")) + "을 쓸 수 있고, 이대로면 "
89             + str(short) + "에 최소 안전잔액 아래로 내려갑니다."
90         )
91         action = "정산 예정 수입 확인하기"
92     else:
93         summary = (
94             "현재 잔액 " + man(facts.get("현재잔액")) + " 기준으로 앞으로 45일간 잔고가 "
95             + "최소 안전잔액 아래로 내려갈 것으로 보이지는 않습니다."
96         )
97         action = "지금 상태 유지하기"
98     tax = facts.get("5월_종합소득세_정산액")
99     if isinstance(tax, (int, float)) and tax < 0:
100         summary += " 5월 종합소득세는 " + man(-tax) + " 환급이 예상됩니다."
101     return {"summary": summary, "action": action, "source": "template"}
102
103
104 @lru_cache(maxsize=64)
105 def _call_gemini_cached(facts_json: str) -> str:
106     """같은 수치면 같은 문장이 나오므로 캐시합니다(화면 재방문 때마다 호출하지 않도록)."""
107     from google import genai
108     from google.genai import types
109
110     client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
111     resp = client.models.generate_content(
112         model=os.environ.get("GEMINI_MODEL", "gemini-3.6-flash"),
113         contents="다음은 이미 계산이 끝난 이 사용자의 수치다:\n" + facts_json,
114         config=types.GenerateContentConfig(
115             system_instruction=SYSTEM_PROMPT,
116             response_mime_type="application/json",
117             temperature=0.2,
118         ),
119     )
120     return resp.text
121
122
123 @router.get("/{user_id}")
124 def get_brief(user_id: str, debug: int = 0, db: Session = Depends(get_db)):
125     facts, shield = _facts(db, user_id)
126     facts_json = json.dumps(facts, ensure_ascii=False, sort_keys=True)
127
128     from app.gemini_client import available
129
130     if not available():
131         result = _fallback(facts)

```

```
132     else:
133         try:
134             parsed = json.loads(_call_gemini_cached(facts_json))
135             result = {
136                 "summary": str(parsed.get("summary", ""))[:400],
137                 "action": str(parsed.get("action", ""))[:60],
138                 "source": "gemini",
139             }
140             if not result["summary"]:
141                 result = _fallback(facts)
142         except Exception as exc:
143             logger.exception("[brief] Gemini 호출 실패 → 템플릿 문장으로 대체")
144             result = _fallback(facts)
145             if debug: # 배포 환경에서 실패 원인을 확인하기 위한 임시 진단
146                 result["error"] = type(exc).__name__ + ": " + str(exc)[:200]
147
148     result["disclaimer"] = "계산된 추정치를 바탕으로 작성된 안내이며, 확정 세액·보험료가 아닙니다."
149     return result
150
```

```

1  """
2  예산통제 API
3
4  - POST /api/v1/budgets          : 카테고리별 예산 한도 설정
5  - GET /api/v1/budgets/usage     : 이번 달 카테고리별 사용량 (Redis에서 실시간으로 읽음) +
6    이미 넘은 알람 임계값(crossed_thresholds)
7  - POST /api/v1/budgets/record-spending : 거래 하나만큼 누적 지출에 더하기 (지금은 수동 호출용.
8    나중엔 카테고리분류 파이프라인에서 거래가 들어올 때마다 자동으로 이 함수를 호출하게 연결합니다)
9  - GET /api/v1/budgets/forecast   : 월말 예상 지출 = 기존 단순 선형 예측 + 이번 달 안에
10   아직 안 나온 정기결제 예상액 (8/30, 소현님의 data_pipeline/recurring_detector.py 연동)
11
12  8/30 업데이트: 소현님이 만든 정기결제 감지(detect_recurring_payments)와 예산 임계값 체크
13  (check_crossed_thresholds)를 실제로 연결했습니다. 기존 Redis 기반 "지금까지 쓴 돈" 계산은
14  그대로 두고, 거기에 "아직 안 나온 정기결제 예상액"만 더하는 방식입니다 - 이미 검증된
15  기존 계산을 갈아엎지 않고 엮는 구조라, 정기결제가 없는 카테고리는 이전과 동일하게 동작합니다.
16
17  9/2 버그 수정: "이번 달"을 실서버 시각(date.today())으로 계산하고 있었는데, mock 데이터가
18  2026-08-22에 끝나서 실제 달력이 9월로 넘어가면(딱 심사 기간, 9/7~9/11) 이번 달 실거래가
19  0건이 되어 예산 사용량이 전부 0으로 보이는 문제가 있었습니다. app/demo_clock.py의
20  get_demo_now()로 "데이터 기준 지금"을 쓰도록 고쳤습니다 - 아래 모든 today는 그 값입니다.
21  """
22  from calendar import monthrange
23  from datetime import datetime
24
25  import pandas as pd
26  from fastapi import APIRouter, Depends, HTTPException
27  from sqlalchemy import text
28  from sqlalchemy.orm import Session
29
30  from app.database import get_db
31  from app.demo_clock import get_demo_now
32  from app.models import BudgetCreate, BudgetForecast, BudgetUsage, RecurringPayment
33  from app.redis_client import add_spending, get_spending
34  from app.security import verify_api_key
35
36  try:
37     from data_pipeline.categorizer import categorize_merchant
38     from data_pipeline.recurring_detector import check_crossed_thresholds, detect_recurring_payments
39 except ImportError: # uvicorn을 data_pipeline 폴더 안에서 띄우는 등 경로가 다를 때 대비
40     from categorizer import categorize_merchant # type: ignore
41     from recurring_detector import check_crossed_thresholds, detect_recurring_payments # type: ignore
42
43  router = APIRouter(prefix="/api/v1/budgets", tags=["budgets"], dependencies=[Depends(verify_api_key)])
44
45  _DEFAULT_THRESHOLDS_PCT = [50, 80, 90, 100, 120] # alert_thresholds가 비어있을 때의 기본값
46
47
48  def _crossed_thresholds_pct(usage_rate: float, alert_thresholds_fraction) -> list[int]:
49      """budgets.alert_thresholds는 0.5/0.8 같은 '비율'로 저장되어 있어서, 소현님 함수가
50      기대하는 '%' 단위(50/80)로 변환해서 넘겨줍니다."""
51      thresholds_pct = (
52          [round(float(t) * 100) for t in alert_thresholds_fraction]
53          if alert_thresholds_fraction
54          else _DEFAULT_THRESHOLDS_PCT
55      )
56      return check_crossed_thresholds(usage_rate, alert_thresholds=thresholds_pct)
57
58
59  def _load_transactions_df(db: Session, user_id: str) -> pd.DataFrame:
60      """detect_recurring_payments()가 기대하는 컬럼(user_id, merchant_name, amount,
61      timestamp) 그대로 유저의 전체 거래 이력을 DataFrame으로 불러옵니다."""
62      rows = db.execute(
63          text(

```

```

64         "SELECT user_id, merchant_name, amount, timestamp FROM transactions "
65         "WHERE user_id = :uid ORDER BY timestamp"
66     ),
67     {"uid": user_id},
68 ).mappings().all()
69 return pd.DataFrame(
70     [{"user_id": r["user_id"], "merchant_name": r["merchant_name"],
71      "amount": float(r["amount"]), "timestamp": r["timestamp"]} for r in rows]
72 )
73
74
75 @router.post("", response_model=BudgetUsage)
76 def create_or_update_budget(body: BudgetCreate, db: Session = Depends(get_db)):
77     """카테고리별 월 예산 한도를 설정합니다. 이미 있으면 한도만 갱신합니다."""
78     # 9/6: user_id를 본문으로 받는 유일한 쓰기 엔드포인트라, 미들웨어(app/demo_guard.py)의
79     # 경로·쿼리 검사에 걸리지 않습니다. 실제로 배포본에서 시도된 비데모 유저의 예산 한도를
80     # 임의로 바꿀 수 있는 걸 확인해서, /risk/assess와 같은 방식으로 여기서 직접 막습니다.
81     from app.demo_guard import allowed_user_ids
82
83     if body.user_id not in allowed_user_ids():
84         raise HTTPException(status_code=403, detail="이 시연 배포는 공개된 데모 계정만 수정할 수 있습니다.")
85
86     row = db.execute(
87         text(
88             """
89             INSERT INTO budgets (user_id, category, period, limit_amount)
90             VALUES (:uid, :cat, :period, :limit_amount)
91             ON CONFLICT (user_id, category, period) DO UPDATE
92             SET limit_amount = EXCLUDED.limit_amount, updated_at = now()
93             RETURNING alert_thresholds
94             """
95         ),
96         {
97             "uid": body.user_id,
98             "cat": body.category,
99             "period": body.period,
100             "limit_amount": body.limit_amount,
101         },
102     ).mappings().first()
103     db.commit()
104
105     spent = get_spending(body.user_id, body.category, body.period, now=get_demo_now(db))
106     usage_rate = (spent / body.limit_amount) if body.limit_amount else 0.0
107     return BudgetUsage(
108         user_id=body.user_id,
109         category=body.category,
110         period=body.period,
111         spent_amount=spent,
112         limit_amount=body.limit_amount,
113         usage_rate=usage_rate,
114         crossed_thresholds=_crossed_thresholds_pct(usage_rate, row["alert_thresholds"] if row else None),
115     )
116
117
118 @router.get("/usage", response_model=list[BudgetUsage])
119 def get_usage(user_id: str, period: str = "monthly", db: Session = Depends(get_db)):
120     """유저가 설정해둔 모든 카테고리의 이번 기간 사용량을 반환합니다."""
121     rows = db.execute(
122         text(
123             "SELECT category, limit_amount, alert_thresholds FROM budgets "
124             "WHERE user_id = :uid AND period = :period"
125         ),
126         {"uid": user_id, "period": period},
127     ).mappings().all()
128
129     if not rows:
130         raise HTTPException(
131             status_code=404,

```

```

132         detail="설정된 예산이 없습니다. 먼저 POST /api/v1/budgets 로 예산을 만들어주세요.",
133     )
134
135     demo_now = get_demo_now(db)
136     result = []
137     for r in rows:
138         spent = get_spending(user_id, r["category"], period, now=demo_now)
139         limit_amount = float(r["limit_amount"])
140         usage_rate = (spent / limit_amount) if limit_amount else 0.0
141         result.append(
142             BudgetUsage(
143                 user_id=user_id,
144                 category=r["category"],
145                 period=period,
146                 spent_amount=spent,
147                 limit_amount=limit_amount,
148                 usage_rate=usage_rate,
149                 crossed_thresholds=_crossed_thresholds_pct(usage_rate, r["alert_thresholds"]),
150             )
151         )
152     return result
153
154
155 @router.post("/record-spending")
156 def record_spending(user_id: str, category: str, amount: float, period: str = "monthly", db: Session = Depends(get_d
b)):
157     """
158     거래 하나만큼 누적 지출에 더합니다.
159     지금은 테스트를 위해 직접 호출하는 용도이고, 나중엔 하은님의 카테고리분류
160     파이프라인이 거래를 저장할 때마다 자동으로 이 로직을 같이 실행하도록 연결할 예정입니다.
161     """
162     new_total = add_spending(user_id, category, amount, period, now=get_demo_now(db))
163     return {"user_id": user_id, "category": category, "period": period, "total_spent": new_total}
164
165
166 @router.get("/forecast", response_model=BudgetForecast)
167 def get_forecast(user_id: str, category: str, period: str = "monthly", db: Session = Depends(get_db)):
168     """
169     월말 예상 지출을 계산합니다.
170
171     기본 예측(base): (지금까지 쓴 돈 / 지난 날짜 수) * 이번 달 전체 날짜 수
172     → "지금까지의 지출 속도가 이번 달 내내 유지된다면"을 가정한 가장 단순한 선형 예측.
173     여기에 8/30부터는, 이 카테고리에 속하는 가맹점 중 이번 달 안에 아직 결제 안 된
174     정기결제(넷플릭스 등)의 예상 금액을 더합니다 - data_pipeline/recurring_detector.py.
175     """
176     row = db.execute(
177         text(
178             "SELECT limit_amount FROM budgets "
179             "WHERE user_id = :uid AND category = :cat AND period = :period"
180         ),
181         {"uid": user_id, "cat": category, "period": period},
182     ).mappings().first()
183     if not row:
184         raise HTTPException(status_code=404, detail="설정된 예산이 없습니다.")
185
186     limit_amount = float(row["limit_amount"])
187     demo_now = get_demo_now(db)
188     spent = get_spending(user_id, category, period, now=demo_now)
189
190     today = demo_now.date()
191     days_in_month = monthrange(today.year, today.month)[1]
192     days_elapsed = today.day
193     daily_rate = spent / days_elapsed if days_elapsed else 0.0
194     base_predicted_total = daily_rate * days_in_month
195
196     # 이 유저의 전체 거래 이력에서 정기결제 후보를 찾고, 이 카테고리에 속하는
197     # 가맹점만, 그리고 "이번 달 안 + 아직 안 지나간" 것만 골라서 더합니다.
198     recurring_payments: list[RecurringPayment] = []

```

```

199     recurring_addition = 0.0
200     tx_df = _load_transactions_df(db, user_id)
201     if not tx_df.empty:
202         recurring_df = detect_recurring_payments(tx_df)
203         if not recurring_df.empty:
204             recurring_df = recurring_df[
205                 recurring_df["merchant_name"].apply(categorize_merchant) == category
206             ]
207             now = datetime.combine(today, datetime.min.time())
208             month_start = now.replace(day=1)
209             next_month = (month_start.replace(day=28) + pd.Timedelta(days=4)).replace(day=1)
210             recurring_df = recurring_df.copy()
211             recurring_df["next_expected_date"] = pd.to_datetime(recurring_df["next_expected_date"])
212             upcoming = recurring_df[
213                 (recurring_df["next_expected_date"] > now) & (recurring_df["next_expected_date"] < next_month)
214             ]
215             for _, r in upcoming.iterrows():
216                 recurring_payments.append(
217                     RecurringPayment(
218                         merchant_name=r["merchant_name"],
219                         expected_amount=int(r["avg_amount"]),
220                         expected_date=r["next_expected_date"].date().isoformat(),
221                     )
222                 )
223             recurring_addition = float(upcoming["avg_amount"].sum())
224
225     predicted_total = base_predicted_total + recurring_addition
226
227     return BudgetForecast(
228         user_id=user_id,
229         category=category,
230         current_spent=spent,
231         predicted_total=predicted_total,
232         base_predicted_total=base_predicted_total,
233         predicted_overage=max(0.0, predicted_total - limit_amount),
234         confidence=min(days_elapsed / days_in_month, 1.0),
235         recurring_payments=recurring_payments,
236     )
237

```

```

1  """
2  데모 페르소나 목록 API (8/30 추가)
3
4  - GET /api/v1/demo/personas
5
6  프론트(데모 페르소나 선택 화면)가 이 API 하나로 5명의 카드 정보를 그림니다.
7  목록의 실제 값은 app/demo_personas.py 한 곳에서만 관리합니다.
8  """
9  from fastapi import APIRouter, Depends
10
11  from app.demo_personas import DEMO_PERSONAS
12  from app.security import verify_api_key
13
14  router = APIRouter(prefix="/api/v1/demo", tags=["demo"], dependencies=[Depends(verify_api_key)])
15
16
17  @router.get("/personas")
18  def list_demo_personas():
19      return {
20          "note": "실제 인물이 아닌, 시연을 위해 준비한 가상 데이터입니다.",
21          "personas": [
22              {
23                  "user_id": p["user_id"],
24                  "name": p["name"],
25                  "job": p["job"],
26                  "tagline": p["tagline"],
27                  "avatar_emoji": p["avatar_emoji"],
28              }
29              for p in DEMO_PERSONAS
30          ],
31      }
32

```

```

1  """
2  AI 절세장부(경비분류) API - 8/28 실제 로직 연동판
3
4  - GET /api/v1/expense/classify?user_id=...&limit=15
5
6  조소현님이 완성한 프롬프트/분류 로직(data_pipeline/expense_classifier.py)을 그대로 가져다 씁니다.
7  이 라우터가 하는 일은 딱 하나 - "DB에서 그 유저의 실제 거래를 읽어와서 classify_one()에
8  하나씩 넘기고, 결과를 프론트와 합된 모양(transactions_queue + current_estimated_refund)으로
9  포장해서 돌려주는 것"입니다. 분류 로직 자체(프롬프트, few-shot, 규칙 기반 풀백)는 전혀
10 손대지 않았습니다.
11
12 유저의 "직업(job)" 정보는 users 테이블의 persona 컬럼을 그대로 씁니다 - 이미
13 '프리랜서 마케터/데이터분석가', '배달_투잡러' 같은 문자열이 들어있어서 조소현님 로직의
14 JOB_CONTEXT_MERCHANTS 판단("개발자"/"마케터" 문자열 포함 여부)과 그대로 맞습니다.
15
16 Mock/실제 API 전환은 이 파일이 아니라 data_pipeline/expense_classifier.py의 MOCK_MODE가
17 결정합니다 (.env에 GEMINI_API_KEY가 있으면 자동으로 실제 Gemini 호출로 전환).
18 """
19 from fastapi import APIRouter, Depends, HTTPException, Query
20 from sqlalchemy import text
21 from sqlalchemy.orm import Session
22
23 from app.database import get_db
24 from app.models import ExpenseClassifyResponse
25 from app.security import verify_api_key
26
27 try:
28     from data_pipeline.expense_classifier import (
29         APPROVAL_THRESHOLD,
30         ASSUMED_MARGINAL_TAX_RATE,
31         classify_one,
32     )
33 except ImportError: # uvicorn을 data_pipeline 폴더 안에서 띄우는 등 경로가 다를 때 대비
34     from expense_classifier import ( # type: ignore
35         APPROVAL_THRESHOLD,
36         ASSUMED_MARGINAL_TAX_RATE,
37         classify_one,
38     )
39
40 router = APIRouter(prefix="/api/v1/expense", tags=["expense"], dependencies=[Depends(verify_api_key)])
41
42
43 @router.get("/classify", response_model=ExpenseClassifyResponse)
44 def classify_expense(user_id: str, limit: int = Query(15, ge=1, le=50), db: Session = Depends(get_db)):
45     """해당 유저의 최근 거래 <limit>건을 실제로 분류해서 스와이프 큐로 반환합니다.
46
47     9/5 수정: limit 음수는 Postgres LIMIT에서 500을 넘고, 과대값은 실제 Gemini 키가 붙은
48     배포에서 LLM 호출이 폭증하므로 Query(1~50)로 제한합니다."""
49     user_row = db.execute(
50         text("SELECT persona FROM users WHERE user_id = :uid"),
51         {"uid": user_id},
52     ).mappings().first()
53     if not user_row:
54         raise HTTPException(status_code=404, detail="존재하지 않는 유저입니다.")
55     job = user_row["persona"] or ""
56
57     rows = db.execute(
58         text(
59             "SELECT merchant_name, amount, timestamp FROM transactions "
60             "WHERE user_id = :uid ORDER BY timestamp DESC LIMIT :limit"
61         ),
62         {"uid": user_id, "limit": limit},
63     ).mappings().all()

```

```
64     if not rows:
65         raise HTTPException(status_code=404, detail="해당 유저의 거래 내역이 없습니다.")
66
67     queue = []
68     total_refund = 0
69     for i, r in enumerate(rows):
70         amount = int(r["amount"])
71         result = classify_one(job, r["merchant_name"], r["timestamp"].hour, amount)
72         queue.append(
73             {
74                 "id": i + 1,
75                 "merchant": r["merchant_name"],
76                 "amount": amount,
77                 "ai_tag": result["ai_tag"],
78                 "prob": result["prob"],
79             }
80         )
81         if result["prob"] >= APPROVAL_THRESHOLD:
82             total_refund += int(amount * ASSUMED_MARGINAL_TAX_RATE)
83
84     return ExpenseClassifyResponse(transactions_queue=queue, current_estimated_refund=total_refund)
85
```

```

1  """
2  Gig-Score API - 9/2 완성판 (8/26 스텝을 설계서 기준 실제 지표로 교체)
3
4  - GET /api/v1/gig-score/{user_id}
5
6  원래 8/26 뼈대는 "거래가 얼마나 많고(activity)" + "지출이 얼마나 안정적인지(stability)"만
7  보는 자리표시자였고, 설계서에 있던 진짜 지표(세금 파킹 성실도, 업무 고정비 유지율,
8  플랫폼별 정산 완료율)는 "8/30-31에 교체 예정"이라고만 적혀 있었는데 끝까지 스텝으로
9  남아있었습니다. 9/2에 수입 플랫폼 연결 기능(income_sources/income_events, 9/2 추가)이
10 생기면서 아래처럼 실제 지표로 교체합니다 - activity(거래 건수)는 "돈을 얼마나 많이
11 버는가"가 아니라 그냥 "데이터가 얼마나 쌓였는가"라 신용 지표로 쓰기엔 부적절해서 뺐습니다.
12
13 1. 플랫폼 연결 성실도 (platform_connect_rate): 등록 가능한 수입 플랫폼 중 실제로
14   "연결"해서 추적 중인 비율. N잡러가 자기 수입원을 얼마나 빠짐없이 챙기고 관리하는지의
15   대리 지표입니다 - income.py의 연결 흐름과 바로 이어집니다.
16 2. 지출 안정성 (spending_stability): 8/26 산식 그대로 유지 - 거래 금액의 변동계수
17   (표준편차/평균)가 낮을수록(지출이 예측 가능할수록) 높은 점수.
18 3. 업무경비 인정을 (expense_diligence): "세금 파킹 성실도"를 새 ML 없이 가장 정직하게
19   근사한 값입니다. data_pipeline/expense_classifier.py(조소현님이 만든 AI 절세장부
20   분류 로직 - app/routers/expense.py가 이미 프로덕션에서 쓰고 있는 바로 그 코드)로
21   최근 거래 15건을 분류해서, 업무 경비로 인정될 확률(prob)이 APPROVAL_THRESHOLD(50)
22   이상인 비율을 씁니다. 경비 증빙을 꾸준히 만들어두는 사람일수록 실제 세금 신고 때
23   유리하다는 논리를 그대로 점수화한 것입니다.
24
25  가중치는 세 지표를 동등에 가깝게(0.35/0.35/0.30) 반영합니다. 1000점 만점.
26  """
27  from fastapi import APIRouter, Depends, HTTPException
28  from sqlalchemy import text
29  from sqlalchemy.orm import Session
30
31  from app.database import get_db
32  from app.models import GigScoreResponse
33  from app.security import verify_api_key
34
35  try:
36      from data_pipeline.expense_classifier import APPROVAL_THRESHOLD, classify_one
37  except ImportError: # uvicorn을 data_pipeline 폴더 안에서 띄우는 등 경로가 다를 때 대비
38      from expense_classifier import APPROVAL_THRESHOLD, classify_one # type: ignore
39
40  router = APIRouter(prefix="/api/v1/gig-score", tags=["gig-score"], dependencies=[Depends(verify_api_key)])
41
42
43  def _platform_connect_rate(db: Session, user_id: str) -> float:
44      """연결 가능한 수입 플랫폼 중 실제로 연결된 비율. income_sources가 아예 없는
45      유저(대모 5명 외 일반 시드 유저)는 이 지표를 계산할 수 없으니 None 취급으로
46      0.0을 반환합니다(가중치는 그대로 두되, 나머지 두 지표로 점수를 매깁니다)."""
47      row = db.execute(
48          text(
49              "SELECT count(*) AS total, count(*) FILTER (WHERE connected) AS connected "
50              "FROM income_sources WHERE user_id = :uid"
51          ),
52          {"uid": user_id},
53      ).mappings().first()
54      if not row or not row["total"]:
55          return 0.0
56      return row["connected"] / row["total"]
57
58
59  def _expense_diligence(db: Session, user_id: str) -> float:
60      """최근 거래 15건을 AI 절세장부 로직으로 분류해서, 업무경비로 인정될 확률이
61      임계값 이상인 비율을 계산합니다. app/routers/expense.py의 classify_expense()와
62      똑같은 호출 방식입니다(거래 없으면 0.0)."""
63      user_row = db.execute(

```

```

64     text("SELECT persona FROM users WHERE user_id = :uid"), {"uid": user_id}
65 ).mappings().first()
66 job = (user_row["persona"] or "") if user_row else ""
67
68 rows = db.execute(
69     text(
70         "SELECT merchant_name, amount, timestamp FROM transactions "
71         "WHERE user_id = :uid ORDER BY timestamp DESC LIMIT 15"
72     ),
73     {"uid": user_id},
74 ).mappings().all()
75 if not rows:
76     return 0.0
77
78 approved = 0
79 for r in rows:
80     result = classify_one(job, r["merchant_name"] or "", r["timestamp"].hour, int(r["amount"]))
81     if result["prob"] >= APPROVAL_THRESHOLD:
82         approved += 1
83 return approved / len(rows)
84
85
86 @router.get("/{user_id}", response_model=GigScoreResponse)
87 def get_gig_score(user_id: str, db: Session = Depends(get_db)):
88     profile = db.execute(
89         text("SELECT * FROM user_risk_profiles WHERE user_id = :uid"), {"uid": user_id}
90 ).mappings().first()
91 if not profile:
92     raise HTTPException(status_code=404, detail="유저 프로파일이 없습니다.")
93
94 # 변동계수(표준편차/평균)가 낮을수록 "지출이 규칙적"이라고 보고 안정성 점수를 높게 줍니다.
95 #
96 # 9/5 수정 - 원래 산식은 `1 - min(cv, 1)`이었는데, 거래 금액 분포는 원래 편차가 커서
97 # (커피 4천원과 월세 50만원이 같은 목록에 있음) 변동계수가 거의 항상 1을 넘습니다.
98 # 그 결과 데모 페르소나 5명 전원이 이 지표에서 정확히 0.0을 받아, 점수의 35%가
99 # 통째로 죽은 채 총점이 293~433점(1000점 만점)으로 낮게 깔리고 사람 간 변별력도
100 # 전혀 없었습니다. 1을 넘으면 무조건 0이 되는 절벽 대신, 변동계수가 커질수록
101 # 완만하게 낮아지는 1/(1+cv)로 바꿉니다 (cv=0.5→0.67, cv=1→0.5, cv=2→0.33).
102 avg_amt = float(profile["avg_transaction_amount"] or 0)
103 std_amt = float(profile["std_transaction_amount"] or 0)
104 stability = 1.0
105 if avg_amt > 0:
106     cv = std_amt / avg_amt
107     stability = 1.0 / (1.0 + max(0.0, cv))
108
109 connect_rate = _platform_connect_rate(db, user_id)
110 expense_diligence = _expense_diligence(db, user_id)
111
112 score = int((connect_rate * 0.35 + stability * 0.35 + expense_diligence * 0.30) * 1000)
113
114 return GigScoreResponse(
115     user_id=user_id,
116     score=score,
117     components={
118         "platform_connect_rate": round(connect_rate, 2),
119         "spending_stability": round(stability, 2),
120         "expense_diligence": round(expense_diligence, 2),
121     },
122     message="수입 플랫폼 연결 성실도 + 지출 안정성 + 업무경비 인정을 기반 종합 점수입니다.",
123 )
124

```

```

1  """
2  대응 가이드 API (9/6 신규) - GET /api/v1/guidance/{user_id}
3
4  무엇을 하나
5  -----
6  계산된 현금흐름·세금 상태를 보고 "지금 이 사람이 고려할 만한 선택지"를 근거와 함께 제시합니다.
7  잔고가 부족할 것으로 보이면 부족을 메우는 순서를, 여유가 있으면 먼저 떼어둘 돈을 안내합니다.
8
9  왜 '상품 추천'이 아니라 '선택지 안내'인가
10 -----
11 처음 검토한 형태는 "여유자금이면 투자상품, 부족하면 대출 상품을 추천"이었는데, 그대로 만들지
12 않았습니다.
13
14 - 자본시장법상 투자권유는 등록된 금융투자업자만 할 수 있고, 대출 모집도 대출모집인 등록이
15 필요합니다. 미등록 서비스가 특정 상품을 권유하는 형태는 위법 소지가 있습니다.
16 - 무엇보다 잔고가 부족할 것으로 예측된 사용자에게 곧바로 고금리 신용대출을 권하는 흐름은
17 이용자에게 해가 될 수 있습니다. 금융소비자보호 관점에서 이 서비스가 취해야 할 태도는
18 "돈을 빌리라"가 아니라 "무엇부터 확인하고, 어떤 순서로 대응할 수 있는지"입니다.
19
20 그래서 이 API는 **특정 금융상품·금융회사를 지목하지 않습니다.** 대신
21 (1) 지금 상태가 왜 그런지 계산 근거를 들고,
22 (2) 비용이 낮은 선택지부터 순서대로 놓고,
23 (3) 각 선택지의 주의점을 함께 적습니다.
24 대출 같은 선택지도 숨기지 않고 마지막 순서에 두고, 조건을 반드시 확인하라는 주의를 붙입니다.
25
26 이 구성은 "맞춤형 금융 서비스"라는 주제에 부합하면서도, 미등록 권유·부채 유도라는 위험을
27 만들지 않습니다.
28 """
29 from fastapi import APIRouter, Depends, HTTPException
30 from sqlalchemy.orm import Session
31
32 from app.cashflow import compute_shield
33 from app.database import get_db
34 from app.security import verify_api_key
35
36 router = APIRouter(prefix="/api/v1/guidance", tags=["guidance"], dependencies=[Depends(verify_api_key)])
37
38 DISCLAIMER = (
39     "특정 금융상품이나 금융회사를 권유하지 않으며, 투자권유·대출모집에 해당하지 않습니다. "
40     "계산된 추정치를 바탕으로 한 일반적인 정보 안내이며, 실제 결정 전 해당 기관에 조건을 확인하세요."
41 )
42
43
44 def _man(v) -> str:
45     return format(int(round((v or 0) / 10000)), ",") + "만원"
46
47
48 @router.get("/{user_id}")
49 def get_guidance(user_id: str, db: Session = Depends(get_db)):
50     shield = compute_shield(db, user_id)
51     if shield is None:
52         raise HTTPException(status_code=404, detail="아직 계산할 수입·지출 데이터가 없습니다.")
53
54     tax = None
55     try:
56         from app.routers.tax import estimate as tax_estimate
57
58         tax = tax_estimate(user_id, db)
59     except Exception:
60         tax = None
61
62     shortage_date = shield.get("minimum_balance_date")
63     available = shield.get("available_cash") or 0

```

```

64     reserve = shield.get("recommended_reserve") or 0
65     items: list[dict] = []
66
67     if shortage_date:
68         # 부족이 예상되는 경우 - 비용이 낮은 순서로 배치합니다.
69         items.append(
70             {
71                 "tone": "warn",
72                 "title": "정산 예정일을 먼저 확인하세요",
73                 "why": shortage_date + "에 잔고가 최소 안전잔액 아래로 내려갈 것으로 계산됩니다.",
74                 "options": [
75                     "연결한 플랫폼의 다음 정산 예정일과 금액을 확인하고, 부족 시점보다 앞선 정산이 있는지 봅니다.",
76                     "플랫폼에 따라 정산 주기를 앞당기거나 조기 정산을 신청할 수 있는 경우가 있습니다.",
77                     "아직 연결하지 않은 수입원이 있다면 연결해 예측에 반영합니다.",
78                 ],
79                 "caution": "조기 정산은 수수료가 붙는 경우가 있으니 조건을 먼저 확인하세요.",
80             }
81         )
82     items.append(
83         {
84             "tone": "warn",
85             "title": "고정 지출 시점을 옮길 수 있는지 봅니다",
86             "why": "부족은 대개 '수입이 오기 전에 고정비가 먼저 빠져나가서' 생깁니다.",
87             "options": [
88                 "구독·보험료 등 자동이체 날짜를 정산일 이후로 옮기면 부족 구간을 피할 수 있습니다.",
89                 "이번 달 예산 초과 카테고리의 지출을 정산일까지 미룹니다.",
90             ],
91             "caution": "이체일 변경은 연체로 처리되지 않도록 기관에 미리 신청해야 합니다.",
92         }
93     )
94     items.append(
95         {
96             "tone": "neutral",
97             "title": "그래도 모자란다면",
98             "why": "위 방법으로 메워지지 않는 금액이 남는 경우에 한해 고려할 수 있는 선택지입니다.",
99             "options": [
100                 "마이너스 통장·비상금 등 이미 보유한 여유 한도를 먼저 씁니다.",
101                 "정책 지원 제도(고용보험 가입 특수형태근로종사자 대상 지원, 소상공인 정책자금 등)에 "
102                 "해당되는지 확인합니다.",
103                 "신용대출·카드론은 금리와 상환 부담이 크므로 마지막 순서로 두고, 반드시 총 상환액을 "
104                 "계산한 뒤 결정합니다.",
105             ],
106             "caution": "이 서비스는 특정 상품을 권유하지 않습니다. 조건·금리는 해당 금융회사에서 직접 확인하세요.",
107         }
108     )
109 else:
110     items.append(
111         {
112             "tone": "safe",
113             "title": "먼저 세금·건강보험료로 떼어둘 돈을 확보하세요",
114             "why": "앞으로 45일간 잔고 부족은 예상되지 않습니다. 지금 여유는 "
115             + _man(available)
116             + "이고, 권장 준비금은 "
117             + _man(reserve)
118             + "입니다.",
119             "options": [
120                 "권장 준비금만큼을 생활비 계좌와 분리해 따로 둥니다.",
121                 "5월 종합소득세와 매달 나가는 건강보험료는 시차를 두고 청구되므로, 미리 떼어두면 "
122                 "그 시점의 현금흐름 충격을 줄일 수 있습니다.",
123             ],
124             "caution": "표시된 세액·보험료는 확정 금액이 아니라 추정치입니다.",
125         }
126     )
127     items.append(
128         {
129             "tone": "neutral",
130             "title": "그 다음 여유분의 운용",
131             "why": "준비금을 떼고도 남는 금액이 있다면 고려할 수 있습니다.",

```

```

132         "options": [
133             "수입이 불규칙한 만큼, 먼저 몇 달치 고정비에 해당하는 비상금을 확보하는 것이 일반적인 순서입니다.",
134             "그 이후의 자금 운용은 본인의 위험 성향과 자금 사용 시점에 따라 달라지므로, "
135             "금융회사나 자격을 갖춘 전문가와 상담해 결정하세요.",
136         ],
137         "caution": "이 서비스는 투자상품을 추천하거나 권유하지 않습니다.",
138     }
139 )
140
141 if tax:
142     hi = tax.get("health_insurance") or {}
143     if hi.get("warning"):
144         items.append(
145             {
146                 "tone": "warn",
147                 "title": "건강보험료 기준선을 확인하세요",
148                 "why": hi["warning"],
149                 "options": [
150                     "연말까지의 예상 소득을 확인해 기준을 넘는 시점을 미리 파악합니다.",
151                     "넘게 되는 경우 보험료가 새로 생기거나 늘어나므로, 그만큼을 미리 준비금에 반영합니다.",
152                 ],
153                 "caution": "실제 부과액은 소득 외 재산 등도 반영되어 달라질 수 있습니다.",
154             }
155         )
156     it = tax.get("income_tax") or {}
157     if isinstance(it.get("balance_due"), (int, float)) and it["balance_due"] < 0:
158         items.append(
159             {
160                 "tone": "safe",
161                 "title": "5월에 돌려받을 세금이 있습니다",
162                 "why": "원천징수된 3.3%가 추정 세액보다 많아 " + _man(-it["balance_due"]) + " 환급이 예상됩니다.",
163                 "options": [
164                     "종합소득세 신고를 해야 환급이 이뤄집니다. 신고하지 않으면 돌려받지 못합니다.",
165                     "업무 관련 지출 증빙을 모아두면 필요경비로 인정받아 환급액이 달라질 수 있습니다.",
166                 ],
167                 "caution": "추정치이며, 실제 환급액은 공제 항목에 따라 달라집니다.",
168             }
169         )
170
171 return {"user_id": user_id, "items": items, "disclaimer": DISCLAIMER}
172

```

```

1  """
2  수입 플랫폼 연결 API - 9/2 추가
3
4  배경: "N잡러는 여러 플랫폼에서 수입이 들쭉날쭉 들어와서 정신없다"는 게 이 서비스가
5  풀어야 할 핵심 문제인데, 지금까지는 지출(리스크/예산/절세) 기능만 있고 정작 "수입이
6  얼마나, 어디서 들어오는지"를 한눈에 보여주는 화면이 없었습니다. 실제 오픈뱅킹/마이데이터
7  연동은 이번 MVP 범위 밖이라, 계좌를 "연결"하면 그 플랫폼의 수입이 드러나는 흐름을
8  더미 데이터(scripts/seed_demo_income.py)로 재연합니다.
9
10 - GET /api/v1/income/sources : 이 유저의 연결 가능한 플랫폼 목록 (연결 여부 포함)
11 - POST /api/v1/income/sources/{id}/connect : "연결하기" - 이미 있던 데이터를 드러냄 (idempotent)
12 - POST /api/v1/income/sources/{id}/disconnect : "연결 해제" - 다시 숨김 (9/2 세 번째 업데이트,
13   기획서 8번 "사용자가 연결 해제·삭제·정정할 수 있는 화면" 요구사항 반영)
14 - GET /api/v1/income/summary : 한 장 요약 홈 카드가 그대로 쓰는 집계
15 - GET /api/v1/income/events : 최근 수입 이벤트 목록 (연결된 플랫폼만)
16 """
17 from datetime import timedelta
18
19 from fastapi import APIRouter, Depends, HTTPException, Query
20 from sqlalchemy import text
21 from sqlalchemy.orm import Session
22
23 from app.database import get_db
24 from app.demo_clock import get_demo_now
25 from app.models import (
26     IncomeByPlatform,
27     IncomeConnectResponse,
28     IncomeDisconnectResponse,
29     IncomeEvent,
30     IncomeSource,
31     IncomeSummary,
32 )
33 from app.security import verify_api_key
34
35 router = APIRouter(prefix="/api/v1/income", tags=["income"], dependencies=[Depends(verify_api_key)])
36
37
38 @router.get("/sources", response_model=list[IncomeSource])
39 def get_sources(user_id: str, db: Session = Depends(get_db)):
40     """이 유저에게 연결 가능한 플랫폼 전체 목록. 연결 안 된 것도 포함해서 보여줍니다
41     (연결 유도 화면이 "연결하면 이만큼 더 보여요"를 말해줄 수 있도록 건수만 미리 노출)."""
42     rows = db.execute(
43         text(
44             """
45             SELECT s.source_id, s.platform_name, s.platform_type, s.icon_emoji,
46                    s.connected, s.connected_at,
47                    (SELECT count(*) FROM income_events e WHERE e.source_id = s.source_id) AS pending_count
48             FROM income_sources s
49             WHERE s.user_id = :uid
50             ORDER BY s.connected DESC, s.created_at
51             """
52         ),
53         {"uid": user_id},
54     ).mappings().all()
55
56     if not rows:
57         raise HTTPException(status_code=404, detail="이 유저에게 연결 가능한 수입 플랫폼이 없습니다.")
58
59     return [
60         IncomeSource(
61             source_id=str(r["source_id"]),
62             platform_name=r["platform_name"],
63             platform_type=r["platform_type"],

```

```

64         icon_emoji=r["icon_emoji"],
65         connected=r["connected"],
66         connected_at=r["connected_at"],
67         pending_event_count=0 if r["connected"] else r["pending_count"],
68     )
69     for r in rows
70 ]
71
72
73 @router.post("/sources/{source_id}/connect", response_model=IncomeConnectResponse)
74 def connect_source(source_id: str, user_id: str, db: Session = Depends(get_db)):
75     """플랫폼 연결하기. 이미 연결돼있으면 그냥 현재 상태를 그대로 돌려줍니다(재시도해도 안전)."""
76     row = db.execute(
77         text(
78             "SELECT source_id, platform_name, connected FROM income_sources "
79             "WHERE source_id = :sid AND user_id = :uid"
80         ),
81         {"sid": source_id, "uid": user_id},
82     ).mappings().first()
83     if not row:
84         raise HTTPException(status_code=404, detail="존재하지 않는 수입 플랫폼입니다.")
85
86     already_connected = bool(row["connected"])
87     if not already_connected:
88         db.execute(
89             text("UPDATE income_sources SET connected = TRUE, connected_at = now() WHERE source_id = :sid"),
90             {"sid": source_id},
91         )
92         db.commit()
93
94     count_row = db.execute(
95         text("SELECT count(*) AS n FROM income_events WHERE source_id = :sid"), {"sid": source_id}
96     ).mappings().first()
97
98     return IncomeConnectResponse(
99         source_id=source_id,
100         platform_name=row["platform_name"],
101         connected=True,
102         newly_connected=not already_connected,
103         revealed_event_count=count_row["n"] if count_row else 0,
104     )
105
106
107 @router.post("/sources/{source_id}/disconnect", response_model=IncomeDisconnectResponse)
108 def disconnect_source(source_id: str, user_id: str, db: Session = Depends(get_db)):
109     """연결 해제. 실제 계좌 연동이라면 이 시점에 마이데이터 접근 권한을 회수하는
110     동작에 해당합니다 - 이 데모에서는 connected를 다시 false로 돌려서, 그 플랫폼의
111     수입이 요약/시물레이션(app/cashflow.py)에서 다시 제외되도록 합니다. 이미 연결 해제된
112     상태에 다시 호출해도 안전합니다(idempotent)."""
113     row = db.execute(
114         text(
115             "SELECT source_id, platform_name FROM income_sources "
116             "WHERE source_id = :sid AND user_id = :uid"
117         ),
118         {"sid": source_id, "uid": user_id},
119     ).mappings().first()
120     if not row:
121         raise HTTPException(status_code=404, detail="존재하지 않는 수입 플랫폼입니다.")
122
123     db.execute(
124         text("UPDATE income_sources SET connected = FALSE, connected_at = NULL WHERE source_id = :sid"),
125         {"sid": source_id},
126     )
127     db.commit()
128
129     return IncomeDisconnectResponse(source_id=source_id, platform_name=row["platform_name"], connected=False)
130
131

```

```

132 @router.get("/summary", response_model=IncomeSummary)
133 def get_summary(user_id: str, period: str = "month", db: Session = Depends(get_db)):
134     """연결된 플랫폼만 집계해서 '한 장 요약' 홈 카드용 데이터를 만듭니다.
135     period: "week"(최근 7일) / "month"(이번 달, 1일부터)."""
136     demo_now = get_demo_now(db)
137     if period == "week":
138         period_start = demo_now - timedelta(days=7)
139         period_label = "이번 주"
140     else:
141         period_start = demo_now.replace(day=1)
142         period_label = "이번 달"
143
144     rows = db.execute(
145         text(
146             """
147             SELECT s.platform_name, s.platform_type, s.icon_emoji,
148                    e.amount, e.status
149             FROM income_events e
150             JOIN income_sources s ON s.source_id = e.source_id
151             WHERE s.user_id = :uid AND s.connected = TRUE AND e.settled_at >= :start
152             """
153         ),
154         {"uid": user_id, "start": period_start},
155     ).mappings().all()
156
157     by_platform: dict[str, IncomeByPlatform] = {}
158     total_settled = 0.0
159     total_upcoming = 0.0
160     for r in rows:
161         amt = float(r["amount"])
162         if r["status"] == "정산완료":
163             total_settled += amt
164         else:
165             total_upcoming += amt
166         key = r["platform_name"]
167         if key not in by_platform:
168             by_platform[key] = IncomeByPlatform(
169                 platform_name=r["platform_name"],
170                 platform_type=r["platform_type"],
171                 icon_emoji=r["icon_emoji"],
172                 total_amount=0.0,
173                 event_count=0,
174             )
175         by_platform[key].total_amount += amt
176         by_platform[key].event_count += 1
177
178     unconnected_row = db.execute(
179         text("SELECT count(*) AS n FROM income_sources WHERE user_id = :uid AND connected = FALSE"),
180         {"uid": user_id},
181     ).mappings().first()
182
183     return IncomeSummary(
184         user_id=user_id,
185         period_label=period_label,
186         total_settled=total_settled,
187         total_upcoming=total_upcoming,
188         by_platform=sorted(by_platform.values(), key=lambda p: -p.total_amount),
189         unconnected_source_count=unconnected_row["n"] if unconnected_row else 0,
190     )
191
192
193 @router.get("/events", response_model=list[IncomeEvent])
194 def get_events(user_id: str, limit: int = Query(20, ge=1, le=200), db: Session = Depends(get_db)):
195     """최근 수입 이벤트 목록 (연결된 플랫폼만, 최신순).
196
197     9/5 수정: limit에 음수가 들어오면 Postgres LIMIT에서 500이 나던 걸 Query(1~200)로 막습니다."""
198     rows = db.execute(
199         text(

```

```

200         """
201         SELECT s.platform_name, s.icon_emoji, e.amount, e.memo, e.status, e.settled_at
202         FROM income_events e
203         JOIN income_sources s ON s.source_id = e.source_id
204         WHERE s.user_id = :uid AND s.connected = TRUE
205         ORDER BY e.settled_at DESC
206         LIMIT :limit
207         """
208     ),
209     {"uid": user_id, "limit": limit},
210 ).mappings().all()
211 return [
212     IncomeEvent(
213         platform_name=r["platform_name"],
214         icon_emoji=r["icon_emoji"],
215         amount=float(r["amount"]),
216         memo=r["memo"],
217         status=r["status"],
218         settled_at=r["settled_at"],
219     )
220     for r in rows
221 ]
222

```

```

1  """
2  리스크방어 API
3  - POST /api/v1/risk/assess : 새 거래 하나를 평가 (룰 엔진 실행 + risk_events 기록)
4  - GET /api/v1/risk/events : 리스크 이벤트 목록 조회
5  - GET /api/v1/risk/score/{user_id} : 유저의 최근 리스크 점수 조회
6  """
7  from datetime import datetime, timedelta
8
9  from fastapi import APIRouter, Depends, HTTPException, Query
10 from sqlalchemy import text
11 from sqlalchemy.orm import Session
12
13 from app.anomaly_model import ANOMALY_HIGH_THRESHOLD, ANOMALY_ALERT_THRESHOLD, score_anomaly
14 from app.database import get_db
15 from app.models import RiskAssessment, RuleAction, RuleResult, Severity, Transaction, UserRiskProfile
16 from app.rule_engine import RuleEngine, rule_results_to_score
17 from app.security import verify_api_key
18
19 router = APIRouter(prefix="/api/v1/risk", tags=["risk"], dependencies=[Depends(verify_api_key)])
20
21 # 심각도/액션 랭킹표. 여러 규칙이 동시에 걸렸을 때 "가장 심각한 것" 하나를 정확히
22 # 골라내기 위한 기준입니다.
23 # (버그 수정: 기존 코드는 `max(results, key=lambda r: r.severity)`로 Severity를
24 # 알파벳 문자열 순으로 비교했습니다. "MEDIUM" > "HIGH" > "CRITICAL" 순서라서,
25 # HIGH와 CRITICAL이 동시에 걸려도 최종 severity가 "MEDIUM"으로 나올 수 있었습니다.
26 # action도 `results[0].action`이라 규칙 정의 순서에 우연히 의존했습니다.)
27 _SEVERITY_RANK = {Severity.LOW: 0, Severity.MEDIUM: 1, Severity.HIGH: 2, Severity.CRITICAL: 3}
28 _ACTION_RANK = {
29     RuleAction.ALLOW: 0,
30     RuleAction.FLAG_FOR_REVIEW: 1,
31     RuleAction.REQUIRE_CONFIRMATION: 2,
32     RuleAction.BLOCK: 3,
33 }
34
35
36 def _load_profile(db: Session, user_id: str) -> UserRiskProfile:
37     row = db.execute(
38         text("SELECT * FROM user_risk_profiles WHERE user_id = :uid"), {"uid": user_id}
39     ).mappings().first()
40     if not row:
41         # 프로파일이 아직 없으면 기본값으로 평가 (신규 유저)
42         return UserRiskProfile(user_id=user_id)
43     return UserRiskProfile(
44         user_id=row["user_id"],
45         avg_transaction_amount=float(row["avg_transaction_amount"]),
46         std_transaction_amount=float(row["std_transaction_amount"]),
47         single_transaction_limit=float(row["single_transaction_limit"]),
48         daily_limit=float(row["daily_limit"]),
49         avg_daily_transactions=float(row["avg_daily_transactions"]),
50         allowed_countries=list(row["allowed_countries"] or ["KR"]),
51     )
52
53
54 def _load_recent_history(db: Session, user_id: str, before: datetime, days: int = 180) -> list[Transaction]:
55     """
56     <before> 시점(=평가 대상 거래의 timestamp) 기준으로 과거 <days>일 이력을 가져옵니다.
57
58     🐛 8/28 버그 수정: 원래는 `datetime.utcnow() - timedelta(days=7)`로, "지금 서버 시각"
59     기준이었습니다. mock_transactions.csv의 실제 데이터는 2026-05-25 ~ 2026-08-22인데,
60     이 서버가 실행되는 "지금"(예: 8/28)과 비교하면 7일 이내로 걸리는 데이터가 사실상 없어서
61     R003(신규가맹점)/R004(불가능한 이동속도)/R006(단기반복)이 실제 DB 데이터로는 거의 항상
62     빈 이력을 받아 오작동했습니다(항상 "이력 없음"으로 취급). 평가 대상 거래 자신의 시각을
63     기준으로 바꾸고, 윈도우도 180일로 넓혔습니다 - Isolation Forest 피쳐(작전 거래와의 거리,

```

```

64 가맹점 방문 빈도, 기기 이력)도 이 함수가 반환하는 history를 그대로 씁니다.
65 """
66 since = before - timedelta(days=days)
67 rows = db.execute(
68     text(
69         "SELECT * FROM transactions WHERE user_id = :uid "
70         "AND timestamp >= :since AND timestamp < :before "
71         "ORDER BY timestamp"
72     ),
73     {"uid": user_id, "since": since, "before": before},
74 ).mappings().all()
75 return [
76     Transaction(
77         transaction_id=str(r["transaction_id"]),
78         user_id=r["user_id"],
79         amount=float(r["amount"]),
80         merchant_id=r["merchant_id"],
81         merchant_name=r["merchant_name"],
82         mcc_code=r["mcc_code"],
83         category=r["category"],
84         timestamp=r["timestamp"],
85         latitude=r["latitude"],
86         longitude=r["longitude"],
87         country=r["country"] or "KR",
88         device_id=r["device_id"],
89     )
90     for r in rows
91 ]
92
93
94 @router.post("/assess", response_model=RiskAssessment)
95 def assess_transaction(txn: Transaction, db: Session = Depends(get_db)):
96     """
97     새 거래를 평가하고, 규칙(또는 AI 이상탐지)에 걸리면 risk_events에 기록합니다.
98
99     8/28: 룰 엔진(rule_prob) + Isolation Forest(anomaly_prob, app/anomaly_model.py)를
100     "확률적 OR"로 앙상블합니다: combined = 1 - (1-rule_prob)*(1-anomaly_prob).
101     두 탐지기 중 하나라도 강하게 의심하면 최종 점수가 올라갑니다. 룰이 이미 CRITICAL이면
102     (rule_prob=1.0) AI 결과와 무관하게 그대로 100점 - 기존 룰 기반 동작은 전혀 안 바뀝니다.
103     반대로 룰이 하나도 안 걸렸는데 AI가 확신하면 (ANOMALY_ALERT_THRESHOLD 이상), 룰 엔진이
104     원천적으로 설계되지 않은 유형(R00x_위치급변/R00y_신규기기 같은)도 "AI001"이라는 가상의
105     rule_id로 risk_events에 남습니다 - 룰 목록에 없다고 조용히 묻히지 않게 하려는 것입니다.
106     """
107     # 9/5: 본문으로 들어오는 user_id는 미들웨어(app/demo_guard.py)가 검사하지 못하므로
108     # 여기서 직접 막습니다 - 임의의 user_id로 남의 이력에 평가 기록을 남길 수 없게.
109     from app.demo_guard import allowed_user_ids
110
111     if txn.user_id not in allowed_user_ids():
112         raise HTTPException(status_code=403, detail="이 시연 배포는 공개된 데모 계정만 평가할 수 있습니다.")
113
114     profile = _load_profile(db, txn.user_id)
115     history = _load_recent_history(db, txn.user_id, before=txn.timestamp)
116
117     engine = RuleEngine(history=history)
118     results = engine.evaluate(txn, profile)
119     rule_prob = rule_results_to_score(results)
120
121     anomaly_prob = score_anomaly(txn, profile, history)
122     if not results and anomaly_prob >= ANOMALY_ALERT_THRESHOLD:
123         ai_severity = Severity.HIGH if anomaly_prob >= ANOMALY_HIGH_THRESHOLD else Severity.MEDIUM
124         ai_action = (
125             RuleAction.REQUIRE_CONFIRMATION if anomaly_prob >= ANOMALY_HIGH_THRESHOLD
126             else RuleAction.FLAG_FOR_REVIEW
127         )
128         results = [
129             RuleResult(
130                 rule_id="AI001",
131                 severity=ai_severity,

```

```

132         description=(
133             f"AI 이상탐지 모델이 이 거래를 이상 패턴으로 판단했습니다 "
134             f"(anomaly_prob={anomaly_prob:.2f}). 걸린 규칙은 없지만 과거 패턴과 크게 달라요."
135         ),
136         action=ai_action,
137     )
138 ]
139
140 combined_prob = 1 - (1 - rule_prob) * (1 - anomaly_prob)
141 score = int(round(combined_prob * 100))
142
143 if results:
144     severity = max(results, key=lambda r: _SEVERITY_RANK[r.severity]).severity
145     action = max(results, key=lambda r: _ACTION_RANK[r.action]).action
146     for r in results:
147         db.execute(
148             text(
149                 "INSERT INTO risk_events "
150                 "(transaction_id, user_id, rule_id, severity, description, action, score) "
151                 "VALUES (:tid, :uid, :rid, :sev, :desc, :act, :score)"
152             ),
153             {
154                 "tid": txn.transaction_id,
155                 "uid": txn.user_id,
156                 "rid": r.rule_id,
157                 "sev": r.severity.value,
158                 "desc": r.description,
159                 "act": r.action.value,
160                 "score": score,
161             },
162         )
163     db.commit()
164 else:
165     severity = Severity.LOW
166     action = RuleAction.ALLOW
167
168 return RiskAssessment(
169     user_id=txn.user_id,
170     transaction_id=txn.transaction_id,
171     score=score,
172     severity=severity,
173     action=action,
174     triggered_rules=results,
175 )
176
177
178 @router.get("/events")
179 def list_risk_events(
180     user_id: str,
181     limit: int = Query(50, ge=1, le=200),
182     db: Session = Depends(get_db),
183 ):
184     """
185     8/30: 프론트(리스크 타임라인 화면)가 가맹점명/금액도 같이 보여줘야 해서, transactions와
186     LEFT JOIN해서 merchant_name/amount/timestamp도 같이 내려줍니다 (risk_events 자체엔
187     없는 컬럼이라, 이전엔 rule_id/severity만으로는 "무슨 거래였는지" 알 수 없었습니다).
188
189     9/5 보안 수정: 예전에는 user_id가 선택값이라 아예 안 넣기면 미들웨어(app/demo_guard.py)의
190     user_id 검사를 우회해서 "전체 유저의 risk_events"가 무인증으로 통째로 조회됐습니다
191     (데모 3명이 아닌 시드 유저까지 포함 - 전형적인 IDOR). user_id를 필수로 바꾸고, 여기서도
192     직접 데모 페르소나 화이트리스트로 한 번 더 막습니다. limit도 음수/과대값이 들어오면
193     Postgres LIMIT에서 500이 나던 걸 Query 제약(1~200)으로 막습니다.
194     """
195     from app.demo_guard import allowed_user_ids
196
197     if user_id not in allowed_user_ids():
198         raise HTTPException(
199             status_code=403,

```

```

200         detail="이 시연 배포는 공개된 데모 계정의 데이터만 조회할 수 있습니다.",
201     )
202
203     query = (
204         "SELECT re.*, t.merchant_name, t.amount AS transaction_amount, t.timestamp AS transaction_timestamp "
205         "FROM risk_events re LEFT JOIN transactions t ON t.transaction_id = re.transaction_id "
206         "WHERE re.user_id = :uid ORDER BY re.created_at DESC LIMIT :limit"
207     )
208     rows = db.execute(text(query), {"uid": user_id, "limit": limit}).mappings().all()
209     return [dict(r) for r in rows]
210
211
212 @router.get("/score/{user_id}")
213 def get_latest_score(user_id: str, db: Session = Depends(get_db)):
214     # 참고: risk_events는 규칙 하나당 한 행이라, 같은 거래에서 여러 규칙이
215     # 동시에 걸리면 created_at이 동일한 행이 여러 개 생깁니다. created_at만으로
216     # 정렬하면 그중 어느 게 뽐힐지 보장이 안 돼서, 심각도 랭킹을 2차 정렬 기준으로
217     # 추가했습니다 (동시간대면 가장 심각한 것 우선).
218     row = db.execute(
219         text(
220             # 8/30: action도 같이 내려줍니다 - 프론트가 "본인 확인이 필요해요" 같은
221             # 안내 문구를 severity가 아니라 실제 action(ALLOW/FLAG_FOR_REVIEW/...)
222             # 기준으로 보여줘야 해서 필요합니다.
223             "SELECT score, severity, action, created_at FROM risk_events "
224             "WHERE user_id = :uid ORDER BY created_at DESC, "
225             "CASE severity "
226             "    WHEN 'CRITICAL' THEN 4 WHEN 'HIGH' THEN 3 "
227             "    WHEN 'MEDIUM' THEN 2 WHEN 'LOW' THEN 1 ELSE 0 END DESC "
228             "LIMIT 1"
229         ),
230         {"uid": user_id},
231     ).mappings().first()
232     if not row:
233         return {"user_id": user_id, "score": 0, "severity": "LOW", "action": "ALLOW", "message": "이력 없음"}
234     return {"user_id": user_id, **dict(row)}
235

```

```

1  """
2  Risk Shield / Dynamic Parking API (9/2 세 번째 업데이트)
3
4  기획서 4번의 대표 차별화 기능인 "Risk Shield"(일별 예상 잔액·최저잔액·현금 부족
5  예상일)와 "Dynamic Parking"(세금·건보료 준비 권장액)을 실제로 구현합니다.
6
7  - GET  /api/v1/shield/{user_id}          : 현재 상태 + 45일 시뮬레이션
8  - GET  /api/v1/shield/{user_id}/scenarios : 기본/입금지연/수입감소 3개 시나리오
9  - GET  /api/v1/shield/{user_id}/settings  : 최소 안전잔액 조회
10 - PUT  /api/v1/shield/{user_id}/settings  : 최소 안전잔액 설정(온보딩)
11
12 이름이 비슷해서 헷갈릴 수 있는데, `/api/v1/risk`(app/routers/risk.py)는 카드
13 이상거래/사기 탐지 룰엔진이고, 이 파일의 `/api/v1/shield`는 완전히 다른 개념인
14 "현금흐름이 바닥나기 전에 미리 경고하는" 기능입니다. 계산 로직은 app/cashflow.py에
15 전부 있고, 이 라우터는 앞게 그 위에 얹은 API 계층입니다.
16 """
17 from datetime import datetime
18
19 from fastapi import APIRouter, Depends, HTTPException
20 from sqlalchemy import text
21 from sqlalchemy.orm import Session
22
23 from app.cashflow import DEFAULT_MIN_SAFETY_BALANCE, compute_scenarios, compute_shield
24 from app.database import get_db
25 from app.models import SafetyBalanceResponse, SafetyBalanceUpdate, ScenarioResponse, ShieldResponse
26 from app.security import verify_api_key
27
28 router = APIRouter(prefix="/api/v1/shield", tags=["shield"], dependencies=[Depends(verify_api_key)])
29
30
31 @router.get("/{user_id}", response_model=ShieldResponse)
32 def get_shield(user_id: str, db: Session = Depends(get_db)):
33     result = compute_shield(db, user_id)
34     if result is None:
35         raise HTTPException(
36             status_code=404,
37             detail="아직 수입/지출 데이터가 없어서 Risk Shield를 계산할 수 없습니다.",
38         )
39     return result
40
41
42 @router.get("/{user_id}/scenarios", response_model=ScenarioResponse)
43 def get_scenarios(user_id: str, db: Session = Depends(get_db)):
44     result = compute_scenarios(db, user_id)
45     if result is None:
46         raise HTTPException(
47             status_code=404,
48             detail="아직 수입/지출 데이터가 없어서 시나리오를 계산할 수 없습니다.",
49         )
50     return result
51
52
53 @router.get("/{user_id}/settings", response_model=SafetyBalanceResponse)
54 def get_settings(user_id: str, db: Session = Depends(get_db)):
55     row = db.execute(
56         text("SELECT min_safety_balance FROM user_settings WHERE user_id = :uid"),
57         {"uid": user_id},
58     ).mappings().first()
59     if row and row["min_safety_balance"] is not None:
60         return {"user_id": user_id, "min_safety_balance": float(row["min_safety_balance"]), "is_default": False}
61     return {"user_id": user_id, "min_safety_balance": float(DEFAULT_MIN_SAFETY_BALANCE), "is_default": True}
62
63

```

```
64 @router.put("/{user_id}/settings", response_model=SafetyBalanceResponse)
65 def update_settings(user_id: str, body: SafetyBalanceUpdate, db: Session = Depends(get_db)):
66     exists = db.execute(text("SELECT 1 FROM users WHERE user_id = :uid"), {"uid": user_id}).first()
67     if not exists:
68         raise HTTPException(status_code=404, detail="존재하지 않는 user_id 입니다.")
69     db.execute(
70         text(
71             "INSERT INTO user_settings (user_id, min_safety_balance, updated_at) "
72             "VALUES (:uid, :bal, :now) "
73             "ON CONFLICT (user_id) DO UPDATE SET min_safety_balance = :bal, updated_at = :now"
74         ),
75         {"uid": user_id, "bal": body.min_safety_balance, "now": datetime.utcnow()},
76     )
77     db.commit()
78     return {"user_id": user_id, "min_safety_balance": body.min_safety_balance, "is_default": False}
79
```

```

1  """
2  직접 입력 시뮬레이션 API (9/6 신규) - POST /api/v1/simulate/cashflow
3
4  무엇을 하나
5  -----
6  시연용 인물의 데이터가 아니라, **보는 사람이 직접 입력한 수입·지출 조건**으로 45일 현금흐름을
7  계산합니다. 정산 주기와 금액만 넣으면 잔고가 언제 최소 안전선을 밑도는지, 정산이 밀리거나
8  수입이 줄면 그 시점이 며칠 앞당겨지는지를 그대로 돌려줍니다.
9
10 왜 필요한가
11 -----
12 지금까지 이 서비스는 준비된 인물 3명을 **구경하는** 형태였습니다. 그러면 보는 사람은
13 "이 사람들 데이터니까 이렇게 나오겠지"에서 멈추고, 자기 상황에 대입해볼 방법이 없습니다.
14 숫자를 직접 넣어 결과가 바뀌는 걸 보는 순간 예측이 남의 것에서 자기 것이 됩니다.
15
16 계산 방식은 기존 엔진(app/cashflow.py)과 동일한 논리를 씁니다 - 정산 주기마다 수입을 배치하고,
17 고정비는 매달 같은 날, 변동비는 매일 균등하게 빼면서 일별 잔액을 누적합니다. DB를 쓰지 않으므로
18 입력값은 저장되지 않고, 계산 결과만 응답으로 돌려줍니다(개인정보를 남기지 않기 위함).
19 """
20 from datetime import date, timedelta
21
22 from fastapi import APIRouter, Depends
23 from pydantic import BaseModel, Field
24
25 from app.security import verify_api_key
26
27 router = APIRouter(prefix="/api/v1/simulate", tags=["simulate"], dependencies=[Depends(verify_api_key)])
28
29 HORIZON_DAYS = 45
30 MAX_SOURCES = 5
31
32
33 class IncomeSourceInput(BaseModel):
34     """수입원 하나. '며칠마다 얼마씩 들어오는가'만 받습니다."""
35
36     name: str = Field(default="수입원", max_length=30)
37     cycle_days: int = Field(ge=1, le=90)
38     amount: float = Field(ge=0, le=100_000_000)
39     days_until_next: int = Field(default=0, ge=0, le=90)
40
41
42 class SimulationInput(BaseModel):
43     current_balance: float = Field(ge=0, le=1_000_000_000)
44     min_safety_balance: float = Field(default=300_000, ge=0, le=100_000_000)
45     fixed_monthly_cost: float = Field(default=0, ge=0, le=100_000_000)
46     fixed_cost_day: int = Field(default=25, ge=1, le=28)
47     daily_variable_cost: float = Field(default=0, ge=0, le=10_000_000)
48     sources: list[IncomeSourceInput] = Field(default_factory=list, max_length=MAX_SOURCES)
49
50
51 def _run(inp: SimulationInput, delay_days: int = 0, income_ratio: float = 1.0) -> list[dict]:
52     """하루씩 잔액을 굴립니다. delay_days는 정산 지연, income_ratio는 수입 감소 가정입니다."""
53     today = date.today()
54     balance = float(inp.current_balance)
55     points = []
56
57     # 각 수입원의 다음 정산일을 미리 배치해둡니다.
58     upcoming = {}
59     for idx, src in enumerate(inp.sources):
60         upcoming[idx] = src.days_until_next + delay_days
61
62     for day in range(HORIZON_DAYS + 1):
63         current = today + timedelta(days=day)

```

```

64     events = []
65
66     if day > 0:
67         balance -= inp.daily_variable_cost
68         if current.day == inp.fixed_cost_day and inp.fixed_monthly_cost:
69             balance -= inp.fixed_monthly_cost
70             events.append("고정비 -" + format(int(inp.fixed_monthly_cost), ",") + "원")
71
72         for idx, src in enumerate(inp.sources):
73             if src.cycle_days <= 0 or src.amount <= 0:
74                 continue
75             if day == upcoming[idx]:
76                 amount = src.amount * income_ratio
77                 balance += amount
78                 events.append(src.name + " +" + format(int(amount), ",") + "원")
79                 upcoming[idx] += src.cycle_days
80
81         points.append({"date": current.isoformat(), "balance": round(balance), "events": events})
82     return points
83
84
85 def _first_shortage(points: list[dict], floor: float):
86     for p in points:
87         if p["balance"] < floor:
88             return p["date"]
89     return None
90
91
92 @router.post("/cashflow")
93 def simulate_cashflow(body: SimulationInput):
94     floor = float(body.min_safety_balance)
95     base = _run(body)
96     scenarios = {
97         "기본": base,
98         "입금지연": _run(body, delay_days=10),
99         "수입감소": _run(body, income_ratio=0.7),
100     }
101     shortages = {k: _first_shortage(v, floor) for k, v in scenarios.items()}
102
103     monthly_income = sum(
104         (s.amount * (30.0 / s.cycle_days)) for s in body.sources if s.cycle_days > 0
105     )
106     monthly_cost = body.fixed_monthly_cost + body.daily_variable_cost * 30
107
108     return {
109         "horizon_days": HORIZON_DAYS,
110         "min_safety_balance": floor,
111         "daily_balances": base,
112         "scenarios": scenarios,
113         "shortage_dates": shortages,
114         "monthly_income_estimate": round(monthly_income),
115         "monthly_cost_estimate": round(monthly_cost),
116         "net_monthly": round(monthly_income - monthly_cost),
117         "note": (
118             "입력값은 저장하지 않고 계산에만 사용됩니다. 시연 인물과 동일한 방식으로 계산하며, "
119             "실제 정산일·금액의 변동은 반영되지 않은 단순화된 추정입니다."
120         ),
121     }
122

```

```

1  """
2  세금 · 건강보험료 추정 API (9/5 신규)
3
4  - GET /api/v1/tax/estimate/{user_id}
5
6  app/tax.py의 계산 엔진에 이 유저의 실제 정산 수입을 넣어서, 내년 5월 종합소득세와
7  건강보험료를 추정합니다. 연 환산은 "최근 90일 정산완료 수입 × 4"로 합니다 - 이 데모의
8  수입 데이터가 90일치라 그 이상은 만들어낼 수 없고, 없는 기간을 0으로 두면 세금이 실제보다
9  훨씬 낮게 나와서 오히려 오해를 줍니다.
10 """
11 from fastapi import APIRouter, Depends, HTTPException
12 from sqlalchemy import text
13 from sqlalchemy.orm import Session
14
15 from app.database import get_db
16 from app.demo_clock import get_demo_now
17 from app.security import verify_api_key
18 from app.tax import DISCLAIMER, estimate_health_insurance, estimate_income_tax, expense_rate_for
19
20 router = APIRouter(prefix="/api/v1/tax", tags=["tax"], dependencies=[Depends(verify_api_key)])
21
22 INCOME_LOOKBACK_DAYS = 90
23
24
25 def load_annual_income(db: Session, user_id: str):
26     """연결된 플랫폼의 최근 90일 정산완료 수입을 연 환산하고, 유형별로 나눠 돌려줍니다.
27
28     반환: (사업소득 연환산, 근로소득 연환산, 플랫폼 유형 목록)
29     '본업' 유형은 근로소득이라 사업소득 경비율·중소세 계산에서 분리합니다.
30     """
31     from datetime import timedelta
32
33     demo_now = get_demo_now(db)
34     since = demo_now - timedelta(days=INCOME_LOOKBACK_DAYS)
35     rows = db.execute(
36         text(
37             """
38             SELECT s.platform_type, COALESCE(SUM(e.amount), 0) AS total
39             FROM income_events e
40             JOIN income_sources s ON s.source_id = e.source_id
41             WHERE s.user_id = :uid AND s.connected = TRUE
42             AND e.status = '정산완료' AND e.settled_at >= :since
43             GROUP BY s.platform_type
44             """
45         ),
46         {"uid": user_id, "since": since},
47     ).mappings().all()
48
49     factor = 365 / INCOME_LOOKBACK_DAYS
50     business, employment, types = 0.0, 0.0, []
51     for r in rows:
52         annual = float(r["total"]) * factor
53         types.append(r["platform_type"])
54         if r["platform_type"] == "본업":
55             employment += annual
56         else:
57             business += annual
58     return business, employment, types
59
60
61 @router.get("/estimate/{user_id}")
62 def estimate(user_id: str, db: Session = Depends(get_db)):
63     business, employment, types = load_annual_income(db, user_id)

```

```
64     if business <= 0 and employment <= 0:
65         raise HTTPException(status_code=404, detail="연결된 수입 플랫폼이 없어 추정할 수 없습니다.")
66
67     rate = expense_rate_for(types)
68     income_tax = estimate_income_tax(business, rate)
69     # 건강보험료는 사업소득 기준으로 봅니다. 본업(근로소득)이 있으면 직장가입자로 보고
70     # 급여 외 소득 2,000만원 초과분에만 소득월액보험료가 붙습니다.
71     health = estimate_health_insurance(business, has_employment_income=employment > 0)
72
73     return {
74         "user_id": user_id,
75         "annual_business_income": round(business),
76         "annual_employment_income": round(employment),
77         "platform_types": sorted(set(types)),
78         "income_tax": income_tax,
79         "health_insurance": health,
80         "monthly_reserve_total": income_tax["monthly_reserve"] + health["total_monthly"],
81         "disclaimer": DISCLAIMER,
82     }
83
```

```

1  """
2  예측 정확도 백테스트 API (9/6 신규) - GET /api/v1/validation/forecast
3
4  왜 만들었나
5  -----
6  이 서비스의 차별점은 "플랫폼별 정산 주기와 반복 지출을 학습해 앞으로를 예측한다"는 것인데,
7  그 예측이 실제로 맞는지 보여주는 수치가 없었습니다. 이상거래 탐지는 정답지 대비 성능을
8  제시할 수 있었지만(재측정 결과 탐지율 67.7% / 정상거래 오탐률 4.9%), 정작 핵심 기능은
9  "그렇듯해 보인다" 이상의 근거가 없었던 셈입니다.
10
11  어떻게 검증하나 (홀드아웃 방식)
12  -----
13  45일 곡선을 만드는 예측은 두 가지 추론 위에 서 있습니다.
14
15  (1) 수입: 플랫폼마다 "다음 정산은 언제, 얼마" - 과거 정산 간격의 중앙값으로 추정
16  (2) 지출: 규칙적으로 반복되는 결제의 "다음 결제일" - 간격 중앙값으로 추정
17
18  두 경우 모두 **마지막 실제 발생 건을 감춘 뒤**, 그 이전 기록만으로 그 시점을 예측하고
19  실제와 비교합니다. 미래를 몰랐던 상태를 그대로 재현하는 것이라, 학습에 쓴 데이터로 자신을
20  평가하는 문제를 피할 수 있습니다.
21
22  무엇을 보고하나
23  -----
24  - 정산일 예측 오차(일): 평균 절대 오차와 ±3일 이내 적중률
25  - 정산 금액 예측 오차(%): 직전 회차들의 평균으로 다음 금액을 추정했을 때의 절대 백분율 오차
26  - 반복 지출 탐지: 같은 방식으로 다음 결제일을 맞춘 비율
27
28  한계도 같이 밝힙니다: 표본은 시연용 가상 데이터이고, 플랫폼당 홀드아웃 1건씩이라
29  표본 수가 많지 않습니다. 실제 정산 데이터로 다시 측정해야 확정적인 수치가 됩니다.
30  """
31  import statistics
32  from datetime import date, timedelta
33
34  from fastapi import APIRouter, Depends
35  from sqlalchemy import text
36  from sqlalchemy.orm import Session
37
38  from app.database import get_db
39  from app.demo_personas import DEMO_PERSONAS
40  from app.security import verify_api_key
41
42  router = APIRouter(prefix="/api/v1/validation", tags=["validation"], dependencies=[Depends(verify_api_key)])
43
44  HIT_TOLERANCE_DAYS = 3
45  MIN_HISTORY = 3 # 간격을 추정하려면 최소 3건(간격 2개)은 있어야 합니다
46
47
48  def _predict_next(dates: list[date], amounts: list[float]) -> tuple[date, float]:
49      """마지막 건을 제외한 기록으로 '다음 발생일과 금액'을 추정합니다.
50      현재 서비스가 쓰는 방식과 동일하게, 간격은 중앙값·금액은 최근 평균을 씁니다."""
51      gaps = [(dates[i] - dates[i - 1]).days for i in range(1, len(dates))]
52      interval = statistics.median(gaps)
53      predicted_date = dates[-1] + timedelta(days=round(interval))
54      predicted_amount = sum(amounts[-4:]) / len(amounts[-4:])
55      return predicted_date, predicted_amount
56
57
58  def _backtest_income(db: Session, user_ids: list[str]) -> dict:
59      rows = db.execute(
60          text(
61              """
62              SELECT s.source_id, s.platform_name, e.settled_at, e.amount
63              FROM income_events e

```

```

64         JOIN income_sources s ON s.source_id = e.source_id
65         WHERE s.user_id = ANY(:uids)
66         ORDER BY s.source_id, e.settled_at
67         ""
68     ),
69     {"uids": user_ids},
70 ).mappings().all()
71
72 by_source: dict = {}
73 for r in rows:
74     by_source.setdefault(str(r["source_id"]), {"name": r["platform_name"], "events": []})
75     by_source[str(r["source_id"])][ "events"].append((r["settled_at"].date(), float(r["amount"])))
76
77 date_errors, amount_errors, details = [], [], []
78 for src in by_source.values():
79     events = src["events"]
80     if len(events) < MIN_HISTORY + 1:
81         continue
82     history, actual = events[:-1], events[-1]
83     pred_date, pred_amount = _predict_next([d for d, _ in history], [a for _, a in history])
84
85     d_err = abs((pred_date - actual[0]).days)
86     a_err = abs(pred_amount - actual[1]) / actual[1] * 100 if actual[1] else 0.0
87     date_errors.append(d_err)
88     amount_errors.append(a_err)
89     details.append(
90         {
91             "platform": src["name"],
92             "predicted_date": pred_date.isoformat(),
93             "actual_date": actual[0].isoformat(),
94             "date_error_days": d_err,
95             "amount_error_pct": round(a_err, 1),
96         }
97     )
98
99 n = len(date_errors)
100 return {
101     "samples": n,
102     "mean_absolute_error_days": round(sum(date_errors) / n, 2) if n else None,
103     "within_3days_rate": round(sum(1 for e in date_errors if e <= HIT_TOLERANCE_DAYS) / n * 100, 1) if n else None,
104     "mean_amount_error_pct": round(sum(amount_errors) / n, 1) if n else None,
105     "details": details,
106 }
107
108
109 def _backtest_recurring(db: Session, user_ids: list[str]) -> dict:
110     """반복 결제로 판정된 (유저, 가맹점) 조합에 대해 같은 홀드아웃 검증을 수행합니다."""
111     import pandas as pd
112
113     from data_pipeline.recurring_detector import detect_recurring_payments
114
115     rows = db.execute(
116         text(
117             "SELECT user_id, merchant_name, amount, timestamp FROM transactions "
118             "WHERE user_id = ANY(:uids) ORDER BY timestamp"
119         ),
120         {"uids": user_ids},
121     ).mappings().all()
122     if not rows:
123         return {"samples": 0, "mean_absolute_error_days": None, "within_3days_rate": None}
124
125     # Postgres NUMERIC은 Decimal로 들어오는데, 탐지 로직이 float와 섞어 연산하므로 여기서 맞춥니다
126     # (Decimal * float은 TypeError가 납니다).
127     df = pd.DataFrame(
128         [
129             {
130                 "user_id": r["user_id"],

```

```

131         "merchant_name": r["merchant_name"],
132         "amount": float(r["amount"]),
133         "timestamp": r["timestamp"],
134     }
135     for r in rows
136 ]
137 )
138 detected = detect_recurring_payments(df)
139
140 errors = []
141 for _, rec in detected.iterrows():
142     grp = df[(df["user_id"] == rec["user_id"]) & (df["merchant_name"] == rec["merchant_name"])]
143     dates = sorted(pd.to_datetime(grp["timestamp"]).dt.date.tolist())
144     if len(dates) < MIN_HISTORY + 1:
145         continue
146     pred_date, _ = _predict_next(dates[:-1], [1.0] * (len(dates) - 1))
147     errors.append(abs((pred_date - dates[-1]).days))
148
149 n = len(errors)
150 return {
151     "samples": n,
152     "mean_absolute_error_days": round(sum(errors) / n, 2) if n else None,
153     "within_3days_rate": round(sum(1 for e in errors if e <= HIT_TOLERANCE_DAYS) / n * 100, 1) if n else None,
154 }
155
156 @router.get("/gemini")
157 def gemini_probe():
158     """어떤 Gemini 모델이 실제로 응답하는지 하나씩 찌러보는 진단용 엔드포인트.
159
160     503(high demand)이 잦아서 "키는 정상인데 호출만 실패"하는 상황이 반복됐는데, 모델별로
161     가용성이 다른지 밖에서는 알 수 없었습니다. 후보 모델을 순서대로 짧게 호출해보고
162     성공/실패와 사유를 그대로 돌려줍니다.
163     """
164
165     import os
166
167     from app.gemini_client import model_chain
168
169     if not os.environ.get("GEMINI_API_KEY"):
170         return {"available": False, "reason": "GEMINI_API_KEY 없음"}
171
172     from google import genai
173     from google.genai import types
174
175     client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
176     results = []
177     for model in model_chain():
178         try:
179             r = client.models.generate_content(
180                 model=model,
181                 contents='{"ping":1} 을 그대로 돌려줘',
182                 config=types.GenerateContentConfig(response_mime_type="application/json", temperature=0),
183             )
184             results.append({"model": model, "ok": True, "sample": (r.text or "")[:60]})
185         except Exception as exc:
186             results.append({"model": model, "ok": False, "error": (type(exc).__name__ + ": " + str(exc))[:160]})
187     return {"available": True, "results": results}
188
189 @router.get("/forecast")
190 def forecast_accuracy(db: Session = Depends(get_db)):
191     """예측 엔진의 홀드아웃 검증 결과. 시연 인물 전체를 대상으로 계산합니다."""
192     user_ids = [p["user_id"] for p in DEMO_PERSONAS]
193     # 한쪽 검증이 실패해도 나머지 결과는 돌려줍니다.
194     try:
195         income = _backtest_income(db, user_ids)
196     except Exception as exc:
197         income = {"samples": 0, "error": type(exc).__name__ + ": " + str(exc)[:120]}

```

```

199     try:
200         recurring = _backtest_recurring(db, user_ids)
201     except Exception as exc:
202         recurring = {"samples": 0, "error": type(exc).__name__ + ": " + str(exc)[:120]}
203     return {
204         "method": (
205             "각 플랫폼·가맹점의 마지막 발생 건을 감춘 뒤, 그 이전 기록만으로 발생일과 금액을 "
206             "예측하고 실제와 비교했습니다(홀드아웃). 예측 방식은 서비스가 실제로 쓰는 것과 동일합니다."
207         ),
208         "income_settlement": income,
209         "recurring_payment": recurring,
210         "limitations": (
211             "표본은 시연용 가상 데이터이며 대상별 홀드아웃 1건씩이라 표본 수가 제한적입니다. "
212             "특히 정산 '날짜'는 시드 데이터가 고정 주기로 생성되어 오차가 거의 0으로 나오는데, "
213             "이는 예측력이 완벽하다는 뜻이 아니라 구현이 의도대로 동작한다는 확인에 가깝습니다. "
214             "실제 예측력을 보여주는 지표는 변동성이 들어있는 '금액' 오차와 반복 지출 발생일 오차이며, "
215             "확정적인 수치는 실제 정산 데이터를 연동한 뒤 재측정해야 합니다."
216         ),
217     }
218

```

```

1  """
2  리스크방어 룰 엔진 (R001 ~ R007)
3
4  설계서(리스크방어 및 예산통제 시스템 아키텍처 설계서.pdf, 3.4.1절)의 의사코드를
5  실제로 동작하는 파이썬 코드로 옮긴 버전입니다.
6
7  포인트: 이 모듈은 DB나 B/C의 데이터가 없어도 지금 바로 테스트할 수 있습니다.
8          tests/test_rule_engine.py 에서 손으로 만든 샘플 거래로 검증합니다.
9  """
10 from __future__ import annotations
11
12 from datetime import timedelta
13 from math import radians, sin, cos, sqrt, atan2
14 from typing import Optional
15
16 from app.models import RuleAction, RuleResult, Severity, Transaction, UserRiskProfile
17
18
19 def haversine_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
20     """두 좌표 사이의 거리(km). 이동 속도 계산(R004)에 씁니다."""
21     R = 6371.0
22     dlat = radians(lat2 - lat1)
23     dlon = radians(lon2 - lon1)
24     a = sin(dlat / 2) ** 2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon / 2) ** 2
25     return R * 2 * atan2(sqrt(a), sqrt(1 - a))
26
27
28 class RuleEngine:
29     """
30     사용법:
31         engine = RuleEngine(history=과거_거래_리스트)
32         results = engine.evaluate(new_transaction, user_profile)
33
34     history는 해당 유저의 "이 거래 이전" 거래 목록입니다.
35     실제 서비스에서는 DB에서 조회해오지만, 테스트에서는 그냥 리스트를 넘기면 됩니다.
36     """
37
38     def __init__(self, history: Optional[list[Transaction]] = None):
39         self.history = sorted(history or [], key=lambda t: t.timestamp)
40
41     # ---- 내부 헬퍼: 과거 거래 조회 ----
42
43     def _daily_total(self, user_id: str, date) -> float:
44         return sum(
45             t.amount
46             for t in self.history
47             if t.user_id == user_id and t.timestamp.date() == date
48         )
49
50     def _is_known_merchant(self, user_id: str, merchant_id: Optional[str]) -> bool:
51         if not merchant_id:
52             return False
53         return any(
54             t.user_id == user_id and t.merchant_id == merchant_id for t in self.history
55         )
56
57     def _last_transaction(self, user_id: str) -> Optional[Transaction]:
58         user_txns = [t for t in self.history if t.user_id == user_id]
59         return user_txns[-1] if user_txns else None
60
61     def _recent_merchant_count(self, user_id: str, merchant_id: Optional[str], since) -> int:
62         if not merchant_id:
63             return 0

```

```

64         return sum(
65             1
66             for t in self.history
67             if t.user_id == user_id
68             and t.merchant_id == merchant_id
69             and t.timestamp >= since
70         )
71
72 # ---- 메인 진입점 ----
73
74 def evaluate(self, txn: Transaction, profile: UserRiskProfile) -> list[RuleResult]:
75     results: list[RuleResult] = []
76
77     # R001: 단일 거래 한도 초과
78     if txn.amount > profile.single_transaction_limit:
79         results.append(
80             RuleResult(
81                 rule_id="R001",
82                 severity=Severity.HIGH,
83                 description=(
84                     f"단일 거래 한도 초과: {txn.amount:,.0f}원 > "
85                     f"{profile.single_transaction_limit:,.0f}원"
86                 ),
87                 action=RuleAction.REQUIRE_CONFIRMATION,
88             )
89         )
90
91     # R002: 일일 누적 거래 한도 초과
92     daily_total = self._daily_total(txn.user_id, txn.timestamp.date())
93     if daily_total + txn.amount > profile.daily_limit:
94         results.append(
95             RuleResult(
96                 rule_id="R002",
97                 severity=Severity.CRITICAL,
98                 description=(
99                     f"일일 한도 초과: 누적 {daily_total:,.0f}원 + "
100                     f"{txn.amount:,.0f}원 > {profile.daily_limit:,.0f}원"
101                 ),
102                 action=RuleAction.BLOCK,
103             )
104         )
105
106     # R003: 신규 가맹점 + 평균 대비 대액
107     if not self._is_known_merchant(txn.user_id, txn.merchant_id):
108         if profile.avg_transaction_amount > 0 and txn.amount > profile.avg_transaction_amount * 3:
109             results.append(
110                 RuleResult(
111                     rule_id="R003",
112                     severity=Severity.MEDIUM,
113                     description="신규 가맹점 + 평균 거래액 3배 초과",
114                     action=RuleAction.FLAG_FOR_REVIEW,
115                 )
116             )
117
118     # R004: 불가능한 이동 속도 (직전 거래와 비교)
119     last_txn = self._last_transaction(txn.user_id)
120     if last_txn and last_txn.latitude is not None and txn.latitude is not None:
121         distance = haversine_km(last_txn.latitude, last_txn.longitude, txn.latitude, txn.longitude)
122         time_diff_hours = (txn.timestamp - last_txn.timestamp).total_seconds() / 3600
123         if time_diff_hours > 0:
124             speed = distance / time_diff_hours
125             if speed > 900: # 항공편보다 빠른 이동 = 물리적으로 불가능
126                 results.append(
127                     RuleResult(
128                         rule_id="R004",
129                         severity=Severity.CRITICAL,
130                         description=f"불가능한 이동 속도: {speed:.0f} km/h",
131                         action=RuleAction.BLOCK,

```

```

132         )
133     )
134
135     # R005: 심야 대액 결제 (02:00 ~ 05:00)
136     if 2 <= txn.timestamp.hour <= 5:
137         if profile.avg_transaction_amount > 0 and txn.amount > profile.avg_transaction_amount * 5:
138             results.append(
139                 RuleResult(
140                     rule_id="R005",
141                     severity=Severity.MEDIUM,
142                     description="심야 시간대 평균 거래액 5배 초과",
143                     action=RuleAction.FLAG_FOR_REVIEW,
144                 )
145             )
146
147     # R006: 동일 가맹점 단기 반복 결제 (5분 내 3회 이상)
148     recent_count = self._recent_merchant_count(
149         txn.user_id, txn.merchant_id, since=txn.timestamp - timedelta(minutes=5)
150     )
151     if recent_count >= 2: # 과거 2회 + 이번 거래 = 3회
152         results.append(
153             RuleResult(
154                 rule_id="R006",
155                 severity=Severity.HIGH,
156                 description=f"5분 내 동일 가맹점 {recent_count + 1}회 결제",
157                 action=RuleAction.REQUIRE_CONFIRMATION,
158             )
159         )
160
161     # R007: 허용 국가 외 결제
162     if txn.country not in profile.allowed_countries:
163         results.append(
164             RuleResult(
165                 rule_id="R007",
166                 severity=Severity.HIGH,
167                 description=f"허용되지 않은 국가 결제: {txn.country}",
168                 action=RuleAction.REQUIRE_CONFIRMATION,
169             )
170         )
171
172     return results
173
174
175 # 규칙별 심각도 -> 점수 환산 (0~1). 나중에 Isolation Forest 결과와 앙상블할 때 씁니다.
176 _SEVERITY_SCORE = {
177     Severity.LOW: 0.2,
178     Severity.MEDIUM: 0.4,
179     Severity.HIGH: 0.7,
180     Severity.CRITICAL: 1.0,
181 }
182
183
184 def rule_results_to_score(results: list[RuleResult]) -> float:
185     """여러 규칙이 동시에 걸렸을 때, 가장 심각한 것을 기준으로 0~1 점수를 냅니다."""
186     if not results:
187         return 0.0
188     return max(_SEVERITY_SCORE[r.severity] for r in results)
189

```

```

1  """
2  API 접근 제어 (8/30 보안 점검 추가)
3
4  기존엔 어떤 인증도 없어서, 배포된 API 주소만 알면 누구나 user_id를 바꿔가며
5  다른 사람의 예산/리스크/질세장부 데이터를 조회할 수 있었습니다 (금융 데이터를
6  다루는 앱에서는 치명적인 문제).
7
8  풀 로그인(회원가입/세션) 시스템은 지금 남은 일정에 비해 과합니다. 대신
9  "이 API를 호출해도 되는 클라이언트인지"만 최소한으로 확인하는 공유 키(shared
10 secret) 방식을 넣었습니다:
11
12 - .env에 API_ACCESS_KEY가 설정되어 있으면: 모든 /api/v1/* 요청에 그 값과 정확히
13 일치하는 `X-API-Key` 헤더가 있어야 합니다. 프론트(하은/소현 쪽 코드)는 API를
14 부를 때마다 이 헤더를 같이 보내야 합니다.
15 - .env에 API_ACCESS_KEY가 없으면(로컬 개발 중 기본값): 인증을 요구하지 않습니다.
16 로컬에서 Swagger UI(/docs)로 바로 테스트할 수 있게 하기 위함입니다.
17
18 ★ 깃허브에 배포할 때는 반드시 API_ACCESS_KEY를 설정한 값으로 .env를 채워서
19 배포하세요 (README 6단계 참고). 이 값 자체도 비밀번호와 같으니 절대 코드에
20 하드코딩하거나 커밋하지 마세요.
21
22 이것으로 "다른 사람인 척 아무 user_id나 넣어 조회"까지 막지는 못합니다(그러려면
23 로그인/세션이 필요합니다) - 최소한 "API 주소를 아는 아무나"가 못 들어오게는
24 막아줍니다. 진짜 로그인 붙이기 전까지의 임시 방어선이라는 점을 README에도
25 분명히 적어뒀습니다.
26 """
27 import os
28 import secrets
29
30 from fastapi import Header, HTTPException
31
32 API_ACCESS_KEY = os.environ.get("API_ACCESS_KEY", "").strip()
33
34
35 def verify_api_key(x_api_key: str = Header(default=None)):
36     """API_ACCESS_KEY가 설정된 배포 환경에서만 강제되는 최소한의 접근 제어.
37     라우터에 `dependencies=[Depends(verify_api_key)]` 로 붙여서 씁니다."""
38     if not API_ACCESS_KEY:
39         # 로컬 개발 기본값: 키를 안 정해뒀으면 막지 않음 (Swagger UI 테스트 편의)
40         return
41     if not x_api_key or not secrets.compare_digest(x_api_key, API_ACCESS_KEY):
42         raise HTTPException(
43             status_code=401,
44             detail="유효한 X-API-Key 헤더가 필요합니다.",
45         )
46

```

```

1  """
2  보안 응답 헤더 미들웨어 (9/2 세 번째 업데이트 - 금융보안원 주최 공모전이라 보안을
3  특별히 더 신경써달라는 요청으로 추가)
4
5  브라우저가 이 헤더들을 보고 스스로 방어하게 만드는, 서버 쪽에서는 몇 줄이면 되는데
6  효과는 큰 대책들입니다. 하나씩:
7
8  - Content-Security-Policy: "이 페이지는 어디서 온 스크립트/스타일/폰트만 믿는다"를
9  선언합니다. 예를 들어 만약 XSS 취약점이 어딘가에 있어서 공격자가 스크립트를
10 주입해도, CSP가 막아주면 실행되지 않습니다. frontend/*.html이 각 페이지마다
11 <script>인라인 코드</script> 블록을 하나씩 쓰고 있어서(외부 api.js 파일 + 페이지별
12 인라인 로직 조합) script-src에 'unsafe-inline'을 넣을 수밖에 없었습니다 - 완벽한
13 CSP는 아니지만, 그래도 외부 도메인에서 스크립트를 불러오는 공격(가장 흔한 XSS
14 공격 형태)은 확실히 막습니다. (다음 단계로 더 강화하려면 인라인 스크립트를 전부
15 별도 .js 파일로 옮기고 nonce/hash 기반 CSP로 바꾸면 되는데, 이번 일정에서는
16 범위 밖이라 다음 할 일로 남깁니다.)
17 - X-Content-Type-Options: nosniff - 브라우저가 파일 확장자/내용을 보고 "이거 사실
18 실행 가능한 스크립트 아냐?"하고 지레짐작(MIME sniffing)하지 못하게 막습니다.
19 - X-Frame-Options / frame-ancestors: 이 사이트를 다른 사이트의 <iframe> 안에 몰래
20 띄워서 사용자를 속이는 클릭재킹(clickjacking) 공격을 막습니다.
21 - Referrer-Policy: 다른 사이트로 이동할 때 이 사이트의 URL을 통째로 넘기지 않도록
22 제한합니다(우리 URL엔 user_id 같은 값이 쿼리 파라미터로 들어갈 수 있어서 중요).
23 - Permissions-Policy: 이 앱은 카메라/마이크/위치 정보를 전혀 안 쓰므로 아예 꺼둡니다.
24 - Strict-Transport-Security(HSTS): 이후 요청을 전부 HTTPS로만 하도록 브라우저에
25 지시합니다. 요청이 HTTPS로 왔을 때만(또는 리버스 프록시가 X-Forwarded-Proto:
26 https로 알려줬을 때만) 붙입니다 - 로컬 http 개발 환경에서는 안 붙습니다.
27 """
28 from __future__ import annotations
29
30 from starlette.middleware.base import BaseHTTPMiddleware
31 from starlette.requests import Request
32 from starlette.responses import Response
33
34 _CSP = (
35     "default-src 'self'; "
36     "script-src 'self' 'unsafe-inline'; "
37     # 9/5 업데이트 - 하은님의 새 frontend/index.html이 Pretendard 가변 폰트를
38     # cdn.jsdelivr.net(GitHub raw 미러)에서 불러옵니다. 기존 CSP는 fonts.googleapis.com만
39     # 허용해서, 이 CDN의 CSS/폰트 파일 둘 다 브라우저가 조용히 차단하고 있었습니다
40     # (콘솔에서만 보이고 화면은 시스템 폰트로 자동 대체되어 눈에 잘 안 띄는 버그).
41     "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://cdn.jsdelivr.net; "
42     "font-src 'self' https://fonts.gstatic.com https://cdn.jsdelivr.net; "
43     "img-src 'self' data; "
44     "connect-src 'self'; "
45     "frame-ancestors 'none'; "
46     "base-uri 'self'; "
47     "form-action 'self'; "
48     "object-src 'none'"
49 )
50
51
52 def is_https_request(request: Request) -> bool:
53     return request.url.scheme == "https" or request.headers.get(
54         "x-forwarded-proto", ""
55     ).lower() == "https"
56
57
58 def build_security_headers(is_https: bool) -> dict[str, str]:
59     """모든 응답에 붙일 보안 헤더 묶음. 미들웨어와, 처리되지 않은 예외를 잡는
60     500 핸들러(app/main.py)가 함께 씁니다 - 아래 미들웨어는 BaseHTTPMiddleware라
61     안쪽에서 예외가 터진 500 응답에는 실행되지 않기 때문에, 그 경로에서도 같은
62     헤더가 빠지지 않도록 함수로 분리했습니다."""
63     headers = {

```

```

64         "X-Content-Type-Options": "nosniff",
65         "X-Frame-Options": "DENY",
66         "Referrer-Policy": "strict-origin-when-cross-origin",
67         "Permissions-Policy": "geolocation=(), microphone=(), camera=(), payment=()",
68         "Content-Security-Policy": _CSP,
69     }
70     if is_https:
71         headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
72     return headers
73
74
75 class SecurityHeadersMiddleware(BaseHTTPMiddleware):
76     async def dispatch(self, request: Request, call_next) -> Response:
77         response = await call_next(request)
78         for key, value in build_security_headers(is_https_request(request)).items():
79             response.headers[key] = value
80         return response
81

```

```

1  """
2  9/2 추가 - 배포 직후 DB가 비어있으면 앱이 스스로 스키마/데모 데이터를 채우는 시작 훅.
3
4  배경(9/2에 발견한 문제): 지금까지 스키마 생성은 docker-compose.yml이 로컬 postgres
5  컨테이너를 "완전히 새로" 띄울 때만 `sql/init.sql`을 자동 실행해주는 방식(공식 postgres
6  이미지의 `/docker-entrypoint-initdb.d/` 관례)에 의존했고, 데모 데이터(5개 페르소나,
7  mock 거래, 예산/리스크/수입) 적재는 `scripts/`.`data/` 폴더의 스크립트를 사람이 직접
8  로컬에서 실행하는 방식이었습니다. 둘 다 "실제 배포 환경(Railway/Render 등, 관리형
9  Postgres를 쓰거나 Dockerfile만으로 빌드하는 경우)"에서는 자동으로 일어나지 않는
10 가정이었습니다 - 그 결과 그대로 배포하면 DB가 완전히 빈 채로 뜨고, 5개 데모 페르소나가
11 전부 404/빈 화면으로 보이는 문제가 있었습니다.
12
13 이 파일은 앱이 시작할 때(app/main.py의 startup 이벤트) 아래 두 단계를 자동으로
14 실행해서, "Dockerfile로 빌드해서 아무 Postgres/Redis에 연결만 시켜주면" 나머지는
15 전부 앱이 알아서 채우도록 만듭니다.
16
17 1. ensure_schema(): sql/init.sql 전체를 그대로 실행합니다. 이 파일은 전부
18 `CREATE TABLE IF NOT EXISTS` / `ON CONFLICT DO NOTHING`으로 작성돼 있어서(원래도
19 docker-compose 로컬 재시작에 안전하도록 그렇게 만들어져 있었음) 스키마가 이미 있어도
20 매번 안전하게 다시 실행할 수 있습니다 - 그래서 별도 "이미 있는지" 확인 없이 매번
21 실행합니다.
22 2. ensure_demo_data_seeded(): transactions/user_risk_profiles/budgets/income_sources
23 각각이 비어있는지 따로 확인해서, 비어있는 단계만 순서대로 채웁니다(아래 파이프라인
24 재실행/재시작 시 이미 끝난 단계를 다시 하지 않도록 - 특히 이미 데이터가 있는데
25 ingestion을 다시 돌리면 transactions를 TRUNCATE하고 다시 채우기 때문에 여기서만
26 엄격하게 "완전히 비어있을 때"로 제한합니다).
27
28 실패해도(예: CSV가 없거나 DB 연결 문제) 예외를 밖으로 던지지 않고 로그만 남깁니다 -
29 시당이 실패했다고 서버 자체가 못 뜨면 원인 파악(헬스체크, 로그)조차 더 어려워지기
30 때문입니다.
31 """
32 import logging
33 from pathlib import Path
34
35 from sqlalchemy import text
36
37 logger = logging.getLogger("sidegig.startup_seed")
38
39 _SQL_INIT_PATH = Path(__file__).resolve().parent.parent / "sql" / "init.sql"
40
41
42 def ensure_schema():
43     """sql/init.sql을 그대로 실행해서 테이블/인덱스가 있는지 보장합니다.
44     (docker-compose의 로컬 postgres 자동초기화에 의존하지 않고, 어떤 Postgres에
45     연결하든 앱이 직접 스키마를 만듭니다 - 관리형 Postgres 배포 대응.)"""
46     from app.database import engine
47
48     if not _SQL_INIT_PATH.exists():
49         logger.warning("[startup_seed] sql/init.sql을 찾을 수 없어 스키마 생성을 건너뛴니다: %s", _SQL_INIT_PATH)
50         return
51
52     sql_text = _SQL_INIT_PATH.read_text(encoding="utf-8")
53     raw_conn = engine.raw_connection()
54     try:
55         cursor = raw_conn.cursor()
56         # psycopg2는 파라미터 없는 execute()에 여러 문장(세미콜론으로 구분, DO $$ ... $$
57         # 블록 포함)을 한 번에 넘기면 simple query protocol로 그대로 실행합니다 -
58         # `psql -f init.sql`과 동일한 방식입니다.
59         cursor.execute(sql_text)
60         raw_conn.commit()
61         cursor.close()
62         logger.info("[startup_seed] 스키마 확인/생성 완료 (sql/init.sql 실행).")
63     except Exception:

```

```

64         raw_conn.rollback()
65         logger.exception("[startup_seed] 스키마 생성 중 오류가 발생했습니다.")
66         raise
67     finally:
68         raw_conn.close()
69
70
71 def _count(db, table: str, where: str = "") -> int:
72     row = db.execute(text(f"SELECT count(*) AS n FROM {table} {where}")).mappings().first()
73     return row["n"] if row else 0
74
75
76 def _ensure_persona_balances():
77     """데모 페르소나의 Risk Shield 시작 잔액을 user_settings에 채웁니다(idempotent).
78
79     이미 값이 들어있으면 덮어쓰지 않습니다 - 화면에서 사용자가 직접 잔액을 수정해본
80     상태를 서버 재시작이 되돌려버리면 안 되기 때문입니다.
81     """
82     from app.database import SessionLocal
83     from app.demo_personas import DEMO_PERSONAS
84
85     db = SessionLocal()
86     try:
87         for p in DEMO_PERSONAS:
88             balance = p.get("current_balance")
89             if balance is None:
90                 continue
91             db.execute(
92                 text(
93                     "INSERT INTO user_settings (user_id, current_balance, updated_at) "
94                     "VALUES (:uid, :bal, now()) "
95                     "ON CONFLICT (user_id) DO UPDATE SET current_balance = "
96                     "COALESCE(user_settings.current_balance, EXCLUDED.current_balance)"
97                 ),
98                 {"uid": p["user_id"], "bal": balance},
99             )
100         db.commit()
101         logger.info("[startup_seed] 페르소나 시작 잔액 확인/설정 완료.")
102     except Exception:
103         db.rollback()
104         logger.exception("[startup_seed] 페르소나 시작 잔액 설정 중 오류.")
105     finally:
106         db.close()
107
108
109 def ensure_demo_data_seeded():
110     from app.database import SessionLocal
111
112     db = SessionLocal()
113     try:
114         txn_count = _count(db, "transactions")
115         profile_count = _count(db, "user_risk_profiles", "WHERE avg_transaction_amount > 0")
116         budget_count = _count(db, "budgets")
117         income_count = _count(db, "income_sources")
118     finally:
119         db.close()
120
121     try:
122         if txn_count == 0:
123             logger.info("[startup_seed] transactions가 비어있어 mock 데이터 적재를 시작합니다...")
124             from data_pipeline.ingestion import run_pipeline
125
126             run_pipeline()
127         else:
128             logger.info("[startup_seed] transactions에 이미 %d건이 있어 적재를 건너웁니다.", txn_count)
129
130         if profile_count == 0:
131             from scripts.build_user_profiles import build_profiles

```

```

132
133         build_profiles()
134
135     if budget_count == 0:
136         from scripts.seed_demo_personas import replay_risk_assessments, seed_budgets_and_spending
137
138         db2 = SessionLocal()
139         try:
140             seed_budgets_and_spending(db2)
141             replay_risk_assessments(db2)
142         finally:
143             db2.close()
144
145     if income_count == 0:
146         from scripts.seed_demo_income import seed_income
147
148         db3 = SessionLocal()
149         try:
150             seed_income(db3)
151         finally:
152             db3.close()
153
154     # 9/5 추가: Risk Shield 시작 잔액(user_settings.current_balance) 채우기.
155     # 위 단계들과 달리 별도 조건으로 두는 이유는, 이 컬럼이 나중에 추가돼서 이미
156     # 시딩이 끝난 DB(=budgets/income이 이미 차 있어 위 분기들이 전부 건너뛰어지는 DB)
157     # 에도 값이 들어가야 하기 때문입니다.
158     _ensure_persona_balances()
159
160     logger.info("[startup_seed] 데모 데이터 시딩 완료.")
161 except Exception:
162     logger.exception("[startup_seed] 데모 데이터 시딩 중 오류가 발생했습니다 (서버는 계속 켜집니다).")
163
164
165 def run_startup_seed():
166     """app/main.py의 startup 이벤트에서 호출하는 진입점."""
167     try:
168         ensure_schema()
169     except Exception:
170         # 스키마 생성 자체가 실패하면(예: DB 연결 불가) 이후 시딩은 의미가 없으니 중단하되,
171         # 서버 프로세스는 그대로 띄워서 /health로 원인을 확인할 수 있게 합니다.
172         return
173     ensure_demo_data_seeded()
174

```

```

1  """
2  종합소득세 · 건강보험료 추정 엔진 (9/5 신규)
3
4  왜 만들었나
5  -----
6  이전까지 app/cashflow.py는 세금·건보료 준비금을 "월평균 수입 × 8%", "× 7%"라는
7  고정 비율로 잡았습니다. 코드 주석에도 "실제 세법·건강보험료 산정식이 아니라 프리랜서에게
8  흔히 권장되는 관행을 단순화한 예시 비율"이라고 적혀 있었습니다. 그런데 이 서비스가 내세우는
9  가치가 "N잡러가 1~2년 뒤에 맞는 세금·건보료를 지금 미리 알려준다"인 이상, 그 숫자의 근거가
10 관행이면 곤란합니다. 이 파일은 실제 제도의 계산 구조를 그대로 옮깁니다.
11
12 무엇을 계산하나
13 -----
14 1) 종합소득세: 플랫폼 정산 수입(사업소득)을 연 환산 → 단순경비율로 필요경비 차감 →
15 기본공제 → 과세표준 → 누진세율표 적용 → 지방소득세 10% 가산 → 이미 원천징수된
16 3.3%를 기납부세액으로 차감. 결과가 양수면 "5월에 더 낼 돈", 음수면 "돌려받을 돈".
17 2) 건강보험료: 연소득 기준 보험료율을 적용하고 장기요양보험료를 엽습니다.
18 직장가입자는 급여 외 소득이 연 2,000만원을 넘는 부분에 대해서만 소득월액보험료를
19 본인이 100% 부담합니다. 피부양자는 연소득 2,000만원을 넘으면 자격을 잃고 지역가입자가
20 되므로, 그 경계에 가까우면 경고를 함께 돌려줍니다.
21
22 한계 (화면에도 반드시 같이 표기)
23 -----
24 - 확정 세액·보험료가 아니라 "지금 흐름이 이어질 경우"의 추정치입니다. 인적공제(부양가족),
25 연금·보험료 공제, 세액공제, 다른 소득(근로/이자/배당), 기납부 지방소득세 등은 반영하지
26 않습니다.
27 - 지역가입자 건강보험료는 실제로는 소득 외에 재산·자동차 점수까지 합산해 산정합니다.
28 이 앱은 재산 정보를 갖고 있지 않으므로 소득 부분만 계산합니다.
29 - 단순경비율은 업종·수입 규모에 따라 다르고 매년 고시됩니다. 여기서는 인적용역 계열의
30 대표값을 쓰되 업종별로 다르게 잡을 수 있게 열어놨습니다.
31 """
32 from __future__ import annotations
33
34 # -----
35 # 제도 상수 (2026년 기준)
36 # -----
37
38 # 종합소득세 누진세율표 - (과세표준 상한, 세율, 누진공제액)
39 # 상한 None은 그 위 전부. 산출세액 = 과세표준 × 세율 - 누진공제액.
40 INCOME_TAX_BRACKETS = [
41     (14_000_000, 0.06, 0),
42     (50_000_000, 0.15, 1_260_000),
43     (88_000_000, 0.24, 5_760_000),
44     (150_000_000, 0.35, 15_440_000),
45     (300_000_000, 0.38, 19_940_000),
46     (500_000_000, 0.40, 25_940_000),
47     (1_000_000_000, 0.42, 35_940_000),
48     (None, 0.45, 65_940_000),
49 ]
50
51 LOCAL_INCOME_TAX_RATE = 0.10 # 지방소득세: 소득세액의 10%
52 WITHHOLDING_RATE = 0.033 # 프리랜서 사업소득 원천징수 3.3%(지방소득세 0.3% 포함)
53 BASIC_DEDUCTION = 1_500_000 # 기본공제(본인) 150만원
54
55 # 단순경비율(업종별 고시값 중 인적용역 계열 대표값). 장부를 쓰지 않는 소규모 사업자가
56 # 증빙 없이 인정받는 필요경비 비율입니다.
57 SIMPLE_EXPENSE_RATES = {
58     "배달": 0.794, # 배달·택배서비스 등 인적용역
59     "프리랜서": 0.642, # 기타 인적용역(강사·디자이너·마케터 등)
60     "콘텐츠": 0.642, # 1인 미디어 콘텐츠 창작자
61     "커머스": 0.560,
62     "본업": 0.0, # 근로소득은 경비율 개념이 없음(별도 처리)
63 }

```

```

64 DEFAULT_EXPENSE_RATE = 0.642
65
66 # 건강보험
67 HEALTH_INSURANCE_RATE = 0.0719          # 2026년 건강보험료율 7.19%
68 LONG_TERM_CARE_RATE = 0.1295            # 장기요양보험료 = 건강보험료 × 12.95%
69 EMPLOYEE_SHARE = 0.5                    # 직장가입자는 회사와 절반씩 부담
70 DEPENDENT_INCOME_LIMIT = 20_000_000     # 피부양자 자격 / 소득월액보험료 기준: 연 2,000만원
71
72
73 def _progressive_tax(tax_base: float) -> float:
74     """과세표준에 누진세율표를 적용해 산출세액을 계산합니다."""
75     if tax_base <= 0:
76         return 0.0
77     for upper, rate, deduction in INCOME_TAX_BRACKETS:
78         if upper is None or tax_base <= upper:
79             return max(0.0, tax_base * rate - deduction)
80     return 0.0
81
82
83 def expense_rate_for(platform_types: list[str] | None) -> float:
84     """연결된 플랫폼 유형들을 보고 적용할 단순경비율을 정합니다.
85
86     여러 유형이 섞인 N잡러가 대부분이라, 해당하는 유형들의 평균을 씁니다
87     (본업=근로소득은 사업소득 경비를 계산에서 제외).
88     """
89     rates = [
90         SIMPLE_EXPENSE_RATES[t]
91         for t in (platform_types or [])
92         if t in SIMPLE_EXPENSE_RATES and t != "본업"
93     ]
94     return sum(rates) / len(rates) if rates else DEFAULT_EXPENSE_RATE
95
96
97 def estimate_income_tax(annual_business_income: float, expense_rate: float = DEFAULT_EXPENSE_RATE) -> dict:
98     """사업소득(플랫폼 정산 수입) 연 환산액으로 종합소득세를 추정합니다.
99
100     반환값의 `balance_due`가 양수면 5월에 추가로 낼 돈, 음수면 돌려받을 돈입니다.
101     """
102     income = max(0.0, float(annual_business_income))
103     necessary_expense = income * expense_rate
104     income_amount = income - necessary_expense          # 소득금액
105     tax_base = max(0.0, income_amount - BASIC_DEDUCTION) # 과세표준
106
107     calculated = _progressive_tax(tax_base)             # 산출세액(소득세)
108     local_tax = calculated * LOCAL_INCOME_TAX_RATE     # 지방소득세
109     total_tax = calculated + local_tax
110     prepaid = income * WITHHOLDING_RATE                # 3.3% 원천징수분
111     balance = total_tax - prepaid
112
113     # 적용된 한계세율(어느 구간에 걸쳐 있는지) - 화면에서 설명용으로 씁니다.
114     marginal = 0.0
115     for upper, rate, _ in INCOME_TAX_BRACKETS:
116         if upper is None or tax_base <= upper:
117             marginal = rate
118             break
119
120     return {
121         "annual_income": round(income),
122         "expense_rate": round(expense_rate, 3),
123         "necessary_expense": round(necessary_expense),
124         "income_amount": round(income_amount),
125         "tax_base": round(tax_base),
126         "marginal_rate": marginal,
127         "calculated_tax": round(calculated),
128         "local_income_tax": round(local_tax),
129         "total_tax": round(total_tax),
130         "prepaid_withholding": round(prepaid),
131         "balance_due": round(balance),

```

```

132     "monthly_reserve": round(max(0.0, balance) / 12),
133 }
134
135
136 def estimate_health_insurance(annual_income: float, has_employment_income: bool = False) -> dict:
137     """건강보험료(+장기요양보험료)를 추정합니다.
138
139     - 직장가입자(has_employment_income=True): 급여 외 소득이 연 2,000만원을 넘는 부분에만
140       '소득월액보험료'가 붙고, 이걸 회사 지원 없이 본인이 100% 부담합니다.
141     - 지역가입자: 소득 전체에 보험료율을 적용합니다(재산·자동차 점수는 이 앱에 정보가 없어
142       계산에 넣지 않습니다 - 실제 고지액은 이보다 높을 수 있습니다).
143     """
144     income = max(0.0, float(annual_income))
145
146     if has_employment_income:
147         excess = max(0.0, income - DEPENDENT_INCOME_LIMIT)
148         base_annual = excess
149         note = (
150             "직장가입자는 급여 외 소득이 연 2,000만원을 넘는 부분에만 소득월액보험료가 "
151             "붙고, 회사 지원 없이 본인이 전액 부담합니다."
152         )
153         subscriber_type = "직장가입자(소득월액보험료)"
154     else:
155         base_annual = income
156         note = (
157             "지역가입자는 소득에 보험료율을 그대로 적용합니다. 실제 고지액은 재산·자동차 "
158             "점수가 더해져 이보다 높을 수 있습니다."
159         )
160         subscriber_type = "지역가입자"
161
162     health_monthly = base_annual * HEALTH_INSURANCE_RATE / 12
163     care_monthly = health_monthly * LONG_TERM_CARE_RATE
164     total_monthly = health_monthly + care_monthly
165
166     # 2,000만원 경계 경고.
167     #
168     # 이 경계는 N잡러에게 두 가지로 작동합니다. 직장가입자는 급여 외 소득이 이 선을 넘는
169     # 순간부터 소득월액보험료가 새로 생기고(회사 지원 없이 전액 본인 부담), 피부양자는
170     # 자격을 잃고 보험료를 직접 내게 됩니다. 둘 다 "넘고 나서" 고지서로 알게 되는 게
171     # 보통이라, 넘기 전에 남은 여유가 얼마인지 알려주는 게 이 서비스의 핵심 가치입니다.
172     dependent_lost = income > DEPENDENT_INCOME_LIMIT
173     headroom = DEPENDENT_INCOME_LIMIT - income
174     ratio = income / DEPENDENT_INCOME_LIMIT if DEPENDENT_INCOME_LIMIT else 0
175
176     def _man(v: float) -> str:
177         return format(int(round(v / 10000)), ",") + "만원"
178
179     if dependent_lost:
180         if has_employment_income:
181             warning = (
182                 "급여 외 소득이 2,000만원을 넘어 소득월액보험료 대상입니다. "
183                 "초과분 " + _man(income - DEPENDENT_INCOME_LIMIT) + "에 대한 보험료는 "
184                 "회사 지원 없이 전액 본인이 부담합니다."
185             )
186         else:
187             warning = (
188                 "연 소득이 2,000만원을 넘어 피부양자 자격 기준을 초과합니다. "
189                 "피부양자였다면 자격을 잃고 보험료를 직접 부담하게 됩니다."
190             )
191     elif ratio >= 0.6:
192         warning = (
193             "앞으로 " + _man(headroom) + "을 더 벌면 2,000만원 기준을 넘습니다"
194             + ("현재 기준의 " + str(int(ratio * 100)) + "%). 넘는 순간 소득월액보험료가 새로 생깁니다."
195               if has_employment_income
196               else "(현재 기준의 " + str(int(ratio * 100)) + "%). 넘으면 피부양자 자격을 잃습니다.")
197     )
198     else:
199         warning = None

```

```

200
201     return {
202         "subscriber_type": subscriber_type,
203         "annual_income": round(income),
204         "premium_base": round(base_annual),
205         "health_monthly": round(health_monthly),
206         "long_term_care_monthly": round(care_monthly),
207         "total_monthly": round(total_monthly),
208         "annual_total": round(total_monthly * 12),
209         "dependent_limit": DEPENDENT_INCOME_LIMIT,
210         "dependent_limit_exceeded": dependent_lost,
211         "dependent_limit_headroom": round(max(0.0, headroom)),
212         "dependent_limit_ratio": round(ratio, 3),
213         "warning": warning,
214         "note": note,
215     }
216
217
218     DISCLAIMER = (
219         "확정 세액·보험료가 아니라 지금 수입 흐름이 이어질 경우의 추정치입니다. "
220         "인적공제·세액공제·다른 소득과 재산 점수는 반영하지 않았습니다. "
221         "실제 신고·고지 금액과 다를 수 있습니다."
222     )
223

```

데이터 파이프라인

nfinity/data_pipeline/categorizer.py

36줄

```
1 # categorizer.py
2 """
3 팀원 C - 가맹점 카테고리 분류 모듈
4 가맹점명(merchant_name) 및 MCC 코드를 기반으로 카테고리를 분류합니다.
5 """
6
7 CATEGORY_MAP = {
8     # 🍔 식비
9     '스타벅스': '식비', '맥도날드': '식비', '배달의민족': '식비', '김밥천국': '식비',
10
11     # 🚗 교통
12     '카카오택시': '교통', '지하철': '교통', '버스': '교통', 'KTX': '교통',
13
14     # 🛒 쇼핑
15     '쿠팡': '쇼핑', '프리스비': '쇼핑', '올리브영': '쇼핑', '무신사': '쇼핑',
16     'Amazon_US': '쇼핑', 'AliExpress_CN': '쇼핑', '금은방_귀금속': '쇼핑',
17
18     # 💻 업무용
19     '어도비(Adobe)': '업무용', 'AWS': '업무용', '노션': '업무용', '패스트파이브': '업무용',
20
21     # 🏥 의료/건강
22     '병원': '의료/건강', '약국': '의료/건강',
23
24     # 🎮 기타 / 유희
25     '유희업소': '기타', 'Steam_Global': '기타', '게임아이템_스토어': '기타', '중고거래_송금': '기타'
26 }
27
28 def categorize_merchant(merchant_name: str) -> str:
29     """
30     가맹점명을 받아 카테고리를 반환합니다.
31     매핑되지 않은 가맹점은 '기타'로 분류합니다.
32     """
33     if not merchant_name or not isinstance(merchant_name, str):
34         return '기타'
35     return CATEGORY_MAP.get(merchant_name.strip(), '기타')
36
```

```

1 # cleanser.py
2 """
3 팀원 C - 데이터 정제 및 이상 데이터 검수 모듈
4 """
5 import pandas as pd
6
7 def clean_transactions(df: pd.DataFrame) -> pd.DataFrame:
8     """
9     mock_transactions.csv 데이터를 읽어와 결측치 처리 및 데이터 타입 정제 수행
10    """
11    df = df.copy()
12
13    # 1. 필수 입력 필드 누락 행 제거
14    df = df.dropna(subset=['user_id', 'amount', 'timestamp']).copy()
15
16    # 2. amount 숫자 타입 변환 및 0 이하 값 처리
17    df['amount'] = pd.to_numeric(df['amount'], errors='coerce')
18    df = df[df['amount'] > 0].copy()
19
20    # 3. timestamp 날짜시간 형변환
21    df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
22    df = df.dropna(subset=['timestamp']).copy()
23
24    # 4. mcc_code 문자열로 포맷팅
25    if 'mcc_code' in df.columns:
26        df['mcc_code'] = df['mcc_code'].fillna('').astype(str).str.replace('.', '', regex=False)
27
28    # 5. row_no 컬럼이 있다면 제거 (DB 스키마와 맞춤)
29    if 'row_no' in df.columns:
30        df = df.drop(columns=['row_no'])
31
32    return df
33

```

```

1  """
2  expense_classifier.py
3  2026 금융 AI Challenge - SideGig AI (AI 경비분류 프롬프트 로직 - 조소현님 설계, 은경 정리/연동)
4
5  역할: data_pipeline/categorizer.py(가맹점명 기반 카테고리 매핑)의 결과를 받아서,
6  '이 결제가 업무 경비로 인정될 확률(prob)'과 'AI 추천 태그(ai_tag)'를 붙이는 레이어.
7
8  ★ API 계약 (필드명 절대 변경 금지 - 프론트 C(하은)님과 이미 이 모양으로 합의됨):
9      {
10         "transactions_queue": [
11             {"id": 1, "merchant": "스타벅스", "amount": 15000, "ai_tag": "업무미팅", "prob": 92},
12             ...
13         ],
14         "current_estimated_refund": 125000
15     }
16
17 -----
18  8/28 은경이 조소현님 원본 파일에서 고친 부분 (버그 수정 - 로직/프롬프트는 그대로 유지):
19
20  1. 🌟 (중요) call_openai() 안에 실제 API 키로 보이는 문자열이 따옴표 없이 코드에 그대로
21  박혀 있었습니다: `os.environ.get(AQ.Ab8RN6...)` - 이걸 "AQ.Ab8RN6..." 라는 존재하지
22  않는 변수를 참조하는 문법 오류라 어차피 실행하면 NameError가 나지만, 진짜 API 키를
23  실수로 코드에 붙여넣은 것처럼 보여서 위험합니다. 소현님께 꼭 확인해서, 만약 진짜 키라면
24  지금 바로 재발급(rotate) 받으시길 권해드립니다 - 이 파일이 팀 zip으로 여러 번
25  돌아다녔으니까요. 이 버전에서는 정상적인 `os.environ.get("GEMINI_API_KEY")` 형태로
26  고쳤습니다.
27
28  2. 파일 위쪽에 MOCK_MODE가 두 번 정의되어 있었습니다 (7번째 줄 False, 46번째 줄 True).
29  두 번째 값이 항상 이기기 때문에 실제로는 계속 Mock으로만 동작했을 거예요. 환경변수
30  기준으로 자동 결정하도록 하나로 정리했습니다 (키가 있으면 실제 호출, 없으면 Mock).
31
32  3. run_batch()의 CSV 경로가 본인 컴퓨터 절대경로(`C:/Users/amro0/...`)로 하드코딩되어
33  있어서 다른 사람 컴퓨터/이 서버에서는 실행이 안 됐습니다 → 프로젝트 기준 상대경로로 변경.
34
35  4. import가 중복되어 있던 것 정리(json/os/random/pandas 위에서 한 번 더 있었음).
36
37  5. 로직(프롬프트, few-shot 예시, 규칙 기반 폴백, 하이브리드 판단)은 전부 소현님이 짰 그대로
38  유지했습니다 - 손댄 건 실행 환경 관련 버그뿐입니다.
39
40 -----
41
42  사용법:
43  - MOCK_MODE: 환경변수 GEMINI_API_KEY가 없으면 자동으로 Mock(규칙 기반 흉내) 모드로 동작.
44  인터넷이 안 되거나 키가 없는 환경에서도 파이프라인 전체가 도는지 바로 확인 가능.
45  - 실제 Gemini API를 쓰려면: `pip install google-genai` 후 .env에 GEMINI_API_KEY=... 추가.
46
47  """
48
49  from __future__ import annotations
50
51  import json
52  import os
53  from functools import lru_cache
54  from pathlib import Path
55
56  import pandas as pd
57
58  # google-genai 패키지가 없어도 이 파일을 import는 할 수 있게 (Mock 모드는 그대로 동작)
59  try:
60      from google import genai
61      from google.genai import types
62  except ImportError:
63      genai = None # type: ignore
64      types = None # type: ignore
65
66  # 키가 설정되어 있으면 실제 API, 없으면 Mock - 매번 코드를 고쳐가며 켜다 켜 필요 없게 자동화
67  # 9/6: gemini-2.0-flash가 서비스 종료되어 호출이 404로 실패하고 있었습니다. 응답 형태가
68  # 폴백과 같아서 밖에서는 알 수 없었고, /health도 "키가 있으니 gemini"라고만 보고했습니다.
69  # 모델명을 환경변수로 빼서 다음에 또 바뀌어도 코드 수정 없이 넘길 수 있게 합니다.
70  GEMINI_MODEL = os.environ.get("GEMINI_MODEL", "gemini-3.6-flash")

```

```

64
65 MOCK_MODE = genai is None or not os.environ.get("GEMINI_API_KEY")
66
67 # 마지막 실제 호출이 성공했는지 (None=아직 호출 안 함). /health가 이 값을 읽어서
68 # "키는 있는데 호출은 실패 중"인 상태를 드러냅니다.
69 LAST_CALL_OK = None
70
71 # 마진 세율 가정 (환급액 계산용 - 실제로는 페르소나별 과세표준에 맞게 조정 필요)
72 ASSUMED_MARGINAL_TAX_RATE = 0.15
73 # 이 확률 미만이면 '경비 승인'으로 자동 집계하지 않음 (환급액 계산 시 임계치)
74 APPROVAL_THRESHOLD = 50
75
76 DATA_DIR = Path(__file__).resolve().parent.parent / "data"
77
78
79 # =====
80 # 1. 프롬프트 설계 (조소현님 원안 그대로)
81 # =====
82 SYSTEM_PROMPT = """당신은 한국 프리랜서/N잡러의 결제 내역을 보고, 해당 결제가
83 '업무상 필요 경비(종합소득세 신고 시 경비 인정 가능)'인지 판단하는 세무 보조 AI입니다.
84
85 - 유저의 직업, 결제 가맹점, 결제 시간대를 함께 고려해서 판단하세요.
86   같은 가맹점이라도 직업/시간대에 따라 업무 경비일 확률이 달라집니다.
87   예) 평일 낮 시간대 카페 결제 + 프리랜서(마케터/디자이너) → 미팅/작업 공간 이용 가능성 높음(90%대)
88       IT 개발자가 소프트웨어/전자기기 판매처에서 결제 → 업무용 장비 가능성 높음(90%대 이상)
89       심야 시간대의 유흥/사행성 업종, 직업과 무관한 고액 결제 → 개인 지출 가능성 높음(10% 이하)
90
91 반드시 아래 JSON 형식으로만 답하세요. 다른 설명은 절대 추가하지 마세요.
92 {"ai_tag": "<10자 이내 짧은 태그, 예: 업무미팅, SW구독, 개인지출>", "prob": <0~100 사이 정수>}
93 """
94
95 FEW_SHOT_EXAMPLES = [
96     {
97         "input": {"직업": "프리랜서_마케터", "가맹점": "스타벅스", "시간대": "평일 오후 2시", "금액": 15000},
98         "output": {"ai_tag": "업무미팅", "prob": 92},
99     },
100     {
101         "input": {"직업": "IT_개발자", "가맹점": "프리스비", "시간대": "평일 오전", "금액": 3000000},
102         "output": {"ai_tag": "업무용장비", "prob": 95},
103     },
104     {
105         "input": {"직업": "IT_개발자", "가맹점": "Adobe Systems", "시간대": "평일 오전", "금액": 62000},
106         "output": {"ai_tag": "SW구독", "prob": 98},
107     },
108     {
109         "input": {"직업": "배달_투잡러", "가맹점": "유흥업소", "시간대": "심야 3시", "금액": 4500000},
110         "output": {"ai_tag": "개인지출", "prob": 3},
111     },
112 ]
113
114
115 def build_user_prompt(job: str, merchant: str, time_period: str, amount: int) -> str:
116     """기획서에 나온 형태 그대로: {"직업":..., "가맹점":..., "시간대":...} 프롬프트 구성"""
117     payload = {"직업": job, "가맹점": merchant, "시간대": time_period, "금액": amount}
118     return json.dumps(payload, ensure_ascii=False)
119
120
121 def time_period_label(hour: int) -> str:
122     if 0 <= hour < 6:
123         return "심야"
124     if 6 <= hour < 12:
125         return "평일 오전"
126     if 12 <= hour < 18:
127         return "평일 오후"
128     return "저녁"
129
130
131 # =====

```

```

132 # 2. Gemini API 호출 (실제 모드)
133 # =====
134 def call_gemini(job: str, merchant: str, time_period: str, amount: int) -> dict:
135     """실제 Gemini API 호출. GEMINI_API_KEY 환경변수가 있을 때만 사용됩니다."""
136     try:
137         api_key = os.environ.get("GEMINI_API_KEY")
138         if not api_key:
139             raise ValueError("환경변수에 GEMINI_API_KEY가 설정되어 있지 않습니다.")
140
141         full_system_instruction = SYSTEM_PROMPT + "\n\n[참고 예시]\n"
142         for ex in FEW_SHOT_EXAMPLES:
143             full_system_instruction += f"입력: {json.dumps(ex['input'], ensure_ascii=False)} / 출력: {json.dumps(ex['output'], ensure_ascii=False)}\n"
144
145         user_content = build_user_prompt(job, merchant, time_period, amount)
146
147         from app.gemini_client import generate_json
148
149         result = generate_json(full_system_instruction, user_content, temperature=0)
150         globals()["LAST_CALL_OK"] = True
151         return {"ai_tag": result["ai_tag"], "prob": int(result["prob"])}
152
153     except Exception as e:
154         globals()["LAST_CALL_OK"] = False
155         globals()["LAST_CALL_ERROR"] = f"{type(e).__name__}: {e}":200]
156         print(f"⚠ Gemini API 호출 실패({e}) → 규칙 기반 폴백 사용")
157         return rule_based_fallback(job, merchant)
158
159
160 # =====
161 # 3. 규칙 기반 폴백 / Mock 모드 (API 없이 로직 검증용)
162 # =====
163 BUSINESS_MERCHANTS = {"어도비(Adobe)": 98, "AWS": 97, "노션": 95, "패스트파이브": 90,
164                       "Adobe Systems": 98}
165 JOB_CONTEXT_MERCHANTS = {"스타벅스": ("업무미팅", 85), "맥도날드": ("업무미팅", 55),
166                          "프리스비": ("업무용장비", 92), "쿠팡": ("업무용구매", 45)}
167 PERSONAL_MERCHANTS = {"금문방_귀금속": 2, "중고거래_송금": 5, "유흥업소": 3,
168                       "Amazon_US": 20, "AliExpress_CN": 15, "Steam_Global": 8,
169                       "게임아이템_스토어": 5, "올리브영": 10, "무신사": 12}
170
171
172 def rule_based_fallback(job: str, merchant: str) -> dict:
173     """LLM 호출 없이/실패 시 대략적인 확률을 매기는 안전망.
174     실제 LLM 판단보다 거칠지만, 데모 중 API 장애로 전체가 멎는 것보다 낫습니다."""
175     if merchant in BUSINESS_MERCHANTS:
176         return {"ai_tag": "업무용구독/장비", "prob": BUSINESS_MERCHANTS[merchant]}
177     if merchant in PERSONAL_MERCHANTS:
178         return {"ai_tag": "개인지출", "prob": PERSONAL_MERCHANTS[merchant]}
179     if merchant in JOB_CONTEXT_MERCHANTS:
180         tag, prob = JOB_CONTEXT_MERCHANTS[merchant]
181         if "개발자" in job or "마케터" in job:
182             return {"ai_tag": tag, "prob": prob}
183         return {"ai_tag": tag, "prob": max(prob - 30, 10)}
184     return {"ai_tag": "미분류", "prob": 30}
185
186
187 def mock_llm_classify(job: str, merchant: str, time_period: str, amount: int) -> dict:
188     """MOCK_MODE용: 실제 LLM 호출 대신 few-shot 예시의 패턴을 흉내낸 결정론적 함수.
189     GEMINI_API_KEY를 설정하면 call_gemini()가 대신 실행됩니다."""
190     base = rule_based_fallback(job, merchant)
191     # 시간대 컨텍스트 살짝 반영 (기획서 예시처럼 '평일 낮'이면 가산)
192     if time_period in ("평일 오전", "평일 오후") and base["prob"] not in (2, 3, 5, 8):
193         base = {"**base", "prob": min(base["prob"] + 5, 99)}
194     if time_period == "심야" and merchant not in BUSINESS_MERCHANTS:
195         base = {"**base", "prob": max(base["prob"] - 20, 1)}
196     return base
197
198

```

```

199 # =====
200 # 4. 하이브리드 판단: 이미 확실한 건 LLM 호출 스킵 (비용/속도 절감)
201 # =====
202 HIGH_CONFIDENCE_BUSINESS = {"어도비(Adobe)", "AWS", "노션", "패스트파이브", "Adobe Systems"}
203 HIGH_CONFIDENCE_PERSONAL = {"금은방_귀금속", "중고거래_송금", "유흥업소", "게임아이템_스토어"}
204
205
206 def classify_one(job: str, merchant: str, hour: int, amount: int) -> dict:
207     time_period = time_period_label(hour)
208
209     # 이미 카테고리가 명백한 건 LLM 호출 없이 즉시 반환 (비용 절감)
210     if merchant in HIGH_CONFIDENCE_BUSINESS:
211         return rule_based_fallback(job, merchant)
212     if merchant in HIGH_CONFIDENCE_PERSONAL:
213         return rule_based_fallback(job, merchant)
214
215     # 애매한 케이스만 LLM(or mock)로 판단
216     if MOCK_MODE:
217         return mock_llm_classify(job, merchant, time_period, amount)
218     return dict(_call_gemini_cached(job, merchant, time_period, int(amount)))
219
220
221 def _redis_key(job: str, merchant: str, time_period: str, amount: int) -> str:
222     # 금액은 만원 단위로 뭉쳐서 캐시 적중률을 높입니다(같은 가맹점·같은 시간대면 금액이
223     # 조금 달라도 업무경비 여부 판단은 거의 같습니다).
224     return f"exp:{job}:{merchant}:{time_period}:{amount // 10000}"
225
226
227 def _cache_get(key: str):
228     try:
229         from app.redis_client import get_redis_client
230
231         raw = get_redis_client().get(key)
232         return json.loads(raw) if raw else None
233     except Exception:
234         return None
235
236
237 def _cache_set(key: str, value: dict) -> None:
238     try:
239         from app.redis_client import get_redis_client
240
241         # 만료를 두지 않습니다 - 한 번 분류한 결과는 계속 재사용해서, 할당량이 소진되거나
242         # 서버가 재시작돼도 화면에는 계속 실제 LLM 결과가 보이게 합니다.
243         get_redis_client().set(key, json.dumps(value, ensure_ascii=False))
244     except Exception:
245         pass
246
247
248 @lru_cache(maxsize=512)
249 def _call_gemini_cached(job: str, merchant: str, time_period: str, amount: int) -> tuple:
250     """같은 조건이면 같은 답이 나오므로 결과를 캐싱합니다. (9/5 추가)
251
252     화면 한 번을 그리는 데 이 함수가 여러 번 불립니다 - 업무경비 분류가 최근 20건,
253     Gig Score가 별도로 15건을 분류하기 때문입니다. 거래 목록에는 같은 가맹점이 반복해서
254     나오는데(구독·단골 카페 등) 캐시가 없으면 그때마다 Gemini를 새로 호출해서, 실제 키를
255     넣는 순간 대시보드 한 장에 수십 번의 LLM 왕복이 생깁니다.
256
257     dict는 캐시가 불가능해서(해시 불가) 항목 튜플로 저장했다가 호출부에서 다시 dict로
258     만듭니다. 프로세스 메모리에만 남고 재시작하면 비워집니다.
259     """
260     # 프로세스 캐시(lru_cache)는 재시작하면 사라지므로, 그 앞에 Redis 영구 캐시를 둡니다.
261     key = _redis_key(job, merchant, time_period, amount)
262     cached = _cache_get(key)
263     if cached:
264         return tuple(cached.items())
265
266     result = call_gemini(job, merchant, time_period, amount)

```

```

267 # 규칙 폴백 결과는 저장하지 않습니다 - 나중에 할당량이 회복됐을 때 실제 LLM 결과로
268 # 채워질 수 있어야 하기 때문입니다.
269 from app.gemini_client import LAST_CALL_OK
270
271 if LAST_CALL_OK:
272     _cache_set(key, result)
273 return tuple(result.items())
274
275
276 # =====
277 # 5. 배치 실행: mock_transactions.csv → API Contract 형태로 출력 (CSV 기준 - 로컬 테스트용)
278 # =====
279 def run_batch(transactions_csv: str | None = None, persona_csv: str | None = None,
280              sample_size=15, marginal_tax_rate=ASSUMED_MARGINAL_TAX_RATE) -> dict:
281     transactions_csv = transactions_csv or str(DATA_DIR / "mock_transactions.csv")
282     persona_csv = persona_csv or str(DATA_DIR / "persona_map.csv")
283
284     df = pd.read_csv(transactions_csv)
285     persona = pd.read_csv(persona_csv)[["user_id", "job"]]
286     df = df.merge(persona, on="user_id", how="left")
287     df["timestamp"] = pd.to_datetime(df["timestamp"])
288
289     # 데모/검증용으로 일부만 샘플링 (전체 4,978건을 다 부르면 비용·시간 부담)
290     sample = df.sample(n=min(sample_size, len(df)), random_state=42).reset_index(drop=True)
291
292     return _build_queue_response(
293         (row["job"], row["merchant_name"], row["timestamp"].hour, int(row["amount"]))
294         for _, row in sample.iterrows()
295     )
296
297
298 def _build_queue_response(rows) -> dict:
299     """(job, merchant, hour, amount) 튜플들을 받아 API 계약 모양으로 조립하는 공통 로직.
300     run_batch()(CSV 기준)와 app/routers/expense.py(실제 DB 기준)가 같이 씁니다."""
301     queue = []
302     total_refund = 0
303     for i, (job, merchant, hour, amount) in enumerate(rows):
304         result = classify_one(job, merchant, hour, amount)
305         queue.append({
306             "id": i + 1,
307             "merchant": merchant,
308             "amount": amount,
309             "ai_tag": result["ai_tag"],
310             "prob": result["prob"],
311         })
312         if result["prob"] >= APPROVAL_THRESHOLD:
313             total_refund += int(amount * ASSUMED_MARGINAL_TAX_RATE)
314     return {"transactions_queue": queue, "current_estimated_refund": total_refund}
315
316
317 if __name__ == "__main__":
318     output = run_batch()
319     print(f"MOCK_MODE = {MOCK_MODE} (실제 API 붙이려면 .env에 GEMINI_API_KEY 추가)")
320     print(json.dumps(output, ensure_ascii=False, indent=2))
321

```

```

1 # ingestion.py
2 """
3 팀원 C - 데이터 수집 및 PostgreSQL DB 적재 파이프라인
4
5 병합 후 변경점 (은경):
6 - categorizer/cleanser를 data_pipeline 패키지 상대 import로 바꿔서, 저장소 루트
7   어디서 실행하든(= `python -m data_pipeline.ingestion`) 동작하도록 했습니다.
8   (기존 `from categorizer import ...`는 이 스크립트가 있는 폴더를 CWD로 하고
9   직접 실행할 때만 동작해서, 다른 위치에서 import하면 실패했습니다.)
10 - CSV 기본 경로를 저장소의 data/ 폴더를 가리키도록 바꿨습니다.
11 """
12 import os
13 from pathlib import Path
14
15 import pandas as pd
16 from sqlalchemy import create_engine, text
17
18 try:
19     from data_pipeline.categorizer import categorize_merchant
20     from data_pipeline.cleanser import clean_transactions
21 except ImportError:
22     # 이 폴더 안에서 직접 `python ingestion.py`로 실행하는 경우를 위한 폴백
23     from categorizer import categorize_merchant
24     from cleanser import clean_transactions
25
26 # 팀원 A의 .env 및 docker-compose 설정 기준 DB 접속 URL
27 DB_URL = os.getenv("DATABASE_URL", "postgresql://sidegig:sidegig_pw@localhost:5432/sidegig")
28
29 DATA_DIR = Path(__file__).resolve().parent.parent / "data"
30
31
32 def sync_users(engine, df_transactions: pd.DataFrame, persona_map_path: str = None):
33     """
34     FOREIGN KEY 외래키 제약조건 방지를 위해, 거래 내역 삽입 전 users 테이블을 먼저 동기화합니다.
35     """
36     print("👤 users 테이블 동기화 시작...")
37
38     if persona_map_path is None:
39         persona_map_path = str(DATA_DIR / "persona_map.csv")
40
41     if os.path.exists(persona_map_path):
42         df_persona = pd.read_csv(persona_map_path)
43         df_persona = df_persona.rename(columns={'persona_name': 'name', 'job': 'persona'})
44         df_users = df_persona[['user_id', 'name', 'persona']].drop_duplicates(subset=['user_id'])
45     else:
46         # persona_map이 없을 경우 transaction의 unique user_id 추출
47         unique_users = df_transactions['user_id'].unique()
48         df_users = pd.DataFrame({
49             'user_id': unique_users,
50             'name': [f'사용자_{i+1}' for i in range(len(unique_users))],
51             'persona': '미지정'
52         })
53
54     with engine.begin() as conn:
55         for idx, row in df_users.iterrows():
56             stmt = text("""
57                 INSERT INTO users (user_id, name, persona)
58                 VALUES (:user_id, :name, :persona)
59                 ON CONFLICT (user_id) DO UPDATE
60                 SET name = EXCLUDED.name, persona = EXCLUDED.persona;
61             """)
62             conn.execute(stmt, {"user_id": row['user_id'], "name": str(row['name']), "persona": str(row['person
a'])})

```

```

63
64     # user_risk_profiles 도 함께 생성
65     stmt_profile = text("""
66         INSERT INTO user_risk_profiles (user_id)
67         VALUES (:user_id)
68         ON CONFLICT (user_id) DO NOTHING;
69     """)
70     conn.execute(stmt_profile, {"user_id": row['user_id']})
71
72     print("✅ users 및 user_risk_profiles 동기화 완료!")
73
74 def run_pipeline(csv_path: str = None, truncate_first: bool = True):
75     """
76     truncate_first=True(기본값): 다시 실행해도 안전하도록 매번 transactions를 비우고
77     새로 적재합니다. (transaction_id가 DB에서 자동 생성되는 UUID라, 이 옵션 없이
78     여러 번 실행하면 매번 중복 적재됩니다 - 개발 중 재실행 시 주의)
79     """
80     print("🚀 [탐원 C 파이프라인] 데이터 적재 시작...")
81
82     if csv_path is None:
83         csv_path = str(DATA_DIR / "mock_transactions.csv")
84
85     if not os.path.exists(csv_path):
86         print(f"❌ Error: {csv_path} 파일을 찾을 수 없습니다.")
87         return
88
89     # 1. CSV 데이터 읽기
90     print(f"1. {csv_path} 데이터 로드 중...")
91     df_raw = pd.read_csv(csv_path)
92
93     # 2. 데이터 정제
94     print("2. 데이터 정제(Cleansing) 진행 중...")
95     df_clean = clean_transactions(df_raw)
96
97     # 3. 카테고리 자동 분류
98     print("3. 가맹점 카테고리 매핑(Categorization) 진행 중...")
99     df_clean['category'] = df_clean['merchant_name'].apply(categorize_merchant)
100
101     # 4. DB 연결
102     print("4. PostgreSQL DB 연결 연결 시도 중...")
103     engine = create_engine(DB_URL)
104
105     # 5. 사용자(Users) 사전 등록 (외래키 에러 방지)
106     sync_users(engine, df_clean)
107
108     if truncate_first:
109         print("4-1. 재실행 대비 기존 transactions 비우는 중...")
110         with engine.begin() as conn:
111             conn.execute(text("TRUNCATE TABLE risk_events, transactions RESTART IDENTITY CASCADE;"))
112
113     # 6. transactions 테이블 적재
114     print("5. transactions 테이블에 데이터 적재 중...")
115     df_clean.to_sql('transactions', engine, if_exists='append', index=False)
116
117     print(f"🎉 [성공] {len(df_clean)}건 적재 완료!")
118
119 if __name__ == "__main__":
120     run_pipeline()
121

```

```

1  """
2  recurring_detector.py
3  2026 금융 AI Challenge - SideGig AI (팀원 B: 소현 - Day 6·8 남은 조각)
4
5  [이 파일이 하는 일 한 줄 요약]
6  mock_transactions.csv를 보고 "매달 반복해서 나가는 결제"(넷플릭스 구독료 같은 것)를
7  자동으로 찾아내고, 그걸 이번 달 예산 예측(/budgets/forecast)에 반영하는 로직입니다.
8
9  [To-do 3가지]
10  1) 정기결제 감지: 같은 유저 + 같은 가맹점 + 비슷한 금액(±5%) + 약 30일 간격 패턴 찾기
11  2) /budgets/forecast 예측식에 "아직 안 나온 정기결제 예상액" 더하기
12  3) budgets.alert_thresholds(50/80/90/100/120%)와 usage_rate 비교해서
13     지금 남은 임계값이 원지 리스트로 반환하기
14
15  [주의] /budgets/forecast 응답에 새로 추가되는 필드명은 은경님이 먼저 제안한 이름을
16  그대로 썼습니다: recurring_payments, crossed_thresholds
17  → 이 이름을 실제로 API에 반영하기 전에 최종 컨펌 받기.
18  """
19
20  import os
21  import pandas as pd
22  from datetime import datetime, timedelta
23
24  # -----
25  # 0. 설정값
26  # -----
27  AMOUNT_TOLERANCE = 0.05      # 금액이 이 비율(±5%) 안에서 다른 "같은 결제"로 취급
28  MIN_OCCURRENCES = 3          # 최소 3번은 반복돼야 "정기결제"로 인정 (2번은 우연히 겹칠 확률이 높음)
29
30  # 9/5 수정: 고정 30일(±5일) 대신 "주기가 일정한가"로 판정합니다 - 아래 detect_recurring_payments
31  # 주석 참고. 주간(7일)·격주(14일)·월간(30일) 결제를 모두 잡되 불규칙 결제는 계속 걸러냅니다.
32  MIN_INTERVAL_DAYS = 5        # 이보다 촘촘하면 정기결제가 아니라 일상 반복 소비로 봄
33  MAX_INTERVAL_DAYS = 40       # 이보다 뜸하면 이 데이터 기간(약 3개월)에서 주기 판단이 어려움
34  INTERVAL_RELATIVE_TOLERANCE = 0.25 # 모든 간격이 중앙값의 ±25% 안에 들어와야 "규칙적"
35
36  # -----
37  # 0-1. ★ 파일 경로 자동탐색 (expense_classifier.py와 똑같은 방식)
38  #
39  # __file__ = 지금 이 코드가 들어있는 파일 자체의 위치.
40  # 이걸 기준으로 삼으면 터미널을 어디서 열든 항상 같은 데이터 폴더를 찾습니다.
41  #
42  # 폴더 구조 가정:
43  #   sidegig_ai/
44  #   |   data/
45  #   |       |   mock_transactions.csv
46  #   |   data_pipeline/
47  #   |       |   recurring_detector.py
48  #   |
49  # -----
50  THIS_FILE_DIR = os.path.dirname(os.path.abspath(__file__))
51  DEFAULT_TX_CSV = os.path.join(THIS_FILE_DIR, "..", "data", "mock_transactions.csv")
52
53  # =====
54  # 1. 정기결제 감지
55  # =====
56  def detect_recurring_payments(df: pd.DataFrame) -> pd.DataFrame:
57      """
58      입력: user_id, merchant_name, amount, timestamp 컬럼이 있는 거래 내역 DataFrame
59      출력: 정기결제로 판단된 (유저, 가맹점) 조합마다 한 줄씩 정리된 DataFrame
60           [user_id, merchant_name, occurrences(반복횟수), avg_amount(평균금액),
61            avg_interval_days(평균간격), last_paid_date(마지막결제일),
62            next_expected_date(다음예상일)]
63  """

```

```

64 동작 순서 (같은 유저+가맹점 그룹마다 반복):
65 1) 시간순으로 정렬한다
66 2) 금액이 서로 ±5% 이내로 비슷한 거래들끼리 하나의 묶음(클러스터)으로 그룹핑한다
67   (같은 가맹점이라도 어쩌다 한 번 다른 금액을 결제했을 수 있으니까)
68 3) 그 묶음 안에서 결제일 간격들을 계산해서, 전부 25~35일 사이인지 확인한다
69 4) 조건을 만족하면 "정기결제"로 인정하고 결과에 추가한다
70 """
71 df = df.copy()
72 df["timestamp"] = pd.to_datetime(df["timestamp"])
73
74 results = []
75
76 # (유저, 가맹점) 쌍마다 따로 검사
77 for (user_id, merchant), group in df.sort_values("timestamp").groupby(["user_id", "merchant_name"]):
78     group = group.reset_index(drop=True)
79     if len(group) < MIN_OCCURRENCES:
80         continue # 애초에 반복 횟수가 부족하면 볼 것도 없음
81
82     used = [False] * len(group) # 이미 다른 클러스터에 포함된 거래는 다시 안 씀
83
84     for i in range(len(group)):
85         if used[i]:
86             continue
87
88         # i번째 거래를 기준으로, 금액이 ±5% 이내인 다른 거래들을 전부 모은다
89         cluster_idx = [i]
90         base_amount = group.loc[i, "amount"]
91         for j in range(i + 1, len(group)):
92             if used[j]:
93                 continue
94             if abs(group.loc[j, "amount"] - base_amount) <= base_amount * AMOUNT_TOLERANCE:
95                 cluster_idx.append(j)
96
97         if len(cluster_idx) < MIN_OCCURRENCES:
98             continue # 이 금액대로는 반복 횟수가 부족함
99
100         cluster = group.loc[cluster_idx].sort_values("timestamp").reset_index(drop=True)
101
102         # 연속된 결제일 사이의 간격(일수)을 계산 (예: [30, 31, 29] 이런 식으로)
103         intervals = cluster["timestamp"].diff().dt.days.dropna()
104         if intervals.empty:
105             continue
106
107         # 9/5 수정 - 원래는 "모든 간격이 25~35일"일 때만 정기결제로 인정했습니다.
108         # 그러다 보니 월 구독만 잡히고 주간(배달 앱 구독)·격주 결제는 전부 놓쳤고,
109         # 실제로 데모 페르소나의 45일 현금흐름 시뮬레이션에 투사된 정기 지출이
110         # 단 한 건도 없어서(수입 계단만 있고 지출은 매일 균등 감소) 예측 그래프가
111         # 사실상 직선이 되는 문제가 있었습니다.
112         #
113         # 대신 "간격이 얼마나 일정한가"로 판정합니다: 간격의 중앙값이 주간~월간
114         # 범위(5~40일) 안에 있고, 모든 간격이 그 중앙값의 ±25% 안에 들어오면
115         # 정기결제로 봅니다. 주기가 7일이든 14일이든 30일이든 규칙적이기만 하면
116         # 잡히고, 불규칙한 결제(스타벅스처럼)는 여전히 걸러집니다.
117         median_interval = float(intervals.median())
118         if not (MIN_INTERVAL_DAYS <= median_interval <= MAX_INTERVAL_DAYS):
119             continue
120         tolerance = median_interval * INTERVAL_RELATIVE_TOLERANCE
121         is_regular = intervals.between(
122             median_interval - tolerance,
123             median_interval + tolerance,
124         ).all()
125         if not is_regular:
126             continue
127
128         # 이 클러스터에 쓰인 거래들은 "이미 처리됨" 표시 (중복 집계 방지)
129         for idx in cluster_idx:
130             used[idx] = True
131

```

```

132         avg_amount = int(cluster["amount"].mean())
133         avg_interval = round(intervals.mean(), 1)
134         last_paid = cluster["timestamp"].max()
135         next_expected = last_paid + timedelta(days=round(avg_interval))
136
137         results.append({
138             "user_id": user_id,
139             "merchant_name": merchant,
140             "occurrences": len(cluster),
141             "avg_amount": avg_amount,
142             "avg_interval_days": avg_interval,
143             "last_paid_date": last_paid.date().isoformat(),
144             "next_expected_date": next_expected.date().isoformat(),
145         })
146
147     return pd.DataFrame(results)
148
149
150 # =====
151 # 2. /budgets/forecast에 "아직 안 나온 정기결제 예상액" 더하기
152 # =====
153 def forecast_month_end(user_id: str, transactions_df: pd.DataFrame, recurring_df: pd.DataFrame,
154                        current_date: datetime) -> dict:
155     """
156     기존 예측 방식: "이번 달 지금까지 쓴 돈 ÷ 지난 날 수" = 하루 평균 지출
157                     → 하루 평균 지출 × 남은 날 수를 더해서 "이번 달 말 예상 총 지출" 계산
158
159     여기에 추가하는 것: 이 유저의 정기결제 중, "다음 예상일이 이번 달 안에 있고
160     아직 지나지 않은 것들"의 금액을 더함 (예: 넷플릭스 결제일이 아직 안 왔으면 그만큼 더 나갈 걸 미리 반영)
161     """
162     month_start = current_date.replace(day=1)
163     days_elapsed = (current_date - month_start).days + 1
164
165     # 이번 달이 총 며칠까지 있는지 계산 (28일이 있는 달에 4일 더한 뒤 그 달 1일로 되돌리는 트릭)
166     next_month = (month_start.replace(day=28) + timedelta(days=4)).replace(day=1)
167     days_in_month = (next_month - month_start).days
168     remaining_days = days_in_month - days_elapsed
169
170     tx = transactions_df.copy()
171     tx["timestamp"] = pd.to_datetime(tx["timestamp"])
172     this_month_tx = tx[(tx["user_id"] == user_id) &
173                        (tx["timestamp"] >= month_start) &
174                        (tx["timestamp"] <= current_date)]
175
176     spent_so_far = int(this_month_tx["amount"].sum())
177     avg_daily_spend = spent_so_far / days_elapsed if days_elapsed > 0 else 0
178     base_forecast = spent_so_far + avg_daily_spend * remaining_days # 기존 방식 그대로
179
180     # 이 유저의 정기결제 중 "이번 달 안에, 아직 안 지나간 것"만 골라냄
181     user_recurring = recurring_df[recurring_df["user_id"] == user_id].copy()
182     recurring_payments = []
183     recurring_addition = 0
184     if not user_recurring.empty:
185         user_recurring["next_expected_date"] = pd.to_datetime(user_recurring["next_expected_date"])
186         upcoming = user_recurring[
187             (user_recurring["next_expected_date"] > current_date) &
188             (user_recurring["next_expected_date"] < next_month)
189         ]
190         for _, row in upcoming.iterrows():
191             recurring_payments.append({
192                 "merchant_name": row["merchant_name"],
193                 "expected_amount": int(row["avg_amount"]),
194                 "expected_date": row["next_expected_date"].date().isoformat(),
195             })
196         recurring_addition = int(upcoming["avg_amount"].sum())
197
198     return {
199         "forecast_amount": int(base_forecast + recurring_addition), # 최종 예측치 (정기결제 반영)

```

```

200     "base_forecast_amount": int(base_forecast),          # 참고용: 정기결제 반영 전 값
201     "recurring_payments": recurring_payments,          # ★ 은경님 제안 필드명
202 }
203
204
205 # =====
206 # 3. 예산 임계값(alert_thresholds) 체크
207 # =====
208 def check_crossed_thresholds(usage_rate: float, alert_thresholds=(50, 80, 90, 100, 120)) -> list:
209     """
210     usage_rate: budgets 테이블의 usage_rate 그대로 (0.83 = 83% 사용했다는 뜻)
211     반환: 지금 usage_rate가 이미 넘은 임계값들의 리스트, 오름차순 정렬
212         예) usage_rate=0.83 -> [50, 80] (90/100/120은 아직 안 넘었으니 제외)
213
214     신호등 색으로 바꾸는 건 프론트(하은님) 몫이라, 여기선 숫자 리스트만 돌려줍니다.
215     """
216     usage_pct = usage_rate * 100
217     return [t for t in sorted(alert_thresholds) if usage_pct >= t]
218
219
220 # =====
221 # 4. 이 파일을 직접 실행했을 때 (python recurring_detector.py)
222 #     - 실제 데이터로 감지 + 합성 데이터로 로직 자체 검증 + 데모 출력까지 한번에
223 # =====
224 if __name__ == "__main__":
225     print(f"[안내] 거래 데이터 경로: {DEFAULT_TX_CSV}\n")
226
227     # --- 4-1. 실제 mock_transactions.csv로 정기결제 탐지 ---
228     real_df = pd.read_csv(DEFAULT_TX_CSV)
229     real_recurring = detect_recurring_payments(real_df)
230     print("=" * 55)
231     print(f"실제 데이터에서 발견된 정기결제 후보: {len(real_recurring)}건")
232     if not real_recurring.empty:
233         print(real_recurring.head(10).to_string(index=False))
234     else:
235         print("(정기결제 패턴이 감지되지 않았습니다. 아래 4-2는 로직 자체 검증용 테스트입니다.)")
236
237     # --- 4-2. 로직이 제대로 동작하는지 확인하는 합성(가짜) 테스트 데이터 ---
238     #     "넷플릭스 매달 17,000원" 패턴은 잡히고, 간격이 불규칙한 스타벅스는 걸러지는지 확인
239     print("\n" + "=" * 55)
240     print("[자체 검증] 넷플릭스 매달 17,000원 패턴 합성 테스트")
241     synthetic = pd.DataFrame([
242         {"user_id": "test-user-1", "merchant_name": "넷플릭스", "amount": 17000, "timestamp": "2026-05-15 10:00:00"},
243         {"user_id": "test-user-1", "merchant_name": "넷플릭스", "amount": 17000, "timestamp": "2026-06-14 10:00:00"},
244         {"user_id": "test-user-1", "merchant_name": "넷플릭스", "amount": 17400, "timestamp": "2026-07-16 10:00:00"},
245         {"user_id": "test-user-1", "merchant_name": "스타벅스", "amount": 5000, "timestamp": "2026-06-01 09:00:00"},
246         {"user_id": "test-user-1", "merchant_name": "스타벅스", "amount": 5200, "timestamp": "2026-06-20 09:00:00"},
247     ])
248     synthetic_recurring = detect_recurring_payments(synthetic)
249     print(synthetic_recurring.to_string(index=False))
250     assert len(synthetic_recurring) == 1, "넷플릭스 패턴 1건만 잡혀야 합니다"
251     assert synthetic_recurring.iloc[0]["merchant_name"] == "넷플릭스"
252     print("✅ 합성 테스트 통과 (넷플릭스만 정기결제로 잡히고, 간격 불규칙한 스타벅스는 제외됨)")
253
254     # --- 4-3. forecast 계산 데모 ---
255     print("\n" + "=" * 55)
256     print("[데모] /budgets/forecast 응답 예시 (정기결제 반영)")
257     demo_tx = pd.concat([synthetic, pd.DataFrame([
258         {"user_id": "test-user-1", "merchant_name": "쿠팡", "amount": 32000, "timestamp": "2026-08-03 14:00:00"},
259         {"user_id": "test-user-1", "merchant_name": "김밥천국", "amount": 8000, "timestamp": "2026-08-07 12:30:00"},
260     ])], ignore_index=True)
261     fake_now = datetime(2026, 8, 10) # 넷플릭스 다음 예상일(8/16)보다 이전 시점
262     forecast = forecast_month_end("test-user-1", demo_tx, synthetic_recurring, fake_now)
263     print(f"기준일: {fake_now.date()} (이번 달 지출 40,000원 발생 + 넷플릭스 8/16 예정)")
264     print(forecast)

```

```
265
266 # --- 4-4. 임계값 체크 데모 ---
267 print("\n" + "=" * 55)
268 print("[데모] check_crossed_thresholds 예시")
269 for rate in [0.35, 0.55, 0.83, 1.05]:
270     print(f"usage_rate={rate} -> crossed_thresholds={check_crossed_thresholds(rate)}")
271
```

스크립트

nfinity/scripts/build_user_profiles.py

93줄

```
1  """
2  Day 3 작업: user_risk_profiles를 "진짜" 값으로 채우는 배치 스크립트.
3
4  지금까지는 sql/init.sql / ingestion.py의 sync_users()가 user_risk_profiles를
5  전부 기본값(단일한도 50만원, 일일한도 100만원 등)으로만 만들어왔습니다.
6  이 스크립트는 각 유저의 실제 거래 이력(transactions 테이블)을 바탕으로
7  평균/표준편차/한도를 계산해서 덮어씁니다.
8
9  설계서(리스크방어 및 예산통제 시스템 아키텍처 설계서.pdf, 3.4.5절)의 로직을
10 단순화해서 옮겼습니다:
11     - avg / std / p95 거래금액
12     - single_transaction_limit = p95 거래금액 * 3
13     - daily_limit = 평균 거래금액 * 하루 평균 거래건수 * 3 (단, single_transaction_limit보다는 크게)
14     - allowed_countries = 'KR' + 이력에서 실제로 5% 이상 비중을 차지한 해외 국가
15
16 실행:
17     python -m scripts.build_user_profiles
18 """
19 import os
20
21 import numpy as np
22 import pandas as pd
23 from sqlalchemy import create_engine, text
24
25 DB_URL = os.getenv("DATABASE_URL", "postgresql://sidegig:sidegig_pw@localhost:5432/sidegig")
26
27 MIN_TXN_FOR_REAL_PROFILE = 5 # 이보다 거래가 적으면 기본값 유지 (신뢰 불가)
28
29
30 def build_profiles():
31     engine = create_engine(DB_URL)
32
33     print("1. transactions 테이블에서 유저별 거래 이력 로드 중...")
34     df = pd.read_sql(text("SELECT user_id, amount, timestamp, country FROM transactions"), engine)
35
36     if df.empty:
37         print("⚠ transactions가 비어 있습니다. 먼저 data_pipeline/ingestion.py를 실행하세요.")
38         return
39
40     updated, skipped = 0, 0
41
42     with engine.begin() as conn:
43         for user_id, g in df.groupby("user_id"):
44             n = len(g)
45             if n < MIN_TXN_FOR_REAL_PROFILE:
46                 skipped += 1
47                 continue
48
49             avg_amt = float(g["amount"].mean())
50             std_amt = float(g["amount"].std(ddof=0) or 0.0)
51             p95_amt = float(np.percentile(g["amount"], 95))
52
53             span_days = max((g["timestamp"].max() - g["timestamp"].min()).days, 1)
54             avg_daily_txns = n / span_days
55
56             single_limit = max(p95_amt * 3, avg_amt * 2) # 최소한 평균의 2배는 되도록 보장
57             daily_limit = max(avg_amt * avg_daily_txns * 3, single_limit * 1.2)
58
59             country_share = g["country"].value_counts(normalize=True)
60             allowed = {"KR"} | set(country_share[country_share >= 0.05].index)
```

```

61
62         conn.execute(
63             text(
64                 """
65                 UPDATE user_risk_profiles
66                 SET avg_transaction_amount = :avg_amt,
67                     std_transaction_amount = :std_amt,
68                     single_transaction_limit = :single_limit,
69                     daily_limit = :daily_limit,
70                     avg_daily_transactions = :avg_daily_txns,
71                     allowed_countries = :allowed,
72                     updated_at = now()
73                 WHERE user_id = :uid
74                 """
75             ),
76             {
77                 "avg_amt": avg_amt,
78                 "std_amt": std_amt,
79                 "single_limit": single_limit,
80                 "daily_limit": daily_limit,
81                 "avg_daily_txns": avg_daily_txns,
82                 "allowed": list(allowed),
83                 "uid": user_id,
84             },
85         )
86         updated += 1
87
88     print(f"✅ 완료: {updated}명 프로파일 갱신, {skipped}명은 거래 이력이 {MIN_TXN_FOR_REAL_PROFILE}건 미만이라 기본값
유지")
89
90
91 if __name__ == "__main__":
92     build_profiles()
93

```

```

1  """
2  9/2 추가 - 수입 플랫폼 연결 데모용 시드 스크립트 (1회성, 재실행해도 안전 - idempotent)
3
4  "여러 플랫폼에서 오는 수입이 정신없다"는 N잡러의 핵심 페인포인트를 보여주기 위해,
5  계좌/플랫폼을 실제로 연동하는 대신(이번 MVP 범위 밖) 각 페르소나별로 그럴듯한 수입
6  플랫폼 2~3개와 그 정산 내역을 미리 만들어둡니다. income_sources.connected가 false인
7  동안 프론트는 이 데이터를 보여주지 않다가, "연결하기" 버튼(POST
8  /api/v1/income/sources/{id}/connect)을 누르는 순간 true로 바뀌면서 이미 DB에 있던
9  이 데이터가 드러납니다 - 실제 마이데이터 연동과 사용자 체감이 비슷하도록 일부러 이렇게
10 설계했습니다.
11
12 각 페르소나마다 플랫폼 하나는 일부러 미연결(connected=False) 상태로 남겨둡니다 -
13 데모에서 "연결하기"를 실제로 눌러볼 거리가 있어야 하기 때문입니다.
14
15 금액/직업은 app/demo_personas.py의 job/story와 mock_transactions.csv 기반 실제 지출
16 규모(scripts/seed_demo_personas.py 실행 결과로 확인한 이번 달 지출 합계)를 참고해서,
17 "이 정도는 벌어야 저 정도를 쓰는 게 말이 된다"는 수준으로 정했습니다(완전 무작위 아님).
18
19 날짜는 app/demo_clock.py의 get_demo_now() 기준(=mock 거래 데이터의 마지막 시각,
20 2026-08-22)으로 계산합니다 - budgets.py와 동일한 "데모 기준 시각"을 씁니다. 그래야
21 "이번 주/이번 달 수입" 집계가 실서버 시각이 아니라 데이터 기준으로 안정적으로 나옵니다.
22
23 실행: python3 scripts/seed_demo_income.py (로컬 PostgreSQL이 떠 있어야 함)
24 """
25 import os
26 import random
27 import sys
28 from datetime import timedelta
29 from pathlib import Path
30
31 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
32
33 os.environ.setdefault("DATABASE_URL", "postgresql://sidegig:sidegig_pw@localhost:5432/sidegig")
34
35 from sqlalchemy import text # noqa: E402
36
37 from app.database import SessionLocal # noqa: E402
38 from app.demo_clock import get_demo_now # noqa: E402
39 from app.demo_personas import DEMO_PERSONAS # noqa: E402
40
41 random.seed(42)
42
43 # 페르소나별 수입 플랫폼 구성. 각 항목: (플랫폼명, 유형, 이모지, 정산주기(일), 1회 평균액,
44 # 변동폭(0~1), 연결 상태). 정산주기가 짧을수록(배달) 이벤트가 자주, 길수록(본업) 드물게
45 # 생성됩니다. 리스트의 마지막 항목을 일부러 미연결로 둡니다.
46 INCOME_PLATFORMS = {
47     "c5dd4004-498c-4409-9472-7a0e457ff8f1": [ # 김민철 - 배달 투잡러
48         ("본업 급여", "본업", "💰", 30, 2400000, 0.02, True),
49         ("배달의민족", "배달", "🍔", 7, 280000, 0.3, True),
50         ("쿠팡이츠", "배달", "🚲", 7, 150000, 0.35, False),
51     ],
52     "6d422b5f-1b23-4312-bbfc-4ef97e5fc845": [ # 박지수 - 프리랜서 마케터
53         ("클라이언트 직접입금", "프리랜서", "💰", 14, 1200000, 0.5, True),
54         ("크몽", "프리랜서", "📁", 10, 450000, 0.4, True),
55         ("숨고", "프리랜서", "🌱", 12, 300000, 0.45, False),
56     ],
57     "131b9d96-aa5a-4938-844d-2d08b7a2df9a": [ # 이하늘 - IT 개발자
58         ("회사 급여", "본업", "💰", 30, 4800000, 0.02, True),
59         ("외주 프로젝트", "프리랜서", "💻", 21, 2000000, 0.6, True),
60         ("개인 유튜브", "콘텐츠", "📺", 30, 90000, 0.5, False),
61     ],
62     "a0598579-8124-47ac-ab26-dc5e2c7c8c1a": [ # 최유진 - 크리에이터 인플루언서
63         ("유튜브 애드센스", "콘텐츠", "📺", 30, 3200000, 0.3, True),

```

```

64     ("브랜드 협찬", "콘텐츠", "👉", 18, 2500000, 0.6, True),
65     ("굿즈 스토어", "커머스", "🛒", 14, 800000, 0.5, False),
66 ],
67 "c781347d-a5e7-47fc-87ab-4f298190cdfb": [ # 정다운 - 감사 겸 배민커넥트
68     ("강의료 정산", "프리랜서", "👤", 30, 1800000, 0.25, True),
69     ("배민커넥트", "배달", "🍕", 7, 220000, 0.3, True),
70     ("온라인 강의 플랫폼", "프리랜서", "🎧", 30, 350000, 0.4, False),
71 ],
72 }
73
74 PERIOD_START_OFFSET_DAYS = 90 # get_demo_now() 기준 90일 전부터 수입 이벤트 생성
75
76
77 def _reset(db, user_ids):
78     db.execute(text("DELETE FROM income_events WHERE user_id = ANY(:uids)"), {"uids": user_ids})
79     db.execute(text("DELETE FROM income_sources WHERE user_id = ANY(:uids)"), {"uids": user_ids})
80     db.commit()
81
82
83 def seed_income(db):
84     demo_now = get_demo_now(db)
85     period_start = demo_now - timedelta(days=PERIOD_START_OFFSET_DAYS)
86     user_ids = list(INCOME_PLATFORMS.keys())
87     _reset(db, user_ids)
88
89     name_by_id = {p["user_id"]: p["name"] for p in DEMO_PERSONAS}
90
91     for uid, platforms in INCOME_PLATFORMS.items():
92         print(f"\n[수입] {name_by_id.get(uid, uid)}")
93         for platform_name, platform_type, icon, cycle_days, avg_amount, variance, connected in platforms:
94             source_id = db.execute(
95                 text(
96                     """
97                     INSERT INTO income_sources
98                     (user_id, platform_name, platform_type, icon_emoji, connected, connected_at)
99                     VALUES (:uid, :name, :type, :icon, :connected, CASE WHEN :connected THEN now() ELSE NULL END)
100                     RETURNING source_id
101                     """
102             ),
103             {"uid": uid, "name": platform_name, "type": platform_type, "icon": icon, "connected": connected},
104         ).scalar()
105
106         # 정산주기(cycle_days)마다 한 건씩, period_start ~ demo_now+7일(정산예정 포함)까지 생성.
107         # demo_now를 넘어가는 회차는 '정산완료', 그 이전은 '정산예정'.
108         t = period_start
109         count, total = 0, 0
110         while t <= demo_now + timedelta(days=7):
111             amount = max(10000, int(avg_amount * (1 + random.uniform(-variance, variance))))
112             status = "정산완료" if t <= demo_now else "정산예정"
113             db.execute(
114                 text(
115                     """
116                     INSERT INTO income_events (source_id, user_id, amount, memo, status, settled_at)
117                     VALUES (:sid, :uid, :amount, :memo, :status, :settled_at)
118                     """
119             ),
120             {
121                 "sid": source_id, "uid": uid, "amount": amount,
122                 "memo": f"{platform_name} 정산", "status": status, "settled_at": t,
123             },
124         )
125         count += 1
126         total += amount
127         t += timedelta(days=cycle_days)
128     db.commit()
129     tag = "연결됨" if connected else "미연결(데모용)"
130     print(f" {icon} {platform_name} ({platform_type}, {tag}): {count}건, 누적 {total:,}원")
131

```

```
132
133 if __name__ == "__main__":
134     db = SessionLocal()
135     try:
136         seed_income(db)
137         print("\n완료.")
138     finally:
139         db.close()
140
```

```

1  """
2  8/30 데모 모드 준비 스크립트 (1회성, 재실행해도 안전 - idempotent)
3
4  공모전 제출은 "내 정보를 직접 입력"하는 게 아니라 "버튼 하나로 미리 만들어둔 인물의
5  폰을 잠깐 구경하는" 데모 방식으로 가기로 했습니다(가상 데이터 사용은 데이콘 운영진이
6  게시판에서 공식 허용 - 화면에 안내 문구만 넣으면 됨). 그래서 이야기가 뚜렷한 5명의
7  페르소나(app/demo_personas.py에 정의, seed_data.py의 story_flag 기준)를 골라서, 그
8  사람들의 *실제* 거래 내역(mock_transactions.csv 기반, DB에 이미 들어있음)을 가지고:
9
10  1. 카테고리별 이번 달 실제 지출 합계를 계산해서 Redis 예산 카운터에 반영합니다
11     (지금까지는 이 카운터가 실제 거래와 연결되지 않은 채 비어 있었습니다 - budgets.py
12     주석에 있던 "나중엔 자동으로 연결" 부분을 데모용으로 채우는 작업입니다. 숫자를
13     지어내는 게 아니라, DB에 이미 있는 실제 mock 거래를 합산해서 반영하는 것입니다).
14  2. 각 페르소나의 전체 거래를 시간순으로 리스크 평가 로직에 실제로 흘려보내서
15     risk_events를 채웁니다 (지금까지는 이 테이블이 비어있어서 리스크 점수가 항상 0이었습니다).
16  3. 카테고리별 예산 한도(budgets 테이블)를 만듭니다 - 한도 자체는 "이 페르소나라면 이
17     정도가 적당하다"는 판단이 들어간 값이지만, 지출액은 위 1번처럼 실제 데이터입니다.
18
19  페르소나 목록 자체는 app/demo_personas.py 한 곳에서만 관리합니다(프론트가 부르는
20  GET /api/v1/demo/personas도 같은 파일을 씁니다) - 목록이 두 군데서 따로 관리되며
21  어긋나는 걸 막기 위함입니다.
22
23  실행: python3 scripts/seed_demo_personas.py (로컬 PostgreSQL/Redis가 떠 있어야 함)
24
25  9/2 버그 수정: "이번 달"을 실서버 시각(date.today())으로 잡고 있었는데, mock 데이터가
26  2026-08-22에 끝나서 실제 달력이 9월로 넘어가면(심사 기간 9/7~9/11 내내 그렇습니다) 이
27  스크립트가 매번 "이번 달 실제 지출 합계 0원"을 채우게 되는 문제가 있었습니다 - 실제로
28  9/2에 재현해서 확인했습니다. app/demo_clock.py의 get_demo_now()(=mock 데이터의 가장
29  최근 거래 시각, 즉 8/22)를 "오늘"로 쓰도록 고쳤습니다. Redis도 이제 이 기준 시각으로
30  키를 저장해야 budgets.py가 조회할 때(같은 함수로 같은 기준 시각을 씀) 값이 맞습니다.
31  """
32  import os
33  import sys
34  from pathlib import Path
35
36  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
37
38  os.environ.setdefault("DATABASE_URL", "postgresql://sidegig:sidegig_pw@localhost:5432/sidegig")
39  os.environ.setdefault("REDIS_URL", "redis://localhost:6379/0")
40
41  from sqlalchemy import text # noqa: E402
42
43  from app.database import SessionLocal # noqa: E402
44  from app.demo_clock import get_demo_now # noqa: E402
45  from app.demo_personas import CATEGORIES, DEMO_PERSONAS, PERIOD # noqa: E402
46  from app.redis_client import add_spending, get_redis_client # noqa: E402
47  from data_pipeline.categorizer import categorize_merchant # noqa: E402
48
49
50  def _reset_demo_redis_keys():
51      """재실행해도 중복 누적되지 않도록, 이 스크립트가 채우는 5명의 budget:* 키를 먼저 지웁니다.
52      (Redis DEL은 와일드카드를 지원하지 않아서 KEYS로 실제 키를 찾아서 지웁니다.)"""
53      client = get_redis_client()
54      for p in DEMO_PERSONAS:
55          keys = client.keys(f"budget:{p['user_id']}:{PERIOD}:*")
56          if keys:
57              client.delete(*keys)
58
59
60  def seed_budgets_and_spending(db):
61      today = get_demo_now(db).date()
62      month_start = today.replace(day=1)
63      print(f"[예산/지출] 이번 달 기준(데모 데이터 기준 시각): {month_start} ~ {today}")

```

```

64
65     for p in DEMO_PERSONAS:
66         uid = p["user_id"]
67         rows = db.execute(
68             text(
69                 "SELECT merchant_name, amount FROM transactions "
70                 "WHERE user_id = :uid AND timestamp >= :start AND timestamp <= :end"
71             ),
72             {"uid": uid, "start": month_start, "end": today},
73         ).mappings().all()
74
75     spend_by_cat = {c: 0.0 for c in CATEGORIES}
76     for r in rows:
77         cat = categorize_merchant(r["merchant_name"])
78         spend_by_cat[cat] = spend_by_cat.get(cat, 0.0) + float(r["amount"])
79
80     for cat in CATEGORIES:
81         limit_amount = p["limits"][cat]
82         db.execute(
83             text(
84                 """
85                 INSERT INTO budgets (user_id, category, period, limit_amount)
86                 VALUES (:uid, :cat, :period, :limit_amount)
87                 ON CONFLICT (user_id, category, period) DO UPDATE
88                 SET limit_amount = EXCLUDED.limit_amount, updated_at = now()
89                 """
90             ),
91             {"uid": uid, "cat": cat, "period": PERIOD, "limit_amount": limit_amount},
92         )
93         spent = spend_by_cat[cat]
94         if spent > 0:
95             add_spending(uid, cat, spent, PERIOD, now=get_demo_now(db))
96     db.commit()
97     total = sum(spend_by_cat.values())
98     print(f" {p['name']}({p['job']}), {p['story']}: 이번 달 실지출 합계 {total:,.0f}원 "
99           f"({', '.join(f'{c} {v:,.0f}' for c, v in spend_by_cat.items() if v > 0)})")
100
101
102 def replay_risk_assessments(db):
103     from app.models import Transaction
104     from app.routers.risk import assess_transaction
105
106     print("\n[리스크] 전체 거래 시간순 재생 (risk_events 채우기)")
107     for p in DEMO_PERSONAS:
108         uid = p["user_id"]
109         # 재실행해도 중복 기록되지 않도록, 이 유저의 기존 risk_events를 먼저 지웁니다.
110         db.execute(text("DELETE FROM risk_events WHERE user_id = :uid"), {"uid": uid})
111         db.commit()
112         rows = db.execute(
113             text("SELECT * FROM transactions WHERE user_id = :uid ORDER BY timestamp"),
114             {"uid": uid},
115         ).mappings().all()
116
117         flagged = 0
118         for r in rows:
119             txn = Transaction(
120                 transaction_id=str(r["transaction_id"]),
121                 user_id=r["user_id"],
122                 amount=float(r["amount"]),
123                 merchant_id=r["merchant_id"],
124                 merchant_name=r["merchant_name"],
125                 mcc_code=r["mcc_code"],
126                 category=r["category"],
127                 timestamp=r["timestamp"],
128                 latitude=r["latitude"],
129                 longitude=r["longitude"],
130                 country=r["country"] or "KR",
131                 device_id=r["device_id"],

```

```
132         )
133         result = assess_transaction(txn, db=db)
134         if result.triggered_rules:
135             flagged += 1
136         print(f" {p['name']}: 거래 {len(rows)}건 중 {flagged}건 리스크 이벤트 기록")
137
138
139 if __name__ == "__main__":
140     _reset_demo_redis_keys()
141     db = SessionLocal()
142     try:
143         seed_budgets_and_spending(db)
144         replay_risk_assessments(db)
145     finally:
146         db.close()
147     print("\n완료.")
148
```

테스트

nfinity/tests/test_rule_engine.py

105줄

```
1 """
2 룰 엔진 단위 테스트. DB/도커 없이 바로 실행 가능합니다.
3
4 실행법:
5 /tmp/venv/bin/python -m pytest tests/test_rule_engine.py -v
6 (또는 프로젝트 루트에서 pip install -r requirements.txt pytest 후 pytest 실행)
7 """
8 from datetime import datetime
9
10 from app.models import RuleAction, Transaction, UserRiskProfile
11 from app.rule_engine import RuleEngine
12
13
14 def make_profile(**kwargs) -> UserRiskProfile:
15     defaults = dict(
16         user_id="u_test",
17         avg_transaction_amount=20000,
18         single_transaction_limit=500000,
19         daily_limit=1000000,
20         allowed_countries=["KR"],
21     )
22     defaults.update(kwargs)
23     return UserRiskProfile(**defaults)
24
25
26 def make_txn(**kwargs) -> Transaction:
27     defaults = dict(
28         user_id="u_test",
29         amount=15000,
30         merchant_id="m_1",
31         merchant_name="테스트 가맹점",
32         timestamp=datetime(2026, 8, 23, 14, 0, 0),
33         country="KR",
34     )
35     defaults.update(kwargs)
36     return Transaction(**defaults)
37
38
39 def test_normal_transaction_triggers_nothing():
40     profile = make_profile()
41     txn = make_txn(amount=15000)
42     engine = RuleEngine(history=[])
43     results = engine.evaluate(txn, profile)
44     assert results == []
45
46
47 def test_single_transaction_limit_exceeded():
48     profile = make_profile(single_transaction_limit=500000)
49     txn = make_txn(amount=900000)
50     engine = RuleEngine(history=[])
51     results = engine.evaluate(txn, profile)
52     rule_ids = [r.rule_id for r in results]
53     assert "R001" in rule_ids
54
55
56 def test_daily_limit_exceeded_blocks():
57     profile = make_profile(daily_limit=1000000)
58     history = [make_txn(amount=800000, timestamp=datetime(2026, 8, 23, 9, 0, 0))]
59     txn = make_txn(amount=300000, timestamp=datetime(2026, 8, 23, 20, 0, 0))
60     engine = RuleEngine(history=history)
```

```

61     results = engine.evaluate(txn, profile)
62     r002 = next(r for r in results if r.rule_id == "R002")
63     assert r002.action == RuleAction.BLOCK
64
65
66 def test_disallowed_country():
67     profile = make_profile(allowed_countries=["KR"])
68     txn = make_txn(country="US")
69     engine = RuleEngine(history=[])
70     results = engine.evaluate(txn, profile)
71     assert any(r.rule_id == "R007" for r in results)
72
73
74 def test_impossible_travel_speed_blocks():
75     profile = make_profile()
76     history = [
77         make_txn(
78             timestamp=datetime(2026, 8, 23, 14, 0, 0),
79             latitude=37.5665, # 서울
80             longitude=126.9780,
81         )
82     ]
83     # 5분 뒤 뉴욕에서 결제 -> 물리적으로 불가능한 이동
84     txn = make_txn(
85         timestamp=datetime(2026, 8, 23, 14, 5, 0),
86         latitude=40.7128, # 뉴욕
87         longitude=-74.0060,
88     )
89     engine = RuleEngine(history=history)
90     results = engine.evaluate(txn, profile)
91     assert any(r.rule_id == "R004" for r in results)
92
93
94 def test_rapid_repeat_payment_same_merchant():
95     profile = make_profile()
96     base = datetime(2026, 8, 23, 14, 0, 0)
97     history = [
98         make_txn(timestamp=base, merchant_id="m_1"),
99         make_txn(timestamp=base.replace(minute=1), merchant_id="m_1"),
100     ]
101     txn = make_txn(timestamp=base.replace(minute=2), merchant_id="m_1")
102     engine = RuleEngine(history=history)
103     results = engine.evaluate(txn, profile)
104     assert any(r.rule_id == "R006" for r in results)
105

```

스키마 · 배포

nfinity/sql/init.sql

164줄

```
1  -- SideGig AI - 초기 DB 스키마 (해커톤 MVP 버전)
2  -- 이 파일은 docker-compose로 PostgreSQL 컨테이너가 처음 돌 때 자동 실행되거나,
3  -- (9/2 추가) app/startup_seed.py가 앱 시작 시 직접 실행합니다 - 관리형 Postgres에
4  -- 배포해도 스키마가 자동으로 만들어지도록 하기 위함입니다.
5
6  -- (참고) gen_random_uuid()는 PostgreSQL 13부터 core에 내장돼 있어서 별도 확장 설치가
7  -- 필요 없습니다. 대부분의 관리형 Postgres(Railway/Render/Neon/Supabase 등)는 이미
8  -- PG14 이상이라 그대로 동작합니다 - 혹시 이보다 오래된 Postgres에 연결해서 아래 테이블
9  -- 생성이 실패한다면 가장 먼저 이 함수 지원 여부를 확인하세요.
10
11 -- 1. 사용자 (퍼스ونا별 유저)
12 CREATE TABLE IF NOT EXISTS users (
13     user_id          VARCHAR(50) PRIMARY KEY,
14     name             VARCHAR(100) NOT NULL,
15     persona          VARCHAR(100),          -- 예: '김지훈-프리랜서', '김민철-생계형투잡러'
16     created_at       TIMESTAMP NOT NULL DEFAULT now()
17 );
18
19 -- 2. 사용자 리스크 프로파일
20 -- 실제로는 UserProfileBuilder가 과거 거래를 분석해 채워주는 값이지만,
21 -- Day1에서는 기본값으로 미리 만들어두고 나중에 배치job으로 갱신합니다.
22 CREATE TABLE IF NOT EXISTS user_risk_profiles (
23     user_id          VARCHAR(50) PRIMARY KEY REFERENCES users(user_id),
24     avg_transaction_amount NUMERIC(14, 2) DEFAULT 0,
25     std_transaction_amount NUMERIC(14, 2) DEFAULT 0,
26     single_transaction_limit NUMERIC(14, 2) DEFAULT 500000, -- 단일 거래 한도(기본값)
27     daily_limit         NUMERIC(14, 2) DEFAULT 1000000, -- 일일 누적 한도(기본값)
28     avg_daily_transactions NUMERIC(10, 2) DEFAULT 0,
29     allowed_countries    TEXT[] DEFAULT ARRAY['KR'], -- 허용 국가 코드 목록
30     updated_at          TIMESTAMP NOT NULL DEFAULT now()
31 );
32
33 -- 3. 거래 내역 (B가 만드는 mock_transactions.csv가 최종적으로 여기로 들어옵니다)
34 CREATE TABLE IF NOT EXISTS transactions (
35     transaction_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
36     user_id           VARCHAR(50) NOT NULL REFERENCES users(user_id),
37     amount            NUMERIC(14, 2) NOT NULL,
38     merchant_id       VARCHAR(50),
39     merchant_name     VARCHAR(200),
40     mcc_code          VARCHAR(10),
41     category          VARCHAR(50),          -- C(하은)의 카테고리 분류기가 채워주는 필드
42     timestamp         TIMESTAMP NOT NULL,
43     latitude          DOUBLE PRECISION,
44     longitude         DOUBLE PRECISION,
45     country           VARCHAR(5) DEFAULT 'KR',
46     device_id         VARCHAR(100),
47     created_at        TIMESTAMP NOT NULL DEFAULT now()
48 );
49
50 CREATE INDEX IF NOT EXISTS idx_transactions_user_time ON transactions (user_id, timestamp);
51
52 -- 4. 리스크 이벤트 (룰 엔진 / AI 모델이 이상 거래를 잡아내면 여기에 기록)
53 CREATE TABLE IF NOT EXISTS risk_events (
54     event_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
55     transaction_id     UUID REFERENCES transactions(transaction_id),
56     user_id           VARCHAR(50) NOT NULL REFERENCES users(user_id),
57     rule_id            VARCHAR(20),          -- 예: 'R001' ~ 'R007'
58     severity          VARCHAR(20),          -- LOW / MEDIUM / HIGH / CRITICAL
59     description        TEXT,
60     action             VARCHAR(30),          -- ALLOW / FLAG_FOR_REVIEW / REQUIRE_CONFIRMATION / BLOCK
```

```

61     score            INTEGER,                -- 0~100 종합 리스크 점수
62     resolved         BOOLEAN DEFAULT FALSE,
63     created_at        TIMESTAMP NOT NULL DEFAULT now()
64 );
65
66 -- 5. 예산 설정 (예산통제 기능용)
67 -- 8/26 백엔드 구축 데이에 UNIQUE 제약을 추가했습니다: 한 유저가 같은 카테고리에
68 -- 같은 기간(monthly 등) 예산을 두 번 설정하면 새 행이 또 생기는 게 아니라
69 -- 기존 값을 덮어써야 하기 때문입니다 (app/routers/budgets.py의 ON CONFLICT가
70 -- 이 제약을 필요로 합니다). 이미 테이블이 있는 사람도 이 파일을 다시 실행하면
71 -- 안전하게 제약만 추가됩니다(IF NOT EXISTS 없이 그냥 실행하면 이미 있다는 예러가
72 -- 나니, docker compose down -v 후 새로 올리거나, 아래 ALTER문만 따로 실행하세요).
73 CREATE TABLE IF NOT EXISTS budgets (
74     budget_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
75     user_id           VARCHAR(50) NOT NULL REFERENCES users(user_id),
76     category          VARCHAR(50) NOT NULL,
77     period            VARCHAR(20) NOT NULL DEFAULT 'monthly', -- weekly/monthly/quarterly
78     limit_amount      NUMERIC(14, 2) NOT NULL,
79     alert_thresholds  NUMERIC(4, 2)[] DEFAULT ARRAY[0.5, 0.8, 0.9, 1.0, 1.2],
80     rollover_enabled  BOOLEAN DEFAULT FALSE,
81     created_at        TIMESTAMP NOT NULL DEFAULT now(),
82     updated_at        TIMESTAMP NOT NULL DEFAULT now(),
83     UNIQUE(user_id, category, period)
84 );
85
86 -- 이미 budgets 테이블이 만들어져 있는 상태에서 이 스크립트를 다시 돌릴 사람들을 위한
87 -- 보강용 ALTER (제약이 이미 있으면 조용히 건너뜁니다).
88 DO $$
89 BEGIN
90     IF NOT EXISTS (
91         SELECT 1 FROM pg_constraint WHERE conname = 'budgets_user_category_period_key'
92     ) THEN
93         ALTER TABLE budgets ADD CONSTRAINT budgets_user_category_period_key
94             UNIQUE (user_id, category, period);
95     END IF;
96 END $$;
97
98 -- 6. 카테고리 매핑 사전 (C가 만드는 가맹점명 키워드 -> 카테고리 매핑을 여기 저장)
99 CREATE TABLE IF NOT EXISTS category_mapping (
100     keyword           VARCHAR(100) PRIMARY KEY,
101     category          VARCHAR(50) NOT NULL
102 );
103
104 -- 7. 수입 플랫폼 연결 (9/2 추가)
105 -- 실제 오픈뱅킹/마이데이터 연동은 이번 MVP 범위 밖이라, "계좌를 연결하면 그 플랫폼의
106 -- 수입이 보인다"는 흐름을 더미 데이터로 재연합니다. connected가 false인 동안은
107 -- 프론트가 income_events를 보여주지 않다가, POST /api/v1/income/sources/{id}/connect를
108 -- 호출하면(=연결 버튼) true로 바뀌면서 이미 DB에 있던 실제(mock) 데이터가 그제서야
109 -- 화면에 드러나는 방식입니다 - 진짜 계좌 연동과 사용자 체감이 최대한 비슷하도록.
110 CREATE TABLE IF NOT EXISTS income_sources (
111     source_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
112     user_id           VARCHAR(50) NOT NULL REFERENCES users(user_id),
113     platform_name      VARCHAR(100) NOT NULL, -- '배달의민족', '크몽', '유튜브' 등
114     platform_type      VARCHAR(30) NOT NULL, -- '배달' / '프리랜서' / '콘텐츠' / '커머스' / '본업'
115     icon_emoji         VARCHAR(10) NOT NULL DEFAULT '👉',
116     connected         BOOLEAN NOT NULL DEFAULT FALSE,
117     connected_at       TIMESTAMP,
118     created_at        TIMESTAMP NOT NULL DEFAULT now(),
119     UNIQUE (user_id, platform_name)
120 );
121
122 -- 8. 플랫폼별 수입/정산 내역 (income_sources 하나당 여러 건)
123 CREATE TABLE IF NOT EXISTS income_events (
124     income_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
125     source_id         UUID NOT NULL REFERENCES income_sources(source_id),
126     user_id           VARCHAR(50) NOT NULL REFERENCES users(user_id),
127     amount            NUMERIC(14, 2) NOT NULL,
128     memo              VARCHAR(200),

```

```

129     status          VARCHAR(20) NOT NULL DEFAULT '정산완료', -- 정산완료 / 정산예정
130     settled_at      TIMESTAMP NOT NULL,
131     created_at       TIMESTAMP NOT NULL DEFAULT now()
132 );
133
134 CREATE INDEX IF NOT EXISTS idx_income_events_user_time ON income_events (user_id, settled_at);
135 CREATE INDEX IF NOT EXISTS idx_income_events_source ON income_events (source_id);
136
137 -- 9. 사용자 설정 (9/2 세 번째 업데이트 - Risk Shield용 최소 안전잔액 온보딩)
138 --   기획서 7-1의 "온보딩: 최소 안전잔액 입력"을 반영합니다. 값이 없으면
139 --   app/cashflow.py의 DEFAULT_MIN_SAFETY_BALANCE가 대신 쓰입니다.
140 CREATE TABLE IF NOT EXISTS user_settings (
141     user_id          VARCHAR(50) PRIMARY KEY REFERENCES users(user_id),
142     min_safety_balance NUMERIC(14, 2),
143     current_balance   NUMERIC(14, 2),
144     updated_at        TIMESTAMP NOT NULL DEFAULT now()
145 );
146
147 -- 9/5: current_balance 추가. 이전에는 Risk Shield의 시작 잔액을
148 -- "연결된 플랫폼 수입 합계 - 전체 지출 합계"로 유도했는데, 수입은 (의도적으로) 연결된
149 -- 플랫폼만 집계하고 지출은 전부 집계하다 보니 잔액이 구조적으로 음수가 나왔습니다
150 -- (5명 중 3명이 마이너스). 오픈뱅킹이 없는 이 MVP에서 정직한 모델은 "사용자가 알려준
151 -- 현재 잔액"을 기준으로 잡고 거기서부터 앞으로를 시뮬레이션하는 것이라, 컬럼을 두고
152 -- 데모 페르소나는 시드로 채웁니다. 이미 테이블이 있는 환경을 위한 보강문입니다.
153 ALTER TABLE user_settings ADD COLUMN IF NOT EXISTS current_balance NUMERIC(14, 2);
154
155 -- 샘플 시드 데이터 (동작 확인용 - 나중에 지워도 됩니다)
156 INSERT INTO users (user_id, name, persona) VALUES
157     ('u_001', '김지훈', '프리랜서 마케터/데이터분석가'),
158     ('u_002', '김민철', '생계형 배달 투잡러')
159 ON CONFLICT (user_id) DO NOTHING;
160
161 INSERT INTO user_risk_profiles (user_id) VALUES
162     ('u_001'), ('u_002')
163 ON CONFLICT (user_id) DO NOTHING;
164

```

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 # 8/30 수정: data_pipeline(카테고리 분류/절세장부 분류/정기결제 감지 로직),
9 # models(학습된 Isolation Forest 모델 .pkl), frontend(데모 화면)가 실제로는
10 # app/ 코드에서 import/서빙되는데 이전엔 COPY가 안 돼 있어서, 이 이미지로
11 # 배포하면 바로 ImportError로 죽는 상태였습니다.
12 #
13 # 9/2 수정: data/.scripts/.sql은 "로컬 배치테스트/개발 전용"이라며 이미지에서 뺐었는데,
14 # 그 판단이 실제로는 배포 버그였습니다 - 관리형 Postgres(Railway/Render 등)에 새로
15 # 배포하면 스키마도 데모 데이터도 전혀 없는 빈 DB로 뜨는데, 그걸 채워주는 스크립트들이
16 # 전부 이 세 폴더 안에 있어서 이미지 안에서 실행할 방법이 없었습니다. app/startup_seed.py가
17 # 앱 시작 시 이 파일들을 이용해 스키마 생성 + 데모 데이터 적재를 자동으로 하므로, 용량이
18 # 작은(전부 합쳐 1MB 미만) 이 세 폴더는 이제 반드시 이미지에 포함해야 합니다.
19 COPY app ./app
20 COPY data_pipeline ./data_pipeline
21 COPY models ./models
22 COPY frontend ./frontend
23 COPY data ./data
24 COPY scripts ./scripts
25 COPY sql ./sql
26
27 # 9/2 세 번째 업데이트(보안 강화): 지금까지는 컨테이너 안에서 root로 앱이 돌고
28 # 있었습니다 - 만약 앱 코드에 원격 코드 실행(RCE)으로 이어지는 취약점이 있다면,
29 # root 권한 그대로 컨테이너 안의 모든 파일을 건드릴 수 있다는 뜻입니다. 이 앱은
30 # 런타임에 /app 아래 아무 파일도 쓰지 않으므로(모델 학습(app/anomaly_model.py의
31 # train())은 개발 중 수동으로만 돌리는 오프라인 스크립트라 컨테이너 안에서 실행되지
32 # 않음, DB/Redis 쓰기는 전부 외부 서비스로 나감) 전용 비루트 유저로 내려도 기능에
33 # 영향이 없습니다.
34 RUN useradd --create-home --uid 1000 appuser && chown -R appuser:appuser /app
35 USER appuser
36
37 EXPOSE 8000
38
39 # 운영 배포용 이미지라 --reload는 뺐습니다(파일 감시 오버헤드 + 배포 환경에서 의미 없음).
40 # 로컬 개발 중 코드 변경을 바로 반영하고 싶으면 docker-compose.yml의 api 서비스에서
41 # volumes 바인드 마운트 + command 오버라이드로 --reload를 켜서 쓰세요.
42 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
43

```

```

1  services:
2    db:
3      image: postgres:15
4      container_name: sidegig_db
5      restart: always
6      environment:
7        POSTGRES_USER: sidegig
8        # 8/30: 하드코딩 대신 .env의 POSTGRES_PASSWORD를 씁니다 (없으면 기존 기본값 유지).
9        # 바꾸려면 .env의 DATABASE_URL 비밀번호도 반드시 같이 바꾸세요.
10       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-sidegig_pw}
11       POSTGRES_DB: sidegig
12       # 9/2 보안 점검: 예전엔 "5432:5432"로 모든 인터페이스에 열려 있었습니다 - 이 파일로
13       # 그대로 공인 IP가 있는 서버에 배포하면(예: VPS에 docker-compose로 직접 배포) 누구나
14       # 인터넷에서 이 포트로 접속을 시도할 수 있었습니다(기본 비밀번호를 안 바꿨다면 더 위험).
15       # api 컨테이너는 이 host 포트 매핑과 무관하게 도커 내부망(db:5432)으로 항상 접속되므로,
16       # 127.0.0.1로 묶어도 앱 동작에는 영향이 없고, 로컬 개발 중 `psql -h localhost`로
17       # 붙는 것도 그대로 됩니다 - 외부에서만 막습니다.
18       ports:
19         - "127.0.0.1:5432:5432"
20       volumes:
21         - pgdata:/var/lib/postgresql/data
22         - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql
23       healthcheck:
24         test: ["CMD-SHELL", "pg_isready -U sidegig"]
25         interval: 5s
26         timeout: 5s
27         retries: 5
28
29     redis:
30       image: redis:7
31       container_name: sidegig_redis
32       restart: always
33       # 9/2 보안 점검: Redis는 기본적으로 비밀번호 인증이 없어서, 예전처럼 "6379:6379"로
34       # 열어두면 공인 IP가 있는 서버에서는 인증 없이 외부에서 누구나 읽고 쓸 수 있었습니다.
35       # db와 동일한 이유로 127.0.0.1에만 묶습니다 - api 컨테이너는 도커 내부망(redis:6379)으로
36       # 접속하므로 영향 없습니다.
37       #
38       # 9/2 세 번째 업데이트: 포트를 막아도 "도커 내부망에 컨테이너 하나만 뚫리면 Redis는
39       # 무방비"라는 문제는 남아있어서, --requirepass로 인증까지 추가합니다. .env의
40       # REDIS_PASSWORD 값을 그대로 씁니다 - REDIS_URL(redis://:<비밀번호>@redis:6379/0)의
41       # 비밀번호와 반드시 일치해야 합니다. redis-py(app/redis_client.py, app/database.py)는
42       # URL에 비밀번호가 있으면 자동으로 AUTH를 보내므로 앱 코드는 수정할 필요가 없습니다
43       # (직접 redis://:test_pw@localhost:PORT/0 형태로 연결해서 확인 완료).
44       command: ["redis-server", "--requirepass", "${REDIS_PASSWORD:-sidegig_redis_pw}"]
45       ports:
46         - "127.0.0.1:6379:6379"
47       healthcheck:
48         test: ["CMD-SHELL", "redis-cli -a ${REDIS_PASSWORD:-sidegig_redis_pw} ping"]
49         interval: 5s
50         timeout: 5s
51         retries: 5
52
53     api:
54       build: .
55       container_name: sidegig_api
56       restart: always
57       env_file:
58         - .env
59       ports:
60         - "8000:8000"
61       depends_on:
62         db:
63           condition: service_healthy

```

```

64     redis:
65         condition: service_healthy
66     volumes:
67         # 로컬 개발 중 코드를 고치면 바로 반영되도록 바인드 마운트 (배포용 이미지 자체는
68         # Dockerfile에서 COPY로 고정됩니다 - 이 volumes는 로컬 docker-compose에서만 씁니다)
69         - ./app:/app/app
70         - ./data_pipeline:/app/data_pipeline
71         - ./frontend:/app/frontend
72         # Dockerfile 기본 CMD(--reload 없음)를 로컬 개발용으로 덮어씁니다.
73         command: ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
74
75     volumes:
76     pgdata:
77

```

nfinity/requirements.txt

27줄

```

1  # 9/2 세 번째 업데이트(보안 강화): pip-audit로 스캔한 결과 python-dotenv/pillow(전이
2  # 의존성)/starlette(전이 의존성, fastapi가 끌고 옴)에서 알려진 취약점 35건이 나와서
3  # 전부 패치된 버전으로 올렸습니다. fastapi==0.121.0 dry-run 확인 결과 pydantic 2.9.2와
4  # 그대로 호환되어(핀 안 올려도 됨) 0.141.1까지 갈 필요는 없었지만, starlette가 0.49.3까지만
5  # 따라오고 일부 취약점(PYSEC-2026-248/249/2280/2281)은 starlette>=1.1.0~1.3.1을 요구해서
6  # fastapi도 0.141.1로, starlette도 1.6.0으로 직접 명시했습니다(둘 다 fastapi의
7  # `starlette>=0.46.0` 제약과 충돌 없음). google-genai는 0.3.0에서 pillow<12.0.0을
8  # 강제해서 pillow==12.3.0 핀과 충돌했는데, google-genai>=1.0.0부터는 pillow가 아예
9  # 필수 의존성에서 빠져서(선택 extra로만 남음) 1.28.0으로 올려 해결했습니다.
10 # 프리 DB로부터의 전체 자동 시딩 + 전 API 엔드포인트(gig-score/income/budgets/connect
11 # 플로우 포함)를 이 조합으로 재검증 완료. pip-audit 재실행 결과 이 5개 패키지 관련
12 # 취약점은 0건(남은 건 pip/setuptools 자체의 빌드 툰체인 취약점으로, requirements.txt에
13 # 없고 배포되는 앱 코드와 무관).
14 fastapi==0.141.1
15 starlette==1.6.0
16 uvicorn[standard]==0.30.6
17 sqlalchemy==2.0.35
18 psycpg2-binary==2.9.9
19 redis==5.0.8
20 pydantic==2.9.2
21 python-dotenv==1.2.3
22 scikit-learn==1.8.0
23 pandas==2.2.3
24 numpy==1.26.4
25 google-genai==1.28.0
26 pillow==12.3.0
27

```

```

1 # Render Blueprint – 완전 무료(free) 플랜만 사용합니다.
2 # 사용법: Render 대시보드 → New + → Blueprint → 이 GitHub 저장소(graymango7/Nfinity) 선택
3 # → Render가 이 파일을 자동으로 읽어서 web/DB/Redis 3개를 한 번에 만듭니다.
4 # 이후로는 이 저장소의 main 브랜치에 git push할 때마다 자동으로 재배포됩니다(autoDeploy).
5
6 services:
7   - name: nfinity-ai
8     type: web
9     runtime: docker
10    plan: free
11    dockerfilePath: ./nfinity/Dockerfile
12    dockerContext: ./nfinity
13    healthCheckPath: /health
14    autoDeploy: true
15    envVars:
16      - key: DATABASE_URL
17        fromDatabase:
18          name: nfinity-db
19          property: connectionString
20      - key: REDIS_URL
21        fromService:
22          name: nfinity-redis
23          type: keyvalue
24          property: connectionString
25      # 대회 운영진이 공식적으로 데모(가상) 데이터 시연을 허용한 구조라, README의
26      # 설계 판단(9-10번)을 그대로 따라 기본값은 바꿨습니다. 참고고 싶으면 Render
27      # 대시보드에서 이 값과 frontend/index.html의 API_KEY 상수를 똑같이 채우세요.
28      - key: API_ACCESS_KEY
29        value: ""
30      # 없으면 앱이 자동으로 Mock 모드(규칙 기반 폴백)로 동작합니다 – 필수 아님.
31      - key: GEMINI_API_KEY
32        sync: false
33
34   - name: nfinity-redis
35     type: keyvalue
36     plan: free
37     ipAllowList: []
38     maxmemoryPolicy: allkeys-lru
39
40 databases:
41   - name: nfinity-db
42     plan: free
43

```

1288줄

```

1 <!--
2 SideGig - 데스크톱 웹 (9/5 재작성, 9/5 2차: 2화면 구조로 분리)
3
4 화면 구성
5 1) 사례 선택 화면 - 3명의 시연용 인물 중 하나를 고릅니다. 각 카드에 현재 현금흐름
6 상태(안전/주의/위험)를 같이 띄워서, 고르기 전에 무엇을 보게 될지 알 수 있게 합니다.
7 2) 대시보드 - 카테고리 4개(현금흐름 / 수입 / 지출 관리 / 거래 보안)로 구획을 나눕니다.
8 이전 버전은 카드 7장을 한 화면에 평평하게 늘어놔서 무엇보다 봐야 할지 알기 어려웠습니다.
9
10 구현 원칙
11 - 외부 라이브러리 없음. 차트는 API 응답으로 직접 그리는 인라인 SVG입니다
12 (app/security_headers.py의 CSP와 충돌하지 않고, 네트워크 실패로 깨질 여지도 없음).
13 - 카드마다 독립적으로 로드/실패합니다. 한 API가 죽어도 나머지 화면은 그대로 뜹니다.
14 - 모든 동적 텍스트는 escapeHtml을 거칩니다(값이 LLM 출력이 될 수 있는 필드가 있어서).
15 -->
16 <title>SideGig - N잡러 현금흐름 관리</title>
17 <link rel="stylesheet"
18 href="https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/variable/pretendardvariable-dynamic-s
ubset.min.css">
19 <style>
20 :root {
21 --bg: #f2f4f6; --card: #fff; --border: #e5e8eb; --border-strong: #d1d6db;
22 --text: #191f28; --text-muted: #4e5968; --text-faint: #8b95a1;
23 --accent: #3182f6; --accent-soft: #e8f3ff;
24 --safe: #00a661; --warn: #ff8a00; --danger: #f04452;
25 --safe-soft: #e7f8f0; --warn-soft: #fff5e6; --danger-soft: #ffe5f0;
26 }
27 * { box-sizing: border-box; }
28 body {
29 margin: 0; background: var(--bg); color: var(--text); font-size: 15px; line-height: 1.5;
30 font-family: "Pretendard Variable", "Pretendard", -apple-system, BlinkMacSystemFont,
31 "Malgun Gothic", "Apple SD Gothic Neo", sans-serif;
32 -webkit-font-smoothing: antialiased; letter-spacing: -0.01em;
33 }
34 .num { font-variant-numeric: tabular-nums; }
35 [hidden] { display: none !important; }
36
37 /* ----- 공통 바 ----- */
38 .topbar { background: rgba(255,255,255,.88); backdrop-filter: saturate(180%) blur(12px);
39 position: sticky; top: 0; z-index: 50; }
40 .topbar-inner { max-width: 1320px; margin: 0 auto; padding: 0 28px; height: 66px;
41 display: flex; align-items: center; gap: 14px; }
42 .brand { font-size: 21px; font-weight: 700; letter-spacing: -0.04em; }
43 .brand span { color: var(--accent); }
44 .spacer { flex: 1; }
45 .ghost-btn { background: #f2f4f6; border: 0; border-radius: 10px;
46 padding: 10px 15px; cursor: pointer; font-family: inherit; font-size: 14px;
47 font-weight: 600; color: var(--text-muted); }
48 .ghost-btn:hover { background: #e8ebee; color: var(--text); }
49
50 /* ----- 1) 선택 화면 ----- */
51 .landing { max-width: 1060px; margin: 0 auto; padding: 64px 28px 80px; }
52 .landing h1 { font-size: 40px; font-weight: 700; letter-spacing: -0.04em; margin: 0 0 16px; line-height: 1.35; }
53 .landing h1 b { color: var(--accent); font-weight: 700; }
54 .landing .lede { font-size: 17px; color: var(--text-muted); max-width: 660px; margin: 0; line-height: 1.6; }
55 .landing .pick { font-size: 15px; font-weight: 600; color: var(--text); margin: 44px 0 16px; }
56 .persona-grid { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 16px; }
57 @media (max-width: 900px) { .persona-grid { grid-template-columns: minmax(0, 1fr); } }
58 .persona-card { background: #fff; border: 0; border-radius: 20px;
59 padding: 26px 24px; text-align: left; cursor: pointer; font-family: inherit;

```

```

60     box-shadow: 0 1px 2px rgba(0,0,0,.03); transition: box-shadow .15s, transform .15s; }
61 .persona-card:hover { box-shadow: 0 8px 24px rgba(0,0,0,.09); transform: translateY(-3px); }
62 .persona-card .emoji { font-size: 34px; display: block; margin-bottom: 14px; }
63 .persona-card .nm { font-size: 19px; font-weight: 700; letter-spacing: -0.03em; }
64 .persona-card .job { font-size: 14px; color: var(--text-muted); margin-top: 3px; }
65 .persona-card .tg { font-size: 14px; color: var(--text-muted); margin: 14px 0; min-height: 63px; line-height: 1.6;
}
66 .persona-card .go { font-size: 14.5px; color: var(--accent); font-weight: 700; margin-top: 10px; }
67
68 /* ----- 2) 데스크톱 ----- */
69 .page { max-width: 1320px; margin: 0 auto; padding: 24px 28px 60px; }
70
71 /* 1단 요약 */
72 .hero { background: #fff; border-radius: 20px; padding: 30px 32px; box-shadow: 0 1px 2px rgba(0,0,0,.03); }
73 .hero-who { font-size: 14px; color: var(--text-muted); font-weight: 600; }
74 .hero-line { font-size: 30px; font-weight: 700; letter-spacing: -0.04em; line-height: 1.35; margin: 8px 0 22px; }
75 .hero-line b { color: var(--accent); }
76 .hero-line b.risk { color: var(--danger); }
77 .hero-stats { display: flex; gap: 40px; flex-wrap: wrap; }
78 .hero-stats .st.k { font-size: 13px; color: var(--text-muted); }
79 .hero-stats .st.v { font-size: 22px; font-weight: 700; letter-spacing: -0.03em; margin-top: 3px; }
80
81 .brief { margin-top: 22px; padding-top: 20px; border-top: 1px solid var(--border);
82     display: flex; gap: 12px; align-items: flex-start; }
83 .brief .bi { font-size: 13px; font-weight: 700; color: var(--accent); background: var(--accent-soft);
84     border-radius: 8px; padding: 4px 9px; flex: none; }
85 .brief .bt { font-size: 15px; line-height: 1.65; color: var(--text-muted); }
86 .brief .ba { display: inline-block; margin-top: 7px; font-size: 14px; font-weight: 700; color: var(--text); }
87
88 /* 2단 할 일 */
89 .todos { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 12px; margin-top: 12px; }
90 @media (max-width: 1080px) { .todos { grid-template-columns: minmax(0, 1fr); } }
91 .todo { background: #fff; border-radius: 14px; padding: 18px 20px; display: flex; gap: 13px;
92     align-items: flex-start; box-shadow: 0 1px 2px rgba(0,0,0,.03); }
93 .todo .ic { width: 34px; height: 34px; border-radius: 11px; display: grid; place-items: center;
94     font-size: 17px; flex: none; }
95 .todo .tt { font-size: 14.5px; font-weight: 700; letter-spacing: -0.02em; }
96 .todo .ds { font-size: 13px; color: var(--text-muted); margin-top: 3px; line-height: 1.5; }
97 .todo .act { margin-top: 10px; }
98
99 .valueprops { display: grid; grid-template-columns: repeat(3, minmax(0,1fr)); gap: 14px; margin-top: 34px; }
100 @media (max-width: 900px) { .valueprops { grid-template-columns: minmax(0,1fr); } }
101 .vp { background: #fff; border-radius: 16px; padding: 20px; box-shadow: 0 1px 2px rgba(0,0,0,.03); }
102 .vp .vpi { font-size: 22px; display: block; margin-bottom: 10px; }
103 .vp b { display: block; font-size: 15px; letter-spacing: -0.02em; margin-bottom: 6px; }
104 .vp span { font-size: 13.5px; color: var(--text-muted); line-height: 1.6; }
105
106 .evidence { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 26px; }
107 .evidence span { background: #fff; border-radius: 999px; padding: 9px 16px; font-size: 13.5px;
108     color: var(--text-muted); box-shadow: 0 1px 2px rgba(0,0,0,.03); }
109 .evidence b { color: var(--text); font-weight: 700; }
110
111 .panehint { font-size: 13.5px; color: var(--text-muted); margin: -2px 0 14px; }
112
113 /* 3단 탭 */
114 .maintabs { display: flex; gap: 4px; margin: 32px 0 16px; border-bottom: 1px solid var(--border); }
115 .maintab { border: 0; background: none; font-family: inherit; font-size: 16px; font-weight: 700;
116     letter-spacing: -0.02em; color: var(--text-faint); padding: 10px 14px 14px; cursor: pointer;
117     border-bottom: 2px solid transparent; margin-bottom: -1px; }
118 .maintab[aria-selected="true"] { color: var(--text); border-bottom-color: var(--text); }
119 .maintab:hover { color: var(--text-muted); }
120
121
122 .grid { display: grid; gap: 16px; }
123 .row-hero { grid-template-columns: minmax(0, 2.2fr) minmax(0, 1fr); }
124 .row-2 { grid-template-columns: repeat(2, minmax(0, 1fr)); }
125 @media (max-width: 1080px) { .row-hero, .row-2 { grid-template-columns: minmax(0, 1fr); } }
126

```

```

127 .card { background: var(--card); border: 0; border-radius: 18px;
128   padding: 24px; min-width: 0; box-shadow: 0 1px 2px rgba(0,0,0,.03); }
129 .card-title { font-size: 16px; font-weight: 700; letter-spacing: -0.02em; margin-bottom: 14px; }
130
131 .metric { font-size: 32px; font-weight: 700; letter-spacing: -0.035em; }
132 .metric-sm { font-size: 22px; font-weight: 700; letter-spacing: -0.03em; }
133 .label { font-size: 13px; color: var(--text-muted); }
134
135 .pill { display: inline-flex; align-items: center; gap: 5px; font-size: 12.5px;
136   font-weight: 700; padding: 4px 10px; border-radius: 999px; }
137 .pill.safe { color: var(--safe); background: var(--safe-soft); }
138 .pill.warn { color: var(--warn); background: var(--warn-soft); }
139 .pill.danger { color: var(--danger); background: var(--danger-soft); }
140 .pill.neutral { color: var(--text-muted); background: #f2f4f7; }
141 .dot { width: 7px; height: 7px; border-radius: 50%; background: currentColor; }
142
143 .tabs { display: inline-flex; background: #f2f4f7; border-radius: 8px; padding: 3px; gap: 2px; }
144 .tab { border: 0; background: none; font-family: inherit; font-size: 12.5px;
145   color: var(--text-muted); padding: 5px 12px; border-radius: 6px; cursor: pointer; }
146 .tab[aria-selected="true"] { background: #fff; color: var(--text); font-weight: 600;
147   box-shadow: 0 1px 2px rgba(16,24,40,0.08); }
148
149 svg.chart { width: 100%; height: auto; display: block; }
150 .legend { display: flex; gap: 16px; flex-wrap: wrap; margin-top: 10px; font-size: 12px; color: var(--text-muted);
151 }
152 .legend i { display: inline-block; width: 12px; height: 2px; vertical-align: middle; margin-right: 5px; }
153
154 .list { display: flex; flex-direction: column; }
155 .list-row { display: flex; align-items: center; gap: 10px; padding: 9px 0; border-top: 1px solid #f2f4f6; }
156 .list-row:first-child { border-top: 0; }
157 .list-row.grow { flex: 1; min-width: 0; }
158
159 .truncate { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
160
161 .bar { height: 6px; border-radius: 999px; background: #f2f4f6; overflow: hidden; }
162 .bar > span { display: block; height: 100%; border-radius: 999px; background: var(--accent); }
163 .bar.over > span { background: var(--danger); }
164 .bar.near > span { background: var(--warn); }
165
166 .gauge-track { height: 8px; border-radius: 999px; background: #f2f4f6; overflow: hidden; margin: 10px 0 6px; }
167 .gauge-track > span { display: block; height: 100%; background: linear-gradient(90deg, #3182f6, #64a8ff); }
168
169 .reasons { margin: 12px 0 0; padding: 0; list-style: none; display: flex; flex-direction: column; gap: 7px; }
170 .reasons li { display: flex; gap: 8px; font-size: 12.5px; color: var(--text-muted); }
171 .reasons li:before { content: ""; width: 5px; height: 5px; border-radius: 50%;
172   background: var(--text-faint); margin-top: 7px; flex: none; }
173
174 .kv { display: flex; justify-content: space-between; align-items: baseline; padding: 7px 0; }
175 .kv + .kv { border-top: 1px solid #f2f4f6; }
176
177 .muted-box { background: #f9fafb; border: 1px solid var(--border); border-radius: 8px;
178   padding: 11px 13px; font-size: 12px; color: var(--text-muted); line-height: 1.55; }
179 /* 입력 요소 */
180
181 .field { display: flex; flex-direction: column; gap: 5px; min-width: 0; }
182 .field > label { font-size: 12px; color: var(--text-muted); }
183
184 input[type="text"], input[type="number"], select {
185   font-family: inherit; font-size: 15px; font-weight: 500; color: var(--text);
186   background: #f2f4f6; border: 1px solid transparent; border-radius: 12px;
187   padding: 12px 14px; width: 100%;
188 }
189
190 input:focus, select:focus { outline: none; background: #fff; border-color: var(--accent); }
191
192 .btn { font-family: inherit; font-size: 15px; font-weight: 700; border-radius: 12px;
193   padding: 13px 18px; cursor: pointer; border: 0; letter-spacing: -0.02em; }
194
195 .btn-primary { background: var(--accent); color: #fff; }
196 .btn-primary:hover { background: #2272e6; }
197 .btn-primary:disabled { background: #b6d4fb; cursor: default; }
198
199 .btn-sm { padding: 9px 14px; font-size: 14px; border-radius: 10px; }
200
201 .btn-outline { background: #f2f4f6; color: var(--text); }
202 .btn-outline:hover { background: #e8ebee; }
203
204 .chip { font-family: inherit; font-size: 13.5px; font-weight: 600; background: #f2f4f6;
205   border: 0; color: var(--text-muted); border-radius: 999px; padding: 9px 14px; cursor: pointer; }

```

```

194 .chip:hover { background: var(--accent-soft); color: var(--accent); }
195 /* 놀러보는 시나리오 버튼 - 입력 없이 클릭만으로 판정을 볼 수 있게 하는 자리 */
196 .scenario { display: flex; align-items: center; gap: 13px; width: 100%; text-align: left;
197     font-family: inherit; background: #f9fafb; border: 1px solid var(--border); border-radius: 14px;
198     padding: 14px 16px; cursor: pointer; margin-bottom: 8px; }
199 .scenario:hover { background: var(--accent-soft); border-color: var(--accent); }
200 .scenario:disabled { opacity: .6; cursor: default; }
201 .scenario .ic { font-size: 20px; flex: none; }
202 .scenario .t { display: block; font-size: 14.5px; font-weight: 700; letter-spacing: -0.02em; color: var(--text); }
203 .scenario .d { display: block; font-size: 13px; color: var(--text-muted); margin-top: 2px; }
204 .scenario .go { margin-left: auto; font-size: 13px; font-weight: 700; color: var(--accent); white-space: nowrap; }
205 details summary::-webkit-details-marker { display: none; }
206 details summary::marker { content: ""; }
207 details summary::before { content: "+ "; }
208 details[open] summary::before { content: "- "; }
209
210 .toast { position: fixed; right: 20px; bottom: 20px; z-index: 100; background: #101828; color: #fff;
211     font-size: 13px; padding: 12px 16px; border-radius: 9px; box-shadow: 0 10px 30px rgba(16,24,40,.24);
212     max-width: 380px; line-height: 1.5; }
213
214 .skeleton { color: var(--text-faint); font-size: 13px; padding: 8px 0; }
215 .err { color: var(--danger); font-size: 12.5px; background: var(--danger-soft);
216     border: 1px solid #fecdc4; border-radius: 7px; padding: 9px 11px; }
217 footer.page-foot { max-width: 1320px; margin: 0 auto; padding: 0 28px 40px;
218     font-size: 12px; color: var(--text-faint); line-height: 1.6; }
219 </style>
220
221 <div class="topbar">
222     <div class="topbar-inner">
223         <div class="brand">Side<span>Gig</span></div>
224         <div class="spacer"></div>
225         <button class="ghost-btn" id="backBtn" hidden>< 다른 사례 보기</button>
226     </div>
227 </div>
228
229 <!-- ===== 1) 사례 선택 ===== -->
230 <div id="viewLanding">
231     <div class="landing">
232         <h1>여러 플랫폼에서 들어오는 수입,<br><b>다음 정산까지 버틸 수 있는지</b> 알려드립니다.</h1>
233         <p class="lede">
234             배달·프리랜서·콘텐츠 등 흩어진 플랫폼의 정산 주기를 학습해 앞으로 45일의 잔액을 예측하고,
235             수입 변동성에 맞춰 세금·건강보험료를 미리 떼어둡니다. 이상거래 탐지와 업무경비 분류도 함께 제공합니다.
236         </p>
237         <div class="valueprops">
238             <div class="vp"><span class="vpi">📉</span><b>다음 정산까지 버틸 수 있는지</b>
239             <span>플랫폼별 정산 주기를 학습해 앞으로 45일의 잔액을 하루 단위로 예측합니다.</span></div>
240             <div class="vp"><span class="vpi">📊</span><b>세금·건보료가 얼마 나올지</b>
241             <span>5월 종합소득세와 건강보험료를 실제 산식으로 계산하고, 2,000만원 기준을 넘기 전에 알려줍니다.</span></div>
242             <div class="vp"><span class="vpi">🛡️</span><b>이상한 결제를 막아주는지</b>
243             <span>규칙 7종과 이상탐지 AI가 결제 순간 위험도를 매겨 승인·본인확인·차단을 정합니다.</span></div>
244         </div>
245         <div class="evidence">
246             <span><b>4,978건</b> 실거래 재현 검증</span>
247             <span><b>73.4%</b> 이상거래 탐지율</span>
248             <span><b>55.5%</b> 탐지 정밀도</span>
249             <span><b>2026년</b> 세율·건보료를 실적용</span>
250         </div>
251         <div style="margin-top:26px">
252             <button class="btn btn-outline" id="tryMineBtn">내 수입으로 직접 계산해보기 →</button>
253         </div>
254         <div class="pick">아래 세 사람 중 한 명을 골라, 그 사람의 실제 데이터로 위 기능들을 확인해보세요.</div>
255         <div class="persona-grid" id="personaGrid">
256             <div class="skeleton">볼러오는 중...</div>
257         </div>
258     </div>
259 </div>
260
261 <!-- ===== 1-2) 직접 입력 시뮬레이터 ===== -->

```

```

262 <div id="viewSim" hidden>
263   <div class="landing" style="max-width:900px">
264     <h1 style="font-size:30px">내 수입으로 계산해보기</h1>
265     <p class="lede">플랫폼마다 <b>며칠 주기로 얼마가</b> 들어오지만 넣으면, 앞으로 45일 잔고를
266       하루 단위로 계산합니다. 입력값은 저장하지 않습니다.</p>
267
268     <div class="card" style="margin-top:26px">
269       <div class="card-title">지금 상황</div>
270       <div style="display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:12px;margin-bottom:6px">
271         <div class="field"><label for="simBal">현재 잔액 (원)</label>
272           <input type="number" id="simBal" value="1000000" step="100000" min="0"></div>
273         <div class="field"><label for="simFloor">최소 안전잔액 (원)</label>
274           <input type="number" id="simFloor" value="300000" step="50000" min="0"></div>
275         <div class="field"><label for="simFixed">매달 고정비 (월세·보험 등)</label>
276           <input type="number" id="simFixed" value="900000" step="50000" min="0"></div>
277         <div class="field"><label for="simDay">고정비 빠지는 날</label>
278           <input type="number" id="simDay" value="25" min="1" max="28"></div>
279         <div class="field"><label for="simVar">하루 생활비 (원)</label>
280           <input type="number" id="simVar" value="35000" step="5000" min="0"></div>
281       </div>
282     </div>
283
284     <div class="card" style="margin-top:16px">
285       <div class="card-title">수입원 <span class="label" style="font-weight:400">· 최대 5개</span></div>
286       <div id="simSources"></div>
287       <button class="btn btn-outline btn-sm" id="simAddBtn" style="margin-top:10px">+ 수입원 추가</button>
288     </div>
289
290     <button class="btn btn-primary" id="simRunBtn" style="width:100%;margin-top:16px">45일 예측 계산하기</button>
291     <div id="simResult" style="margin-top:20px"></div>
292   </div>
293 </div>
294
295 <!-- ===== 2) 대시보드 ===== -->
296 <div id="viewDash" hidden>
297   <div class="page">
298     <!-- 1단: 지금 상태 한 문장 + 핵심 숫자. 스크롤 없이 이 서비스가 무엇을 말해주는지 끝냅니다. -->
299     <div class="hero">
300       <div class="hero-who" id="heroWho"></div>
301       <div class="hero-line" id="heroLine">불러오는 중...</div>
302       <div class="hero-stats" id="heroStats"></div>
303       <div id="brief"></div>
304     </div>
305
306     <!-- 2단: 지금 하면 좋은 일. 읽기만 하는 대시보드가 아니라 행동으로 이어지게 합니다. -->
307     <div class="todos" id="todos"></div>
308
309     <!-- 3단: 상세. 한 화면에 다 쌓지 않고 탭으로 나눕니다. -->
310     <nav class="maintabs" id="mainTabs" role="tablist"></nav>
311
312     <div id="pane-cash">
313       <div class="panehint">앞으로 들어올 정산과 나갈 고정비를 이어붙여, 잔액이 언제 위험해지는지 하루 단위로 계산합니다.
314     </div>
315     <div class="grid row-hero">
316       <div class="card">
317         <div style="display:flex;align-items:baseline;gap:10px;margin-bottom:6px">
318           <div class="card-title" style="margin:0">45일 현금흐름 예측</div>
319           <div class="right" style="margin-left:auto"><div class="tabs" id="scenarioTabs" role="tablist"></div></div>
320         </div>
321         <div id="shieldSummary"></div>
322         <div id="chartWrap"><div class="skeleton">불러오는 중...</div></div>
323         <div class="legend" id="chartLegend"></div>
324       </div>
325       <div class="card">
326         <div class="card-title">세금 · 건강보험료 전망</div>
327         <div id="parking"><div class="skeleton">불러오는 중...</div></div>
328       </div>

```

```

328     </div>
329 </div>
330
331 <div id="pane-income" hidden>
332     <div class="panehint">여러 플랫폼에 흩어진 정산 내역을 한 곳에 모읍니다. 연결한 플랫폼만 예측·세금 계산에 반영됩니
다.</div>
333     <div class="grid row-2">
334         <div class="card">
335             <div class="card-title">플랫폼별 수입 <span class="label" id="incomePeriod"></span></div>
336             <div id="income"><div class="skeleton">불러오는 중...</div></div>
337             <div id="sources"></div>
338         </div>
339         <div class="card">
340             <div class="card-title">Gig Score <span class="label" style="font-weight:400">· 관리 성실도 지표</span></di
v>
341             <div id="gig"><div class="skeleton">불러오는 중...</div></div>
342         </div>
343     </div>
344 </div>
345
346 <div id="pane-spend" hidden>
347     <div class="panehint">카테고리별로 이번 달 예산을 얼마나 썼는지, 그리고 어떤 결제가 업무 경비로 인정될 수 있는지 보
니다.</div>
348     <div class="grid row-2">
349         <div class="card">
350             <div class="card-title">이번 달 예산</div>
351             <div id="budget"><div class="skeleton">불러오는 중...</div></div>
352         </div>
353         <div class="card">
354             <div class="card-title">업무경비 자동 분류</div>
355             <div id="expense"><div class="skeleton">불러오는 중...</div></div>
356         </div>
357     </div>
358 </div>
359
360 <div id="pane-guide" hidden>
361     <div class="panehint">지금 계산된 상황에서 고려할 수 있는 선택지를, 왜 그런지와 함께 정리했습니다.</div>
362     <div class="card"><div id="guidance"><div class="skeleton">불러오는 중...</div></div></div>
363 </div>
364
365 <div id="pane-sec" hidden>
366     <div class="panehint">보이스피싱·영의도용처럼 본인이 하지 않은 결제를 걸러내는 기능입니다.</div>
367     <div class="grid row-2">
368         <div class="card">
369             <div class="card-title">이런 결제가 시도되면 어떻게 될까요?</div>
370             <div class="label" style="margin-bottom:16px;line-height:1.6">
371                 SideGig은 결제가 일어나는 순간 규칙 7종과 이상탐지 AI로 위험도를 매겨
372                 <b style="color:var(--text)">승인 · 본인확인 · 차단</b> 중 하나를 정합니다.
373                 아래 상황을 눌러 실제로 어떻게 판정하는지 확인해보세요.
374             </div>
375             <div id="assessPresets"></div>
376             <div id="assessResult" style="margin-top:16px"></div>
377         </div>
378         <details id="manualBox" style="margin-top:18px">
379             <summary style="cursor:pointer;font-size:14px;font-weight:600;color:var(--text-muted)">
380                 직접 입력해서 시험해보기
381             </summary>
382             <div style="margin-top:14px">
383                 <div style="display:grid;grid-template-columns:1.4fr 1fr;gap:10px;margin-bottom:10px">
384                     <div class="field"><label for="afMerchant">가맹점</label>
385                     <input type="text" id="afMerchant" value="스타벅스 강남점"></div>
386                     <div class="field"><label for="afAmount">금액 (원)</label>
387                     <input type="number" id="afAmount" value="5600" min="0" step="100"></div>
388                 </div>
389                 <div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:10px;margin-bottom:14px">
390                     <div class="field"><label for="afHour">결제 시각</label><select id="afHour"></select></div>
391                     <div class="field"><label for="afCountry">결제 국가</label>
392                     <select id="afCountry"><option value="KR">대한민국</option><option value="US">미국</option>

```

```

393         <option value="JP">일본</option><option value="CN">중국</option></select></div>
394     <div class="field"><label for="afDevice">결제 기기</label>
395         <select id="afDevice"><option value="known">평소 쓰던 기기</option>
396         <option value="new">처음 보는 기기</option></select></div>
397     </div>
398     <button class="btn btn-primary" id="assessBtn" style="width:100%">이 결제로 검사하기</button>
399 </div>
400 </details>
401 </div>
402 <div class="card">
403     <div style="display:flex;align-items:baseline;margin-bottom:14px">
404         <div class="card-title" style="margin:0">탐지 이력</div>
405         <span style="margin-left:auto" id="riskBadge"></span>
406     </div>
407     <div id="risk"><div class="skeleton">불러오는 중...</div></div>
408 </div>
409 </div>
410 </div>
411 </div>
412 </div>
413
414 <footer class="page-foot" id="disclaimer"></footer>
415
416 <script>
417 (function () {
418     "use strict";
419     var API = "/api/v1";
420     // 9/5: 배포 서버에서 API_ACCESS_KEY(app/security.py)를 컷다면, 이 값에 같은 키를 넣어
421     // 다시 배포하세요. 비어 있으면(로컬/기본 데모) 헤더를 안 붙입니다. 이걸 넣기 전엔
422     // 프론트가 인증 헤더를 아예 안 보내서, 서버 키만 켜면 데모 전체가 401로 죽었습니다.
423     var API_KEY = "";
424     function authHeaders(base) {
425         base = base || {};
426         if (API_KEY) base["X-API-Key"] = API_KEY;
427         return base;
428     }
429     var state = { personas: [], userId: null, persona: null, scenarios: null, shield: null,
430         tax: null, sources: [], budgets: [], scenarioKey: "기본", pane: "cash" };
431
432     function escapeHtml(s) {
433         return String(s == null ? "" : s).replace(/[\<>'"]/, function (c) {
434             return { "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39;" }[c];
435         });
436     }
437     function won(n) { return Math.round(Number(n) || 0).toLocaleString("ko-KR") + "원"; }
438     function manwon(n) {
439         var v = Number(n) || 0;
440         if (Math.abs(v) >= 10000) return (v / 10000).toFixed(Math.abs(v) >= 100000 ? 0 : 1) + "만";
441         return String(Math.round(v));
442     }
443     function mmdd(iso) { var p = String(iso).slice(5, 10).split("-"); return p[0] + "/" + p[1]; }
444     function el(id) { return document.getElementById(id); }
445     function fail(id, msg) { el(id).innerHTML = '<div class="err">' + escapeHtml(msg) + '</div>'; }
446     function get(path) {
447         return fetch(API + path, { headers: authHeaders({ Accept: "application/json" }) }).then(function (r) {
448             if (!r.ok) throw new Error("HTTP " + r.status);
449             return r.json();
450         });
451     }
452     // 백엔드가 주는 영문 등급/규칙 코드는 내부용 표현입니다. 사용자에게는 한글로 보여주고,
453     // 규칙 코드(R003 등)는 근거를 확인하고 싶은 사람만 볼 수 있게 툴팁으로 넘깁니다.
454     function sevLabel(s) { return { LOW: "정상", MEDIUM: "주의", HIGH: "경고", CRITICAL: "위험" }[s] || s; }
455     // 경비 인정 확률을 3단계로 나눠, 낮은 항목이 "AI가 잘못 골랐다"처럼 보이지 않게 합니다.
456     function expenseTone(p) { return p >= 50 ? "safe" : p >= 30 ? "warn" : "neutral"; }
457     function expenseTag(p) { return p >= 50 ? "경비 인정 가능" : p >= 30 ? "확인 필요" : "개인지출 추정"; }
458     function gigGrade(v) { return v >= 800 ? "우수" : v >= 700 ? "양호" : v >= 500 ? "보통" : "주의"; }
459
460     function riskClass(level) {

```

```

461     if (level === "위험" || level === "CRITICAL" || level === "HIGH") return "danger";
462     if (level === "주의" || level === "MEDIUM") return "warn";
463     return "safe";
464 }
465
466 // ----- 화면 전환 -----
467 function showLanding() {
468     el("viewSim").hidden = true;
469     el("viewDash").hidden = true;
470     el("viewLanding").hidden = false;
471     el("backBtn").hidden = true;
472     try { history.replaceState(null, "", location.pathname); } catch (e) {}
473     window.scrollTo(0, 0);
474 }
475 function showDash() {
476     el("viewSim").hidden = true;
477     el("viewLanding").hidden = true;
478     el("viewDash").hidden = false;
479     el("backBtn").hidden = false;
480     window.scrollTo(0, 0);
481 }
482 el("backBtn").addEventListener("click", showLanding);
483 el("tryMineBtn").addEventListener("click", showSim);
484 el("simAddBtn").addEventListener("click", function () {
485     if (simSources.length >= 5) return;
486     var next = SIM_PRESETS[simSources.length % SIM_PRESETS.length];
487     simSources.push(Object.assign({}, next));
488     renderSimSources();
489 });
490 el("simRunBtn").addEventListener("click", runSimulation);
491
492 // ----- 선택 화면 -----
493 function renderPersonaGrid() {
494     el("personaGrid").innerHTML = state.personas.map(function (p) {
495         return '<button class="persona-card" data-uid="' + escapeHtml(p.user_id) + '>' +
496             '<span class="emoji">' + escapeHtml(p.avatar_emoji) + '</span>' +
497             '<div class="nm">' + escapeHtml(p.name) + '</div>' +
498             '<div class="job">' + escapeHtml(p.job) + '</div>' +
499             '<div class="tg">' + escapeHtml(p.tagline) + '</div>' +
500             '<div id="pstat-' + escapeHtml(p.user_id) + '></div>' +
501             '<div class="go">대시보드 보기 →</div></button>';
502     }).join("");
503     Array.prototype.forEach.call(el("personaGrid").querySelectorAll(".persona-card"), function (b) {
504         b.addEventListener("click", function () { selectUser(b.getAttribute("data-uid")); });
505     });
506
507     // 카드마다 현재 현금흐름 상태를 덧붙입니다(고르기 전에 무엇을 보게 될지 알 수 있게).
508     state.personas.forEach(function (p) {
509         get("/shield/" + p.user_id).then(function (sh) {
510             var box = el("pstat-" + p.user_id);
511             if (!box) return;
512             box.innerHTML = '<span class="pill ' + riskClass(sh.risk_level) + '" style="margin-bottom:4px">' +
513                 '<span class="dot"></span>현금흐름 ' + escapeHtml(sh.risk_level) + '</span>' +
514                 (sh.minimum_balance_date
515                     ? '<div class="label" style="margin-top:6px">' + escapeHtml(mmd(sh.minimum_balance_date)) +
516                       " 잔고 부족 예상</div>"
517                     : '<div class="label" style="margin-top:6px">45일 내 잔고 부족 예상 없음</div>');
518             box.innerHTML += sh.risk_level === "CRITICAL" ? "위험" : sh.risk_level === "HIGH" ? "위험" : sh.risk_level === "MEDIUM" ? "주의" : "안전";
519         });
520     });
521
522 // ----- 차트 -----
523 function drawChart(points, minSafety, shortageDate) {
524     if (!points || !points.length) return '<div class="skeleton">표시할 데이터가 없습니다.</div>';
525     var W = 900, H = 300, PL = 62, PR = 18, PT = 16, PB = 30;
526     var iw = W - PL - PR, ih = H - PT - PB;
527     var vals = points.map(function (p) { return p.balance; }).concat([minSafety, 0]);
528     var lo = Math.min.apply(null, vals), hi = Math.max.apply(null, vals);

```

```

529     if (hi === lo) hi = lo + 1;
530     var pad = (hi - lo) * 0.12; lo -= pad; hi += pad;
531     function X(i) { return PL + (iw * i) / (points.length - 1 || 1); }
532     function Y(v) { return PT + ih - (ih * (v - lo)) / (hi - lo); }
533
534     var s = "", t, i;
535     for (t = 0; t <= 4; t++) {
536         var v = lo + ((hi - lo) * t) / 4, y = Y(v);
537         s += '<line x1="' + PL + '" y1="' + y + '" x2="' + (W - PR) + '" y2="' + y + '" stroke="#f2f4f6" stroke-width
="1"/>';
538         s += '<text x="' + (PL - 10) + '" y="' + (y + 4) + '" text-anchor="end" font-size="11" fill="#8b95a1">' + manw
on(v) + "</text>";
539     }
540     var step = Math.max(1, Math.round(points.length / 6));
541     for (i = 0; i < points.length; i += step) {
542         s += '<text x="' + X(i) + '" y="' + (H - 8) + '" text-anchor="middle" font-size="11" fill="#8b95a1">' + mmd(p
oints[i].date) + "</text>";
543     }
544     var line = points.map(function (p, k) { return (k ? "L" : "M") + X(k).toFixed(1) + " " + Y(p.balance).toFixed
(1); }).join(" ");
545     var area = line + " L" + X(points.length - 1).toFixed(1) + " " + Y(lo).toFixed(1) + " L" + X(0).toFixed(1) + " "
+ Y(lo).toFixed(1) + " Z";
546     s += '<path d="' + area + '" fill="url(#g1)"/>';
547     s += '<path d="' + line + '" fill="none" stroke="#3182f6" stroke-width="2.6" stroke-linejoin="round"/>';
548
549     var ys = Y(minSafety);
550     s += '<line x1="' + PL + '" y1="' + ys + '" x2="' + (W - PR) + '" y2="' + ys + '" stroke="#f04452" stroke-width
="1.3" stroke-dasharray="5 4"/>';
551     s += '<text x="' + (W - PR) + '" y="' + (ys - 7) + '" text-anchor="end" font-size="11" fill="#f04452">최소 안전
잔액' + manwon(minSafety) + "</text>";
552
553     if (shortageDate) {
554         for (var k2 = 0; k2 < points.length; k2++) {
555             if (points[k2].date === shortageDate) {
556                 var xs = X(k2);
557                 s += '<line x1="' + xs + '" y1="' + PT + '" x2="' + xs + '" y2="' + (PT + ih) + '" stroke="#f04452" stroke
-width="1.2" stroke-dasharray="3 3" opacity="0.8"/>';
558                 s += '<circle cx="' + xs + '" cy="' + Y(points[k2].balance) + '" r="4.5" fill="#f04452"/>';
559                 break;
560             }
561         }
562     }
563     points.forEach(function (p, k3) {
564         if (!p.events || !p.events.length) return;
565         s += '<circle cx="' + X(k3) + '" cy="' + Y(p.balance) + '" r="3.6" fill="fff" stroke="#3182f6" stroke-width
="2"/>' +
566         "<title>" + escapeHtml(p.date + " · " + p.events.join(" / ")) + "</title></circle>";
567     });
568     return '<svg class="chart" viewBox="0 0 ' + W + " " + H + '" role="img" aria-label="45일 현금흐름 예측 그래프">'
+
569     '<defs><linearGradient id="g1" x1="0" y1="0" x2="0" y2="1">' +
570     '<stop offset="0%" stop-color="#3182f6" stop-opacity="0.16"/>' +
571     '<stop offset="100%" stop-color="#3182f6" stop-opacity="0.01"/></linearGradient></defs>' + s + "</svg>";
572 }
573
574 function renderChart() {
575     var sh = state.shield;
576     if (!sh) return;
577     var pts, shortage = null;
578     if (state.scenarioKey === "기본") {
579         pts = sh.daily_balances; shortage = sh.minimum_balance_date;
580     } else {
581         var sc = state.scenarios && state.scenarios.scenarios ? state.scenarios.scenarios[state.scenarioKey] : null;
582         pts = sc || sh.daily_balances;
583         for (var i = 0; i < pts.length; i++) {
584             if (pts[i].balance < sh.min_safety_balance) { shortage = pts[i].date; break; }
585         }
586     }

```

```

587     el("chartWrap").innerHTML = drawChart(pts, sh.min_safety_balance, shortage);
588     var lines = ['<span><i style="background:#3182f6"></i>예상 잔액</span>',
589                 '<span><i style="background:#f04452"></i>최소 안전잔액</span>'];
590     if (state.scenarioKey === "기본") lines.push("<span>◦ 점 = 정산·정기결제 발생일 (마우스를 올리면 내역)</span>");
591     else {
592         lines.push("<span>가정에 따른 예상 경로입니다 - N잡러에게 가장 찾은 두 상황(정산 지연·일감 감소)을 적용했습니다</span>");
593         var base = sh.minimum_balance_date;
594         if (shortage && base) {
595             var gap = Math.round((new Date(base) - new Date(shortage)) / 86400000);
596             if (gap > 0) lines.push('<span style="color:#f04452;font-weight:700">기본 대비 ' + gap + "일 앞당겨짐</span>');
597         } else if (shortage && !base) {
598             lines.push('<span style="color:#f04452;font-weight:700">기본에서는 없던 잔고 부족이 생깁니다</span>');
599         }
600     }
601     if (shortage) lines.push('<span style="color:#f04452">■ ' + escapeHtml(mmdd(shortage)) + " 잔고 부족 예상</span>");
602     el("chartLegend").innerHTML = lines.join("");
603 }
604
605 function renderScenarioTabs() {
606     var keys = ["기본", "입금지연", "수입감소"];
607     var labels = { "기본": "기본", "입금지연": "정산 10일 지연", "수입감소": "수입 30% 감소" };
608     el("scenarioTabs").innerHTML = keys.map(function (k) {
609         return '<button class="tab" role="tab" data-k="' + k + '" aria-selected="' + (k === state.scenarioKey) + '">' + labels[k] + "</button>";
610     }).join("");
611     Array.prototype.forEach.call(el("scenarioTabs").querySelectorAll(".tab"), function (b) {
612         b.addEventListener("click", function () {
613             state.scenarioKey = b.getAttribute("data-k");
614             renderScenarioTabs(); renderChart();
615         });
616     });
617 }
618
619 // ----- 섹션 -----
620 function renderShield(sh) {
621     state.shield = sh;
622     var srcNote = sh.balance_source === "user_set" ? "연결 계좌 기준" : "최근 30일 수입·지출로 추정";
623     var days = null;
624     if (sh.minimum_balance_date) {
625         days = Math.round((new Date(sh.minimum_balance_date) - new Date(String(sh.calculated_at).slice(0, 10))) / 86400000);
626     }
627     el("shieldSummary").innerHTML =
628         '<div style="display:flex;gap:34px;align-items:flex-end;flex-wrap:wrap;margin-bottom:10px">' +
629         '<div><div class="label">현재 잔액 <span style="color:var(--text-faint)">· ' + escapeHtml(srcNote) + "</span></div>" +
630         '<div class="metric num">' + won(sh.current_balance) + "</div></div>" +
631         '<div><div class="label">세금·건보료 제외 가용액</div><div class="metric-sm num">' + won(sh.available_cash) +
632         "</div></div>" +
633         '<div><div class="label">45일 내 잔고 부족</div><div class="metric-sm">' +
634         (sh.minimum_balance_date
635             ? '<span style="color:var(--danger)">' + escapeHtml(mmdd(sh.minimum_balance_date)) + " (" + days + "일 후)</span>"
636             : '<span style="color:var(--safe)">예상 없음</span>') + "</div></div>" +
637         '<div style="margin-left:auto"><span class="pill ' + riskClass(sh.risk_level) + '"><span class="dot"></span>' +
638         escapeHtml(sh.risk_level) + "</span></div></div>" +
639         '<ul class="reasons">' + (sh.risk_reasons || []).map(function (r) { return "<li>" + escapeHtml(r) + "</li>";
640     }).join("") + "</ul>";
641     renderScenarioTabs();
642     renderChart();
643 }
644

```

```

645 function renderIncome(sum) {
646   el("incomePeriod").textContent = ". " + sum.period_label;
647   var rows = (sum.by_platform || []).map(function (p) {
648     return '<div class="list-row"><span style="font-size:17px">' + escapeHtml(p.icon_emoji) + "</span>" +
649       '<span class="grow"><span class="truncate" style="display:block;font-weight:550">' + escapeHtml(p.platform_name) + "</span>" +
650       '<span class="label">' + escapeHtml(p.platform_type) + ". " + p.event_count + "건</span></span>" +
651       '<span class="num" style="font-weight:600">' + won(p.total_amount) + "</span></div>";
652   }).join("");
653   el("income").innerHTML =
654     '<div class="label">정산 완료</div>' +
655     '<div class="metric num" style="margin-bottom:12px">' + won(sum.total_settled) + "</div>" +
656     '<div class="list">' + (rows || '<div class="skeleton">연결된 플랫폼이 없습니다.</div>') + "</div>" +
657     (sum.unconnected_source_count
658       ? '<div class="muted-box" style="margin-top:12px">아직 연결하지 않은 플랫폼이 ' + sum.unconnected_source_count +
659         '곳 있습니다. 연결할수록 현금흐름 예측이 정확해집니다.</div>' : "");
660   }
661
662 function renderGig(g) {
663   var pct = Math.max(0, Math.min(100, (g.score / 1000) * 100));
664   var names = { platform_connect_rate: "수입 플랫폼 연결률", spending_stability: "지출 안정성", expense_diligence:
"업무경비 인정률" };
665   var comp = Object.keys(g.components || {}).map(function (k) {
666     return '<div class="kv"><span class="label">' + escapeHtml(names[k] || k) + '</span><span class="num">' +
667       Math.round(g.components[k] * 100) + "점</span></div>";
668   }).join("");
669   el("gig").innerHTML =
670     '<div style="display:flex;align-items:baseline;gap:6px"><span class="metric num">' + g.score +
671     '</span><span class="label">/ 1000</span>' +
672     '<span class="pill ' + (g.score >= 700 ? "safe" : g.score >= 500 ? "warn" : "neutral") +
673     '" style="margin-left:auto">' + gigGrade(g.score) + "</span></div>" +
674     '<div class="gauge-track" style="position:relative;overflow:visible">' +
675     '<span style="width:' + pct.toFixed(1) + "%"></span>' +
676     '<span style="position:absolute;left:70%;top:-3px;width:2px;height:14px;background:var(--text-faint);border-
radius:1px"></span>' +
677     "</div>" +
678     '<div class="label" style="margin-bottom:10px">누금은 양호 기준선 700점</div>' + comp +
679     '<div class="muted-box" style="margin-top:12px">여러 플랫폼의 정산 이력·지출 안정성·경비 인정률을 하나로 묶은 자체
지표입니다. ' +
680     "금융기관의 신용점수가 아닙니다.</div>";
681   }
682
683 function renderBudget(rows) {
684   if (!rows || !rows.length) { el("budget").innerHTML = '<div class="skeleton">설정된 예산이 없습니다.</div>'; return; }
685   rows.sort(function (a, b) { return (b.usage_rate || 0) - (a.usage_rate || 0); });
686   el("budget").innerHTML = '<div class="list">' + rows.map(function (b) {
687     var pct = (b.usage_rate || 0) * 100;
688     var cls = pct >= 100 ? "over" : pct >= 80 ? "near" : "";
689     var color = pct >= 100 ? "var(--danger)" : pct >= 80 ? "var(--warn)" : "var(--text-muted)";
690     return '<div class="list-row" style="display:block;padding:10px 0">' +
691       '<div style="display:flex;gap:8px;align-items:baseline;margin-bottom:6px">' +
692       '<span style="font-weight:550">' + escapeHtml(b.category) + "</span>" +
693       '<span class="num" style="margin-left:auto;color:' + color + ';font-weight:600">' + pct.toFixed(0) + "%</s
pan></div>" +
694       '<div class="bar ' + cls + '"><span style="width:' + Math.min(100, pct).toFixed(1) + "%"></span></div>' +
695       '<div class="label num" style="margin-top:5px">' + won(b.spent_amount) + " / " + won(b.limit_amount) + "</di
v></div>";
696   }).join("") + "</div>";
697   }
698
699 function renderRisk(score, events) {
700   el("riskBadge").innerHTML = '<span class="pill ' + riskClass(score.severity) + '"><span class="dot"></span>' +
701     escapeHtml(sevLabel(score.severity)) + ". " + score.score + "점</span>";
702   if (!events || !events.length) { el("risk").innerHTML = '<div class="skeleton">탐지된 이상거래가 없습니다.</div
>'; return; }
703   el("risk").innerHTML = '<div class="list">' + events.slice(0, 6).map(function (e) {

```

```

704     return '<div class="list-row">' +
705         '<span class="pill ' + riskClass(e.severity) + ' ' style="min-width:52px;justify-content:center">' + escapeHt
ml(sevLabel(e.severity)) + "</span>" +
706         '<span class="grow"><span class="truncate" style="display:block;font-weight:550">' + escapeHtml(e.merchant_n
ame || "화면에서 직접 검사한 거래") +
707         '(e.transaction_amount ? ' <span class="num" style="color:var(--text-muted);font-weight:400">' + won(e.tran
saction_amount) + "</span>" : "") + "</span>" +
708         '<span class="label truncate" style="display:block" title="' + escapeHtml(e.rule_id) + '">' + escapeHtml(e.d
escription) + "</span></span>" +
709         '<span class="label num">' + escapeHtml(e.transaction_timestamp ? mmdd(e.transaction_timestamp) : "") + "</s
pan></div>";
710     }).join("") + "</div>";
711 }
712
713 function renderExpense(x) {
714     var q = x.transactions_queue || [];
715     var head = '<div style="display:flex;gap:28px;margin-bottom:12px">' +
716         '<div><div class="label">업무경비 후보</div><div class="metric-sm num">' +
717         q.filter(function (t) { return t.prob >= 50; }).length + "건</div></div>" +
718         '<div><div class="label">예상 환급액</div><div class="metric-sm num">' + won(x.current_estimated_refund) + "</
div></div></div>";
719     if (!q.length) { el("expense").innerHTML = head + '<div class="skeleton">분류할 거래가 없습니다.</div>'; return;
}
720     // 이 목록은 "업무경비 후보"만이 아니라 최근 거래 전체의 분류 결과입니다. 제목만 보고
721     // 유흥업소 같은 개인지출까지 경비 후보로 오해할 수 있어, 무엇을 보는 목록인지 밝히고
722     // 인정 확률이 높은 것부터 정렬합니다(50% 이상이 경비 후보).
723     q = q.slice().sort(function (a, b) { return (b.prob || 0) - (a.prob || 0); });
724     el("expense").innerHTML = head +
725         '<div class="label" style="margin-bottom:8px">최근 거래 ' + q.length + '건의 분류 결과 · ' +
726         '<span style="color:var(--safe);font-weight:600">초록</span>은 경비 인정 가능, ' +
727         '<span style="color:var(--warn);font-weight:600">노랑</span>은 직접 확인이 필요한 건입니다</div>' +
728         '<div class="list">' + q.map(function (t) {
729         return '<div class="list-row">' +
730             '<span class="grow"><span class="truncate" style="display:block;font-weight:550">' + escapeHtml(t.merchant)
+ "</span>" +
731             '<span class="label">' + escapeHtml(t.ai_tag) + " · " + expenseTag(t.prob) + "</span></span>" +
732             '<span class="num label">' + won(t.amount) + "</span>" +
733             '<span class="pill ' + expenseTone(t.prob) + ' ' style="min-width:52px;justify-content:center">' + t.prob +
"%</span></div>";
734         }).join("") + "</div>";
735     }
736
737     // ----- 직접 입력 시뮬레이터 -----
738     //
739     // 준비된 인물을 구경만 하는 대신, 보는 사람이 자기 조건을 넣어 결과가 바뀌는 걸 확인할 수
740     // 있게 하는 화면입니다. 계산은 서버(POST /api/v1/simulate/cashflow)가 하고, 그리는 방식은
741     // 대시보드 차트와 같은 함수를 그대로 씁니다.
742     var SIM_PRESETS = [
743         { name: "배달", cycle_days: 7, amount: 280000, days_until_next: 3 },
744         { name: "프리랜서 용역", cycle_days: 14, amount: 1200000, days_until_next: 9 },
745         { name: "본업 급여", cycle_days: 30, amount: 2400000, days_until_next: 20 }
746     ];
747     // 기본값은 "배달 하나로 버티는" 조건입니다. 수입원을 둘 이상 넣으면 월 순증가로 돌아서
748     // 45일 안에는 부족이 생기지 않아, 처음 계산했을 때 이 기능이 무엇을 알려주는지가 드러나지
749     // 않습니다. 그래서 기본은 부족이 실제로 보이는 조건으로 두고, 사용자가 수입원을 더하면
750     // 부족 시점이 사라지는 변화를 직접 보게 했습니다.
751     var simSources = SIM_PRESETS.slice(0, 1).map(function (p) { return Object.assign({}, p); });
752     var simScenarioKey = "기본";
753     var simData = null;
754
755     function renderSimSources() {
756         el("simSources").innerHTML = simSources.map(function (s, i) {
757             return '<div style="display:grid;grid-template-columns:1.3fr 1fr 1fr 1fr auto;gap:9px;align-items:end;margin-b
ottom:10px">' +
758                 '<div class="field"><label>이름</label><input type="text" data-i="' + i + '" data-k="name" value="' + escape
Html(s.name) + '"></div>' +
759                 '<div class="field"><label>며칠마다</label><input type="number" data-i="' + i + '" data-k="cycle_days" value
="' + s.cycle_days + '" min="1" max="90"></div>' +

```

```

760     '<div class="field"><label>1회 금액</label><input type="number" data-i="' + i + '" data-k="amount" value="'
+ s.amount + '" step="10000" min="0"></div>' +
761     '<div class="field"><label>다음까지</label><input type="number" data-i="' + i + '" data-k="days_until_next"
value="' + s.days_until_next + '" min="0" max="90"></div>' +
762     '<button class="btn btn-outline btn-sm" data-del="' + i + '">삭제</button></div>';
763     }).join("") || '<div class="skeleton">수입원을 하나 이상 추가해 주세요.</div>';
764
765     Array.prototype.forEach.call(el("simSources").querySelectorAll("input"), function (inp) {
766         inp.addEventListener("input", function () {
767             var i = Number(inp.getAttribute("data-i")), k = inp.getAttribute("data-k");
768             simSources[i][k] = k === "name" ? inp.value : Number(inp.value);
769         });
770     });
771     Array.prototype.forEach.call(el("simSources").querySelectorAll("button[data-del]"), function (b) {
772         b.addEventListener("click", function () {
773             simSources.splice(Number(b.getAttribute("data-del")), 1);
774             renderSimSources();
775         });
776     });
777 }
778
779 function renderSimResult() {
780     if (!simData) return;
781     var pts = simData.scenarios[simScenarioKey] || simData.daily_balances;
782     var shortage = simData.shortage_dates[simScenarioKey];
783     var floor = simData.min_safety_balance;
784     var baseShort = simData.shortage_dates["기본"];
785
786     var head;
787     if (shortage) {
788         var d = shortage.slice(5).split("-");
789         var days = Math.round((new Date(shortage) - new Date(pts[0].date)) / 86400000);
790         head = '<div class="hero-line" style="font-size:24px;margin:0 0 14px">' +
791             '<b class="risk">' + Number(d[0]) + "월 " + Number(d[1]) + "일</b>, " + days +
792             "일 뒤에 잔고가 " + manwon(floor) + "원 아래로 떨어집니다</div>";
793     } else {
794         head = '<div class="hero-line" style="font-size:24px;margin:0 0 14px">앞으로 45일은 <b>잔고 걱정 없이</b> 지낼
수 있습니다</div>';
795     }
796
797     var tabs = ["기본", "입금자연", "수입감소"];
798     var labels = { "기본": "기본", "입금자연": "정산 10일 자연", "수입감소": "수입 30% 감소" };
799     var tabHtml = '<div class="tabs" id="simTabs" style="margin-bottom:12px">' + tabs.map(function (k) {
800         return '<button class="tab" data-k="' + k + '" aria-selected="' + (k === simScenarioKey) + '">' + labels[k] +
"</button>";
801     }).join("") + "</div>";
802
803     var delta = "";
804     if (simScenarioKey !== "기본" && shortage && baseShort) {
805         var gap = Math.round((new Date(baseShort) - new Date(shortage)) / 86400000);
806         if (gap > 0) delta = '<span style="color:var(--danger);font-weight:700">기본 대비 ' + gap + "일 앞당겨짐</span>";
807     } else if (simScenarioKey !== "기본" && shortage && !baseShort) {
808         delta = '<span style="color:var(--danger);font-weight:700">기본에서는 없던 잔고 부족이 생깁니다</span>';
809     }
810
811     el("simResult").innerHTML = '<div class="card">' + head +
812         '<div style="display:flex;gap:30px;flex-wrap:wrap;margin-bottom:14px">' +
813         '<div><div class="label">월 예상 수입</div><div class="num" style="font-size:20px;font-weight:700">' +
814             won(simData.monthly_income_estimate) + "</div></div>' +
815         '<div><div class="label">월 예상 지출</div><div class="num" style="font-size:20px;font-weight:700">' +
816             won(simData.monthly_cost_estimate) + "</div></div>' +
817         '<div><div class="label">월 순증감</div><div class="num" style="font-size:20px;font-weight:700;color:' +
818             (simData.net_monthly >= 0 ? "var(--safe)" : "var(--danger)") + '">' +
819             (simData.net_monthly >= 0 ? "+" : "") + won(simData.net_monthly) + "</div></div></div>' +
820     tabHtml +
821     drawChart(pts, floor, shortage) +
822     '<div class="legend"><span><i style="background:#3182f6"></i>예상 잔액</span>' +

```

```

823         '<span><i style="background:#f04452"></i>최소 안전잔액</span>' + (delta ? "<span>" + delta + "</span>" : "")
+ "</div>" +
824         '<div class="muted-box" style="margin-top:14px">' + escapeHtml(simData.note) + "</div></div>";
825
826     Array.prototype.forEach.call(el("simTabs").querySelectorAll(".tab"), function (b) {
827         b.addEventListener("click", function () { simScenarioKey = b.getAttribute("data-k"); renderSimResult(); });
828     });
829 }
830
831 function runSimulation() {
832     var btn = el("simRunBtn");
833     btn.disabled = true; btn.textContent = "계산 중...";
834     send("POST", "/simulate/cashflow", {
835         current_balance: Number(el("simBal").value) || 0,
836         min_safety_balance: Number(el("simFloor").value) || 0,
837         fixed_monthly_cost: Number(el("simFixed").value) || 0,
838         fixed_cost_day: Number(el("simDay").value) || 25,
839         daily_variable_cost: Number(el("simVar").value) || 0,
840         sources: simSources
841     }).then(function (d) {
842         simData = d; simScenarioKey = "기본"; renderSimResult();
843     }).catch(function () {
844         el("simResult").innerHTML = '<div class="err">계산에 실패했습니다. 입력값을 확인해 주세요.</div>';
845     }).then(function () { btn.disabled = false; btn.textContent = "45일 예측 계산하기"; });
846 }
847
848 function showSim() {
849     el("viewLanding").hidden = true; el("viewDash").hidden = true;
850     el("viewSim").hidden = false; el("backBtn").hidden = false;
851     renderSimSources();
852     window.scrollTo(0, 0);
853 }
854
855 // ----- 1단: 요약 -----
856 /** 이 사람의 상태를 한 문장으로 압축합니다. 사용자가 앱을 켜서 가장 먼저 묻는 질문
857  * ("나 지금 괜찮나?")에 스크롤 없이 답하는 자리입니다. */
858 function renderHero() {
859     var sh = state.shield, cur = state.persona;
860     if (!sh) return;
861     el("heroWho").textContent = cur ? cur.name + " · " + cur.job : "";
862
863     var line;
864     if (sh.minimum_balance_date) {
865         var d = String(sh.minimum_balance_date).slice(5).split("-");
866         var days = Math.round((new Date(sh.minimum_balance_date) - new Date(String(sh.calculated_at).slice(0, 10))) /
867 864000000);
868         line = '<b class="risk">' + Number(d[0]) + "월 " + Number(d[1]) + "일</b>, " + days +
869             "일 뒤에 잔고가 " + manwon(sh.min_safety_balance) + "원 아래로 떨어질 것 같아요";
870     } else {
871         line = "앞으로 45일은 <b>잔고 걱정 없이</b> 지낼 수 있어요";
872     }
873     el("heroLine").innerHTML = line;
874
875     // 다음 입금 예정: 시뮬레이션에 잡힌 첫 수입 이벤트
876     var next = null;
877     for (var i = 0; i < (sh.daily_balances || []).length; i++) {
878         var p = sh.daily_balances[i];
879         if (p.events && p.events.some(function (e) { return e.indexOf("수입") >= 0; })) { next = p; break; }
880     }
881     var nextText = "예정 없음";
882     if (next) {
883         var m = /\+([\d,]+)원/.exec(next.events.join(" "));
884         nextText = mmd(next.date) + (m ? " · " + m[1] + "원" : "");
885     }
886     el("heroStats").innerHTML =
887         '<div class="st"><div class="k">현재 잔액</div><div class="v num">' + won(sh.current_balance) + "</div></div>"
+
888         '<div class="st"><div class="k">세금·건보료 빼고 쓸 수 있는 돈</div><div class="v num">' + won(sh.available_cas

```

```

h) + "</div></div>" +
888     '<div class="st"><div class="k">다음 입금</div><div class="v num">' + escapeHtml(nextText) + "</div></div>";
889 }
890
891 /** 계산된 수치를 문장으로 풀어주는 AI 브리핑. 실패해도 화면에 영향이 없도록
892  * 아무 것도 그리지 않고 조용히 넘어갑니다(백엔드가 템플릿 문장으로 이미 풀백함). */
893 function renderBrief(b) {
894     if (!b || !b.summary) { el("brief").innerHTML = ""; return; }
895     el("brief").innerHTML = '<div class="brief"><span class="bi">AI 브리핑</span>' +
896         '<span><span class="bt">' + escapeHtml(b.summary) + "</span>" +
897         (b.action ? '<span class="ba">' + escapeHtml(b.action) + "</span>" : "") + "</span></div>";
898 }
899
900 // ----- 2단: 할 일 -----
901 /** 각 카드가 "지금 하면 좋은 것" 하나씩입니다. 우선순위가 높은 3개만 보여줍니다. */
902 function renderTodos() {
903     var items = [];
904     var sh = state.shield, tax = state.tax, sources = state.sources || [], budgets = state.budgets || [];
905
906     var pending = sources.filter(function (s) { return !s.connected; });
907     if (pending.length) {
908         items.push({
909             pri: 1, icon: "🔗", tone: "accent",
910             title: pending[0].platform_name + " 연결하기",
911             desc: "연결하면 이 수입까지 반영해서 예측이 더 정확해져요.",
912             action: '<button class="btn btn-primary btn-sm" data-connect="' + escapeHtml(pending[0].source_id) + '">연결
913             하기</button>'
914         });
915     }
916     if (tax && tax.health_insurance && tax.health_insurance.warning) {
917         items.push({ pri: 2, icon: "🏠", tone: "warn", title: "건강보험료 주의", desc: tax.health_insurance.warning });
918     }
919     if (tax && tax.income_tax) {
920         var bal = tax.income_tax.balance_due;
921         items.push(bal < 0
922             ? { pri: 3, icon: "💰", tone: "safe", title: "5월에 " + won(-bal) + " 돌려받아요",
923                 desc: "3.3% 원천징수가 실제 세금보다 많이 떼였습니다. 종합소득세 신고를 하면 환급됩니다." }
924             : { pri: 1, icon: "💰", tone: "warn", title: "세금 " + won(bal) + " 준비하세요",
925                 desc: "5월 신고 때 낼 돈이에요. 매달 " + won(tax.income_tax.monthly_reserve) + "씩 떼어두면 됩니다." });
926     }
927     var over = budgets.filter(function (b) { return (b.usage_rate || 0) >= 1; });
928     if (over.length) {
929         items.push({ pri: 2, icon: "📊", tone: "warn", title: over.length + "개 예산 초과",
930             desc: over.map(function (b) { return b.category; }).join(", ") + " 한도를 넘었어요." });
931     }
932     if (sh && sh.risk_level !== "안전") {
933         items.push({ pri: 2, icon: "⚠️", tone: "warn", title: "현금흐름 " + sh.risk_level,
934             desc: (sh.risk_reasons || [])[0] || "" });
935     }
936     items.sort(function (a, b) { return a.pri - b.pri; });
937     var bg = { accent: "var(--accent-soft)", warn: "var(--warn-soft)", safe: "var(--safe-soft)" };
938     el("todos").innerHTML = items.slice(0, 3).map(function (t) {
939         return '<div class="todo"><div class="ic" style="background:' + bg[t.tone] + '>' + t.icon + "</div>" +
940             '<div><div class="tt">' + escapeHtml(t.title) + '</div><div class="ds">' + escapeHtml(t.desc) + "</div>" +
941             (t.action ? '<div class="act">' + t.action + "</div>" : "") + "</div></div>";
942     }).join("");
943
944     var cb = el("todos").querySelector("button[data-connect]");
945     if (cb) cb.addEventListener("click", function () { connectSource(cb.getAttribute("data-connect"), cb); });
946 }
947
948 /** 대응 가이드 - 상품을 추천하지 않고, 상황 근거와 선택지·주의점을 함께 보여줍니다. */
949 function renderGuidance(g) {
950     var items = (g && g.items) || [];
951     if (!items.length) { el("guidance").innerHTML = '<div class="skeleton">표시할 안내가 없습니다.</div>'; return; }
952     var bg = { safe: "var(--safe-soft)", warn: "var(--warn-soft)", neutral: "#f2f4f6" };

```

```

953     var fg = { safe: "var(--safe)", warn: "var(--warn)", neutral: "var(--text-muted)" };
954     el("guidance").innerHTML = items.map(function (it, i) {
955         return '<div style="padding:' + (i ? "18px 0 0" : "0") + '>' +
956             (i ? '<div style="border-top:1px solid var(--border);margin-bottom:18px"></div>' : "") +
957             '<div style="display:flex;align-items:center;gap:9px;margin-bottom:6px">' +
958                 '<span style="width:8px;height:8px;border-radius:50%;background:' + fg[it.tone] + '></span>' +
959                 '<span style="font-size:16px;font-weight:700;letter-spacing:-0.02em">' + escapeHtml(it.title) + "</span></div>" +
div>" +
960         '<div style="background:' + bg[it.tone] + ';border-radius:10px;padding:11px 13px;font-size:13.5px;' +
961             'color:var(--text-muted);margin-bottom:10px">' + escapeHtml(it.why) + "</div>" +
962         '<ul style="margin:0 0 8px;padding-left:18px">' +
963             (it.options || []).map(function (o) {
964                 return '<li style="font-size:14px;line-height:1.65;margin-bottom:5px">' + escapeHtml(o) + "</li>";
965             }).join("") + "</ul>" +
966         (it.caution ? '<div class="label" style="color:var(--text-faint)">※ ' + escapeHtml(it.caution) + "</div>" :
"" ) +
967         "</div>";
968     }).join("") +
969     '<div class="muted-box" style="margin-top:18px">' + escapeHtml(g.disclaimer || "") + "</div>";
970 }
971
972 // ----- 3단: 탭 -----
973 var PANES = [
["cash", "현금흐름"],
["income", "수입"],
["spend", "지출"],
["guide", "대응 가이드"],
["sec", "거래 보
안"]
];
974 function renderMainTabs() {
975     el("mainTabs").innerHTML = PANES.map(function (p) {
976         return '<button class="maintab" role="tab" data-p="' + p[0] + '" aria-selected="' +
977             (p[0] === state.pane) + '>' + p[1] + "</button>";
978     }).join("");
979     PANES.forEach(function (p) { el("pane-" + p[0]).hidden = p[0] !== state.pane; });
980     Array.prototype.forEach.call(el("mainTabs").querySelectorAll(".maintab"), function (b) {
981         b.addEventListener("click", function () {
982             state.pane = b.getAttribute("data-p");
983             renderMainTabs();
984             // 차트는 숨겨진 동안 폭을 0으로 계산하므로, 보이게 된 뒤 다시 그림니다.
985             if (state.pane === "cash") renderChart();
986         });
987     });
988 }
989
990 // ----- 쓰기 동작 (실제로 서버 상태를 바꿉니다) -----
991 function send(method, path, body) {
992     return fetch(API + path, {
993         method: method,
994         headers: authHeaders({ "Content-Type": "application/json", Accept: "application/json" }),
995         body: body ? JSON.stringify(body) : undefined
996     }).then(function (r) {
997         if (!r.ok) throw new Error("HTTP " + r.status);
998         return r.json();
999     });
1000 }
1001
1002 var toastTimer = null;
1003 function toast(msg) {
1004     var box = document.getElementById("toastBox");
1005     if (!box) {
1006         box = document.createElement("div");
1007         box.id = "toastBox"; box.className = "toast";
1008         document.body.appendChild(box);
1009     }
1010     box.textContent = msg;
1011     box.hidden = false;
1012     clearTimeout(toastTimer);
1013     toastTimer = setTimeout(function () { box.hidden = true; }, 4200);
1014 }
1015
1016 /** 수입 플랫폼 연결/해제. 연결 상태는 현금흐름 예측에 그대로 반영되므로,
1017  * 성공하면 수입·예측을 모두 다시 불러와 화면 전체를 최신 상태로 맞춥니다. */

```

```

1018 function renderSources(list) {
1019     var pending = (list || []).filter(function (s) { return !s.connected; });
1020     var connected = (list || []).filter(function (s) { return s.connected; });
1021     if (!pending.length) {
1022         el("sources").innerHTML = '<div class="muted-box" style="margin-top:12px">' +
1023             "수입 플랫폼 " + connected.length + "곳이 모두 연결되어 있습니다. 예측에 전부 반영됩니다.</div>";
1024         return;
1025     }
1026     el("sources").innerHTML = '<div style="margin-top:14px;padding-top:14px;border-top:1px solid var(--border)">' +
1027         '<div class="label" style="margin-bottom:8px">연결하지 않은 플랫폼 - 연결하면 이 수입까지 예측에 반영됩니다</div>'
1028         +
1029         pending.map(function (s) {
1030             return '<div class="list-row"><span style="font-size:17px">' + escapeHtml(s.icon_emoji) + "</span>" +
1031                 '<span class="grow"><span style="font-weight:550">' + escapeHtml(s.platform_name) + "</span>" +
1032                 '<span class="label" style="display:block">' + escapeHtml(s.platform_type) +
1033                 " · 정산 내역 " + s.pending_event_count + "건 대기</span></div>" +
1034                 '<button class="btn btn-primary btn-sm" data-sid="' + escapeHtml(s.source_id) + '">연결하기</button></div>'
1035             >';
1036         }).join("") + "</div>";
1037
1038     Array.prototype.forEach.call(el("sources").querySelectorAll("button[data-sid]"), function (b) {
1039         b.addEventListener("click", function () { connectSource(b.getAttribute("data-sid"), b); });
1040     });
1041
1042     /** 플랫폼을 연결하면 예측에 들어가는 수입이 늘어나므로, 수입·예측·세금·요약을 모두
1043     * 다시 불러와 화면 전체를 최신 상태로 맞춥니다. */
1044     function connectSource(sid, btn) {
1045         btn.disabled = true; btn.textContent = "연결 중...";
1046         send("POST", "/income/sources/" + sid + "/connect?user_id=" + state.userId)
1047             .then(function (res) {
1048                 toast(res.platform_name + " 연결 완료 - 정산 내역 " + res.revealed_event_count +
1049                     "건을 찾았습니다. 예측을 다시 계산합니다.");
1050                 return Promise.all([
1051                     loadIncome(state.userId),
1052                     get("/shield/" + state.userId),
1053                     get("/shield/" + state.userId + "/scenarios"),
1054                     get("/tax/estimate/" + state.userId).catch(function () { return null; })
1055                 ]);
1056             })
1057             .then(function (r) {
1058                 state.scenarios = r[2];
1059                 if (r[3]) { state.tax = r[3]; renderParking(); }
1060                 renderShield(r[1]);
1061                 renderHero();
1062                 renderTodos();
1063             })
1064             .catch(function () { btn.disabled = false; btn.textContent = "연결하기"; toast("연결하지 못했습니다."); });
1065     }
1066
1067     function loadIncome(uid) {
1068         return Promise.all([get("/income/summary?user_id=" + uid), get("/income/sources?user_id=" + uid)])
1069             .then(function (r) { state.sources = r[1]; renderIncome(r[0]); renderSources(r[1]); })
1070             .catch(function () { fail("income", "수입 정보를 불러오지 못했습니다."); });
1071     }
1072
1073     /** 세금·건강보험료 전망 카드. app/tax.py의 실제 산식 결과를 그대로 보여줍니다. */
1074     function renderParking() {
1075         var t = state.tax, sh = state.shield;
1076         if (!t) {
1077             el("parking").innerHTML = '<div class="skeleton">연결된 수입 플랫폼이 없어 추정할 수 없습니다.</div>';
1078             return;
1079         }
1080         var it = t.income_tax, h = t.health_insurance, bal = it.balance_due;
1081         el("parking").innerHTML =
1082             '<div class="label">내년 5월 종합소득세</div>' +
1083             '<div class="metric num" style="margin:2px 0 4px;color:' + (bal < 0 ? "var(--safe)" : "var(--text)") + '>' +
1084             (bal < 0 ? "+" + won(-bal) : won(bal)) + "</div>" +

```

```

1084     '<div class="label" style="margin-bottom:16px">' +
1085     (bal < 0 ? "둘러받을 것으로 보여요" : "매달 " + won(it.monthly_reserve) + "씩 모으면 됩니다") + "</div>" +
1086     '<div class="kv"><span class="label">연 사업소득(환산)</span><span class="num">' + won(it.annual_income) + "</s
pan></div>" +
1087     '<div class="kv"><span class="label">필요경비 ' + Math.round(it.expense_rate * 100) + "%</span><span class=\
um\">-\" +
1088     won(it.necessary_expense) + "</span></div>" +
1089     '<div class="kv"><span class="label">과세표준 · 세율</span><span class="num">' + won(it.tax_base) +
1090     " · " + Math.round(it.marginal_rate * 100) + "%</span></div>" +
1091     '<div class="kv"><span class="label">산출세액(지방세 포함)</span><span class="num">' + won(it.total_tax) + "</s
pan></div>" +
1092     '<div class="kv"><span class="label">이미 낸 원천징수 3.3%</span><span class="num">->' + won(it.prepaid_withhold
ing) + "</span></div>" +
1093     '<div style="margin:18px 0 4px;padding-top:16px;border-top:1px solid var(--border)">' +
1094     '<div class="label">건강보험료 (' + escapeHtml(h.subscriber_type) + ")</div>" +
1095     '<div class="metric-sm num" style="margin-top:2px">월 ' + won(h.total_monthly) + "</div></div>" +
1096     (h.warning ? '<div class="muted-box" style="margin-top:12px;background:var(--warn-soft);color:var(--warn)">' +
1097     escapeHtml(h.warning) + "</div>" : "") +
1098     '<div class="muted-box" style="margin-top:12px">' + escapeHtml(t.disclaimer) + "</div>" +
1099     (sh ? '<div style="margin-top:16px;padding-top:16px;border-top:1px solid var(--border)">' +
1100     '<label class="label" for="safetyInput">최소 안전잔액 (이 밑으로 떨어지면 경고)</label>' +
1101     '<div style="display:flex;gap:8px;margin-top:8px">' +
1102     '<input type="number" id="safetyInput" step="50000" min="0" value="' + Math.round(sh.min_safety_balance) +
1103     "'>' +
1104     '<button class="btn btn-outline btn-sm" id="safetyBtn" style="white-space:nowrap">적용</button>' +
1105     "</div></div>" : "");
1106     if (el("safetyBtn")) el("safetyBtn").addEventListener("click", applySafety);
1107 }
1108
1109 function applySafety() {
1110     var v = Number(el("safetyInput").value);
1111     if (!(v >= 0)) return;
1112     var btn = el("safetyBtn");
1113     btn.disabled = true; btn.textContent = "...";
1114     send("PUT", "/shield/" + state.userId + "/settings", { min_safety_balance: v })
1115     .then(function () { return Promise.all([get("/shield/" + state.userId), get("/shield/" + state.userId + "/scen
arios")]); })
1116     .then(function (r) {
1117         state.scenarios = r[1];
1118         renderShield(r[0]); renderHero(); renderTodos(); renderParking();
1119         toast("최소 안전잔액을 " + won(v) + "으로 저장하고 예측을 다시 계산했습니다.");
1120     })
1121     .catch(function () { btn.disabled = false; btn.textContent = "적용"; toast("저장하지 못했습니다."); });
1122 }
1123
1124 /** 거래 위험도 즉시 검사 - POST /risk/assess. 실제 룰 엔진 + Isolation Forest 앙상블을
1125  * 그대로 통과하며, 판정 결과는 risk_events에 기록되어 옆 탭지 이력에도 바로 반영됩니다. */
1126 // 각 항목은 "연제.어디서.얼마"를 사람 말로 적어둡니다. 사용자가 가맹점명이나 시각을
1127 // 직접 타이핑하지 않아도, 눌러보기만 하면 판정 과정을 볼 수 있게 하는 게 목적입니다.
1128 var PRESETS = [
1129     { icon: "☕", label: "평소처럼 카페에서 결제", detail: "오후 2시 · 스타벅스 · 5,600원",
1130       merchant: "스타벅스 강남점", amount: 5600, hour: 14, country: "KR", device: "known" },
1131     { icon: "🏠", label: "새벽에 금은방에서 큰 금액", detail: "새벽 2시 · 종로금은방 · 320만원",
1132       merchant: "종로금은방", amount: 3200000, hour: 2, country: "KR", device: "known" },
1133     { icon: "🌐", label: "해외 사이트에서 결제", detail: "새벽 3시 · 미국 · 60만원",
1134       merchant: "Steam_Global", amount: 600000, hour: 3, country: "US", device: "known" },
1135     { icon: "📺", label: "처음 보는 기기에서 결제", detail: "오전 11시 · 쿠팡 · 48만원",
1136       merchant: "쿠팡", amount: 480000, hour: 11, country: "KR", device: "new" }
1137 ];
1138
1139 function setupAssess() {
1140     var hourSel = el("afHour");
1141     if (!hourSel.options.length) {
1142         var opts = "";
1143         for (var h = 0; h < 24; h++) opts += '<option value="' + h + '">' + (h < 10 ? "0" + h : h) + ":00</option>";
1144         hourSel.innerHTML = opts;
1145         hourSel.value = "14";

```

```

1146 }
1147 el("assessPresets").innerHTML = PRESETS.map(function (p, i) {
1148     return '<button class="scenario" data-i="' + i + '">' +
1149         '<span class="ic">' + p.icon + "</span>" +
1150         '<span><span class="t">' + escapeHtml(p.label) + "</span>" +
1151         '<span class="d">' + escapeHtml(p.detail) + "</span></span>" +
1152         '<span class="go">검사 →</span></button>';
1153 }).join("");
1154 Array.prototype.forEach.call(el("assessPresets").querySelectorAll(".scenario"), function (b) {
1155     b.addEventListener("click", function () {
1156         var p = PRESETS[Number(b.getAttribute("data-i"))];
1157         // 폼에도 채워둡니다 - "직접 입력" 을 펼치면 방금 검사한 값이 그대로 보여서
1158         // 무엇을 바꿀 수 있는지 알기 쉽습니다.
1159         el("afMerchant").value = p.merchant; el("afAmount").value = p.amount;
1160         el("afHour").value = String(p.hour); el("afCountry").value = p.country; el("afDevice").value = p.device;
1161         runAssess(b);
1162     });
1163 });
1164 }
1165
1166 function runAssess(srcBtn) {
1167     var btn = srcBtn || el("assessBtn");
1168     var restore = btn.innerHTML;
1169     btn.disabled = true;
1170     if (btn.classList.contains("scenario")) {
1171         var go = btn.querySelector(".go"); if (go) go.textContent = "검사 중...";
1172     } else { btn.textContent = "검사 중..."; }
1173
1174     // 평가 시각은 데모 기준 시각(가장 최근 거래일)의 해당 시간으로 맞춥니다.
1175     // 실제 서버 시각으로 보내면 mock 데이터 기간을 한참 벗어나 과거 이력 기반 규칙
1176     // (단기 반복·이동속도 등)이 비교할 이력을 못 찾습니다.
1177     var base = state.shield && state.shield.calculated_at ? String(state.shield.calculated_at).slice(0, 10)
1178         : new Date().toISOString().slice(0, 10);
1179     var hh = String(el("afHour").value); if (hh.length < 2) hh = "0" + hh;
1180     var payload = {
1181         user_id: state.userId,
1182         merchant_name: el("afMerchant").value || "미상 가맹점",
1183         amount: Number(el("afAmount").value) || 0,
1184         timestamp: base + "T" + hh + ":30:00",
1185         country: el("afCountry").value,
1186         device_id: el("afDevice").value === "new" ? "device_new_" + Date.now() : "device_primary"
1187     };
1188
1189     // 조치별로 "그래서 사용자에게 무슨 일이 일어나는지"까지 한 줄로 풀어 씁니다.
1190     var ACTIONS = {
1191         ALLOW: { label: "그대로 승인", desc: "평소 패턴과 크게 다르지 않아 결제가 정상 진행됩니다." },
1192         FLAG_FOR_REVIEW: { label: "표시해두고 승인", desc: "결제는 진행되지만 확인이 필요한 거래로 남겨둡니다." },
1193         REQUIRE_CONFIRMATION: { label: "본인 확인 요청", desc: "결제를 잠시 멈추고 본인이 맞는지 확인을 요청합니다." },
1194         BLOCK: { label: "결제 차단", desc: "위험이 크다고 판단해 결제를 막고 즉시 알립니다." }
1195     };
1196
1197     send("POST", "/risk/assess", payload).then(function (res) {
1198         var act = ACTIONS[res.action] || { label: res.action, desc: "" };
1199         el("assessResult").innerHTML =
1200             '<div style="background:#f9fafb;border-radius:14px;padding:18px">' +
1201             '<div style="display:flex;align-items:center;gap:10px;flex-wrap:wrap">' +
1202             '<span class="metric-sm">' + escapeHtml(act.label) + "</span>" +
1203             '<span class="pill ' + riskClass(res.severity) + '"><span class="dot"></span>위험도 ' +
1204             res.score + "점</span></div>" +
1205             '<div class="label" style="margin-top:6px">' + escapeHtml(act.desc) + "</div>" +
1206             (res.triggered_rules && res.triggered_rules.length
1207                 ? '<div class="label" style="margin:14px 0 6px">이렇게 판단한 이유 ' +
1208                     res.triggered_rules.length + "가지</div>" +
1209                     '<div class="list">' + res.triggered_rules.map(function (t) {
1210                         return '<div class="list-row"><span class="pill ' + riskClass(t.severity) + '" style="min-width:52px;justify-content:center">' + escapeHtml(sevLabel(t.severity)) + "</span>"
1211                     }) +
1212                     '<span class="grow" title="' + escapeHtml(t.rule_id) + '">' + escapeHtml(t.description) + "</span>"

```

```

</div>";
1213     }).join("") + "</div>"
1214     : '<div class="label" style="margin-top:10px">걸린 규칙이 없습니다 - 이 사용자의 평소 결재 패턴과 크게 다르
지 않습니다.</div>') +
1215     '<div class="label" style="margin-top:14px;color:var(--text-faint)">' +
1216     "이 판정은 옆의 탐지 이력에도 방금 기록되었습니다.</div>" +
1217     "</div>";
1218     return Promise.all([get("/risk/score/" + state.userId), get("/risk/events?user_id=" + state.userId + "&limit=1
0")]);
1219     }).then(function (r) { renderRisk(r[0], r[1]); })
1220     .catch(function () { el("assessResult").innerHTML = '<div class="err">검사에 실패했습니다. 잠시 후 다시 시도해
주세요.</div>'; })
1221     .then(function () { btn.disabled = false; btn.innerHTML = restore; });
1222 }
1223 // 이벤트 객체가 runAssess의 첫 인자로 들어가지 않도록 감싸서 넘깁니다
1224 // (그대로 넘기면 btn 자리에 MouseEvent가 들어와 classList 접근에서 터집니다).
1225 el("assessBtn").addEventListener("click", function () { runAssess(); });
1226
1227 // ----- 로딩 -----
1228 function selectUser(uid) {
1229     state.userId = uid;
1230     state.scenarioKey = "기본";
1231     state.pane = "cash";
1232     state.shield = null; state.scenarios = null; state.tax = null;
1233     state.sources = []; state.budgets = [];
1234     state.persona = state.personas.filter(function (p) { return p.user_id === uid; })[0] || null;
1235
1236     showDash();
1237     renderMainTabs();
1238     try { history.replaceState(null, "", "?user=" + uid); } catch (e) {}
1239
1240     el("heroLine").textContent = "불러오는 중...";
1241     el("heroStats").innerHTML = ""; el("todos").innerHTML = ""; el("brief").innerHTML = "";
1242     ["shieldSummary", "parking", "income", "gig", "budget", "risk", "expense"].forEach(function (id) {
1243         el(id).innerHTML = '<div class="skeleton">불러오는 중...</div>';
1244     });
1245     el("sources").innerHTML = ""; el("assessResult").innerHTML = "";
1246     el("guidance").innerHTML = '<div class="skeleton">불러오는 중...</div>';
1247     el("chartWrap").innerHTML = '<div class="skeleton">불러오는 중...</div>';
1248     el("chartLegend").innerHTML = ""; el("riskBadge").innerHTML = "";
1249     setupAssess();
1250
1251     // 요약과 할 일은 여러 API 결과를 합쳐 만들기 때문에, 각각 도착하는 대로 다시 그립니다.
1252     get("/shield/" + uid).then(function (sh) {
1253         renderShield(sh); renderHero(); renderTodos(); renderParking();
1254     }).catch(function () {
1255         fail("shieldSummary", "현금흐름 예측을 불러오지 못했습니다.");
1256         el("heroLine").textContent = "현금흐름 예측을 불러오지 못했습니다.";
1257         el("chartWrap").innerHTML = "";
1258     });
1259     get("/shield/" + uid + "/scenarios").then(function (s) { state.scenarios = s; }).catch(function () {});
1260     get("/brief/" + uid).then(renderBrief).catch(function () { el("brief").innerHTML = ""; });
1261     get("/guidance/" + uid).then(renderGuidance)
1262         .catch(function () { fail("guidance", "대응 가이드를 불러오지 못했습니다."); });
1263     get("/tax/estimate/" + uid).then(function (t) { state.tax = t; renderParking(); renderTodos(); })
1264         .catch(function () { state.tax = null; renderParking(); });
1265     loadIncome(uid).then(renderTodos);
1266     get("/gig-score/" + uid).then(renderGig).catch(function () { fail("gig", "Gig Score를 불러오지 못했습니다."); });
1267     get("/budgets/usage?user_id=" + uid + "&period=monthly").then(function (rows) {
1268         state.budgets = rows; renderBudget(rows); renderTodos();
1269     }).catch(function () { fail("budget", "예산 정보를 불러오지 못했습니다."); });
1270     Promise.all([get("/risk/score/" + uid), get("/risk/events?user_id=" + uid + "&limit=10")])
1271         .then(function (r) { renderRisk(r[0], r[1]); }).catch(function () { fail("risk", "이상거래 정보를 불러오지 못했
습니다."); });
1272     get("/expense/classify?user_id=" + uid + "&limit=20").then(renderExpense)
1273         .catch(function () { fail("expense", "경비 분류를 불러오지 못했습니다."); });
1274 }
1275

```

```
1276 get("/demo/personas").then(function (d) {
1277     state.personas = d.personas || [];
1278     el("disclaimer").textContent = d.note || "";
1279     renderPersonaGrid();
1280     var q = new URLSearchParams(location.search).get("user");
1281     var ids = state.personas.map(function (p) { return p.user_id; });
1282     if (q && ids.indexOf(q) >= 0) selectUser(q); // 링크로 특정 인물을 바로 열 수 있게
1283 }).catch(function () {
1284     el("personaGrid").innerHTML = '<div class="err">데모 데이터를 불러오지 못했습니다. 잠시 후 새로고침해 주세요.</div>';
1285 });
1286 })();
1287 </script>
1288
```